

ELK

Indexation, recherche et visualisation de données

Vincent LE ROUX



Contenu du cours

JOUR 1 : Installation, architecture, concepts

- Présentation de la stack Elastic
- Architecture de la stack (on-premise, cloud)
- Installation de la stack (Docker)
- Configuration / paramétrage des briques
- Mise à l'échelle de la solution / stress tests
- Administration de la solution avec Kibana
- Monitoring / surveillance de la solution
- Etude de dimensionnement (sharding / replication)
- Présentation d'Apache Lucene
- Principe d'indexation de contenu
- Le scoring et la pertinence des recherches

JOUR 2 : Usages de la stack

- Les cas d'usages d'une stack ELK
- Découverte du langage DSL : pour réaliser des recherches
- Les modes avancés des recherches (boost, surlignage, etc.)
- Introduction à Kibana
- Le principe d'index Pattern
- Le menu **Discover** : découvrir ses données
- Le menu **Visualize** : pour créer ses visualisations unitaires
- Le menu **Dashboards** : pour consolider les visualisations
- Graphes et Machine Learning
- Ouverture vers X-Pack
- ILM : Gestion de la durée de rétention



Objectifs pédagogiques

Objectifs pédagogiques

- Comprendre les principes clés d'installation et de configuration d'Elasticsearch, Logstash et Kibana
- Évaluer les possibilités offertes par Elasticsearch, Logstash et Kibana
- Savoir utiliser Elasticsearch, Logstash et Kibana pour indexer, chercher et visualiser des données et des documents
- Découvrir les principales approches de développement d'applications

Prérequis :

- Connaissances de base en développement et en administration du système d'exploitation Windows ou Linux/Unix



Demat!

Je m’appelle Vincent LE ROUX

Je suis Consultant Décisionnel depuis 2008 mais pas seulement… Je suis également Data Engineer, Data Scientist, Manager, Formateur, (breton)…

Vous me trouverez facilement sur **LinkedIn**



Mon travail au quotidien

Réalisation d'études de choix d'outils : En BI ou en Big Data, vers quelle solution me tourner, en fonction de mes cas d'usages et de ma culture d'entreprise?

Architecte Data : Comment intégrer une solution dans mon SI? On-premise ou Cloud (ou hybride)?

Consultant Data :Mise en oeuvre de la solution, modélisation de données

Formateur : Formateur sur des sujets décisionnels et Big Data



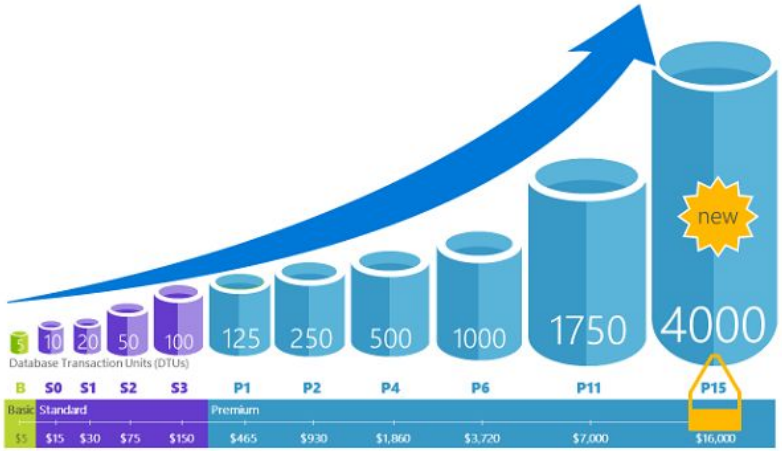
1. LE NOSQL

Brève introduction avant Elasticsearch

LES LIMITES DES SGBDR

Les SGBD actuels ont des limites :

- **Performance** : les requêtes SQL ne sont plus performantes lorsque le SGBD est trop volumineux (Les jointures sont coûteuses)
- **Scalabilité** : augmentation de la RAM, de la CPU
- **Schémas figés** : une fois que le schéma de BDD a été créé, implémenter des modifications devient plus difficile
- **Format des données** : les données respectent strictement le schéma





QUESTIONS

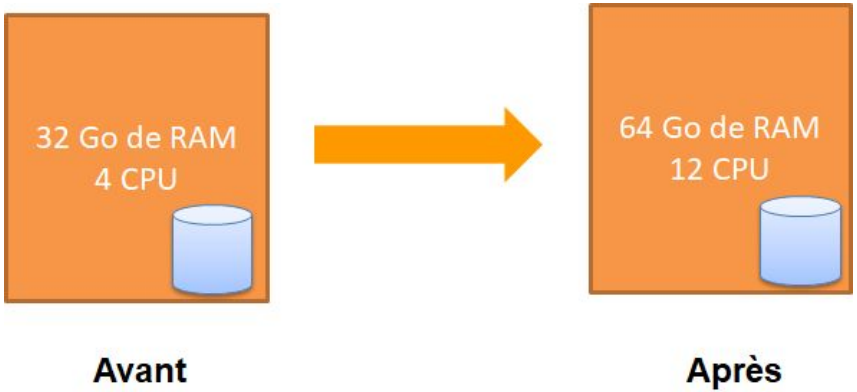


Répondez aux questions suivantes :

- Comment les bases NOSQL garantissent des temps de réponses optimisés?
- Comment les bases NOSQL permettent le stockage de données hétérogènes?

SCALABILITE VERTICALE

Scalabilité verticale (Mise à l'échelle) : Pas adapté pour le Big Data!



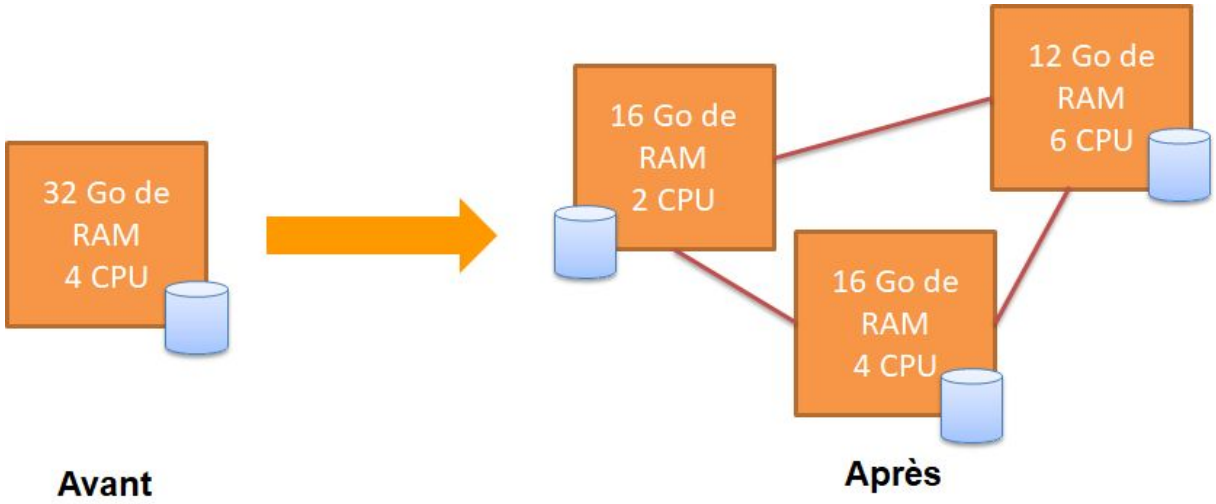
RAM : Mémoire vive

CPU : Unité de calcul

Limites : A un moment donné, l'ajout de CPU et RAM ne permet plus de faire des gains en termes de performance

SCALABILITE HORIZONTALE

Scalabilité horizontale : Parfait pour le Big Data!



Intérêts :

- Les données sont distribuées (Bonne tolérance à la panne)
- Les calculs sont distribués (Tous les nœuds du cluster travaillent pour avoir une réponse rapide)

MOUVEMENT NOSQL



Pour contourner les limites des **SGBDR**, apparition du mouvement **NOSQL** (**N**ot **O**nly **SQL**).

Attention : le NOSQL n'est pas un langage d'interrogation de base de données, ni même l'absence de SQL!

Il s'agit d'une famille de bases de données que l'on peut interroger avec d'autres biais que le SQL.

Exemples :

- Elasticsearch ou CouchDB se requêtent via HTTP
- MongoDB se requête en JavaScript
- ...



MOUVEMENT NOSQL

Les bases de données NOSQL fonctionnent généralement en mode **distribué** (Plusieurs noeuds forment un cluster). La donnée est alors **répliquée** sur les noeud, de deux manières distinctes :

- **Synchrone** : tous les noeuds voient la même donnée au même moment. En cas de mise à jour / ajout / suppression, il peut y avoir un délai de latence avant que la donnée soit répliquée partout. Idéal quand il est impératif que la donnée soit tout le temps la même partout
- **Asynchrone** : les noeuds ne voient pas forcément la même donnée au même moment. Idéal quand il est impératif que les temps de réponses soient très rapides (quitte à ce que tout le monde n'ait pas forcément le même résultat)

MOUVEMENT NOSQL

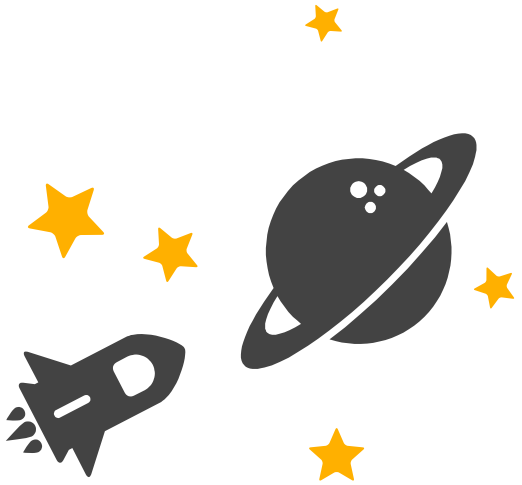


Les bases de données NOSQL sont classées en 4 types de bases :

- **Clé-valeur (Key-value)** : Redis, Riak, Oracle Berkely DB, Amazon SimpleDB, Apache Ignite, etc.
- **Orientée colonne (Column-oriented)** : HBase, Apache Parquet, Google BigTable, Apache Cassandra, etc.
- **Orientée document (Document)** : MongoDB, CouchDB, Elasticsearch, etc.
- **Graphe (Graph)** : Neo4j, etc.



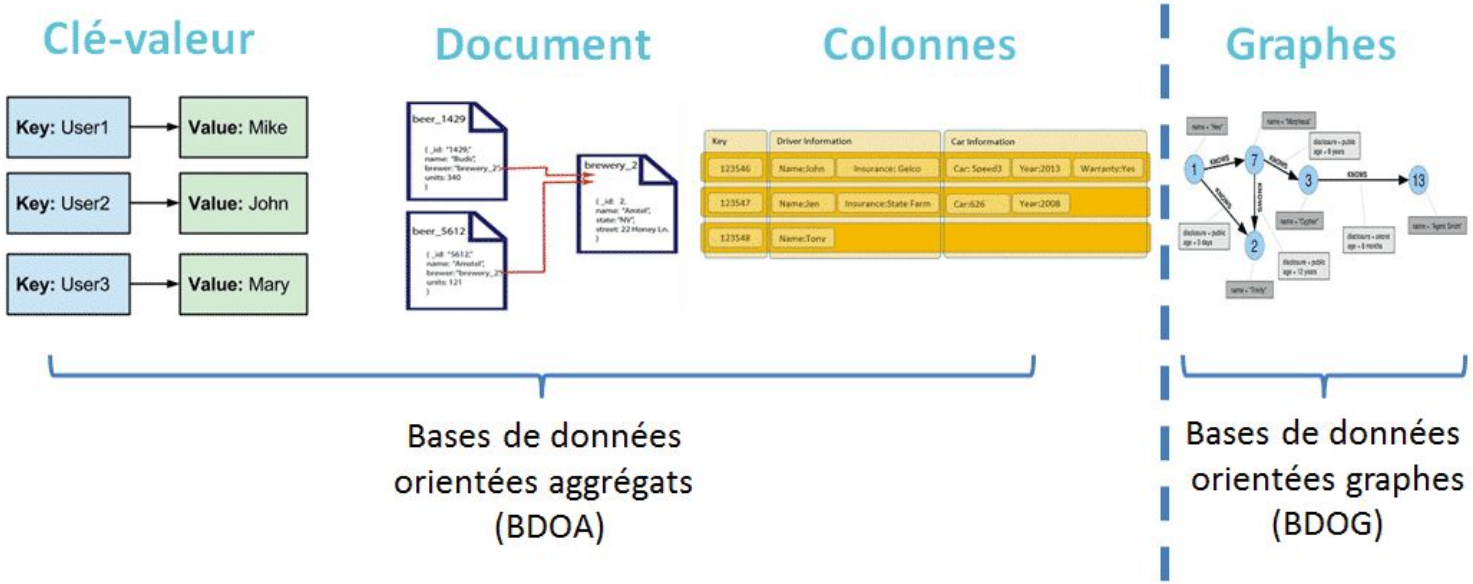
EXERCICE



Préparer une analyse plus complète des 4 types de bases de données citées ci-avant

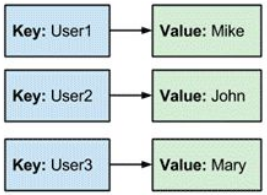
FAMILLES DE BASES DE DONNÉES NOSQL

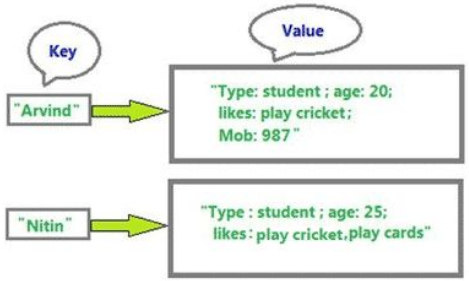
Les bases de données NOSQL sont classées en 4 types de bases :



CLÉ-VALEUR

Les données sont stockées en clé-valeur : une clé plus un BLOB (dans lequel on peut mettre : nombre, date, texte, XML, photo, vidéo, structure objet) :





Facilement scalable

Temps de réponse en écriture / lecture très bas

Mises à jour compliquées

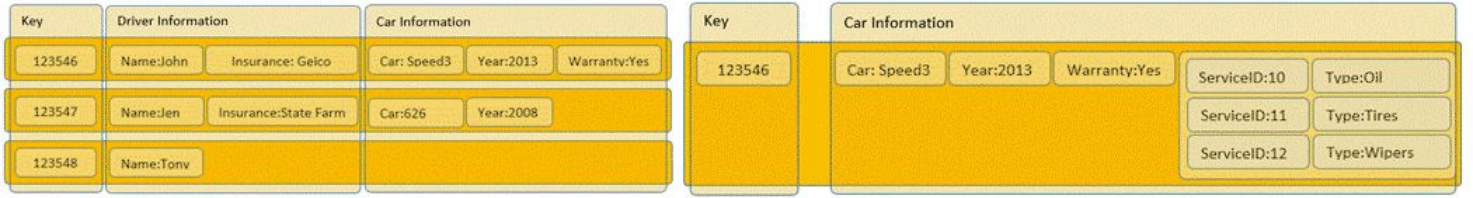
Requêtes rudimentaires

Exemples d'implémentation

Amazon DynamoDB (Beta) ORACLE BERKELEY DB 11g redis

ORIENTÉ COLONNES

Ces bases de données se rapprochent des bases de données relationnelles, à ceci près qu'elles permettent de remplir un nombre de colonnes variables :



Key	Name	Insurance	Car	Year	Warranty
123456	John	Geico	Speed3	2013	Yes
123457	Jen	State Farm	626	2008	NULL
123458	Tony	NULL	NULL	NULL	NULL

ServiceID	Type	Key
10	Oil	123456
11	Tires	123456
12	Wipers	12345

+

Capacité de stockage accrue

Accès rapide aux données

−

Efficace surtout pour des données de même type et similaires

Requêtage limité

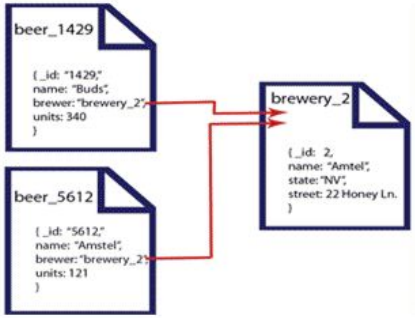
Exemples d'implémentation

APACHE HBASE

cassandra

ORIENTÉ DOCUMENTS

Ces bases de données stockent des données semi-structurées : le contenu est formaté JSON ou XML, mais la structure n'est pas contrainte :



+

Requétage plus complet

Flexibilité

Evolutif au cours du temps

−

Duplication des données

Cohérence pas forcément assurée

Exemples d'implémentation

CouchDB relax

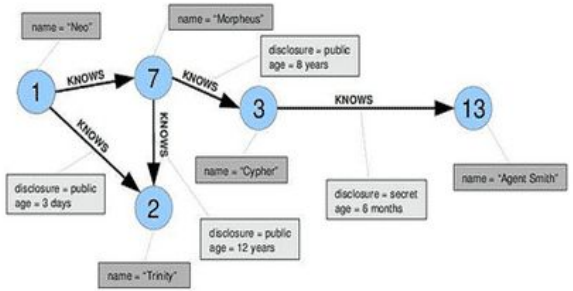
mongoDB

terrastore

elasticsearch

ORIENTÉ GRAPHES

Ces bases de données, basées sur la théorie des graphes, sont gérées par noeuds, relations et propriétés. Elles gèrent des données spatiales, sociales ou financières (dépôts / retraits) :



Adapté à la gestion de données relationnelles

Architecture modelable

Architecture limitée à certains cas

Exemple d'implémentation

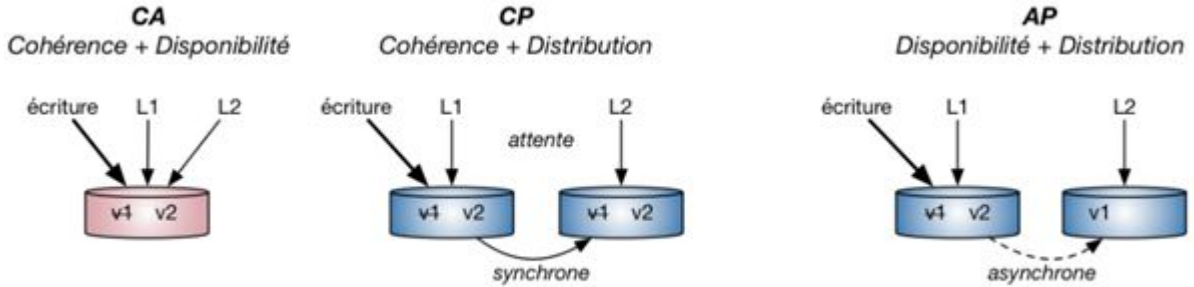
TITAN

Neo4j

THEOREME DE CAP

En 2000, proposition d'un théorème pour caractériser les bases de données SQL et NOSQL :

- **C** (Consistency = Cohérence) : une donnée n'a qu'un seul état visible, quel que soit le nombre de réplicas
- **A** (Availability = Disponibilité) : Les clients doivent pouvoir lire et écrire en permanence
- **P** (Partition Tolerance = Donnée distribuée) : le système est distribué et la donnée est répliquée



CA - AP - CP



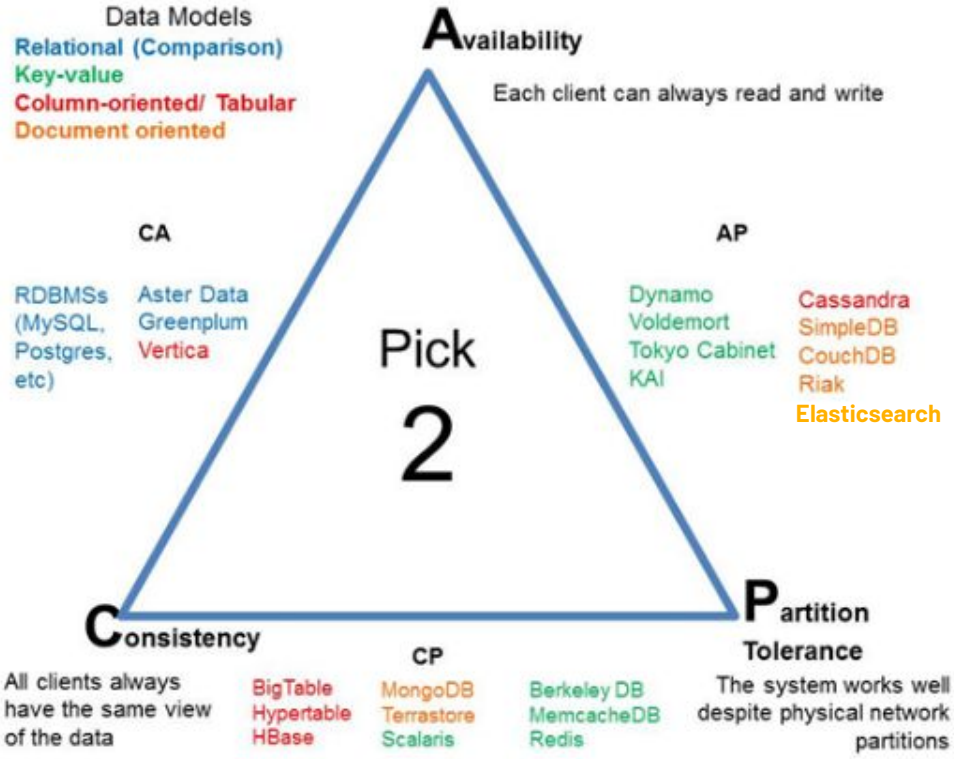
THEOREME DE CAP

3 couples possibles :

- **CA (Consistency-Availability)** :
  - Opérations concurrentes sur la même donnée
  - Envoi des données sans délai d'attente
  - Possible uniquement avec les SGBDR : pas de distribution
- **CP (Consistency-Partition Tolerance)** :
  - Donnée répliquée, autorisant la tolérance à la panne
  - La donnée doit être cohérente : protocole synchrone de réplication
  - Latence dans les temps de retour de la dernière version des données
- **AP (Availability-Partition Tolerance)** :
  - Donnée répliquée, autorisant la tolérance à la panne
  - Accès très rapide à la donnée
  - Protocole de mise à jour asynchrone des réplicats
  - Deux requêtes ne renvoient pas forcément le même résultat

TRIANGLE DE CAP

Des éléments vu ci-avant, il devient possible de choisir une base de données NOSQL en fonction de ces caractéristiques :

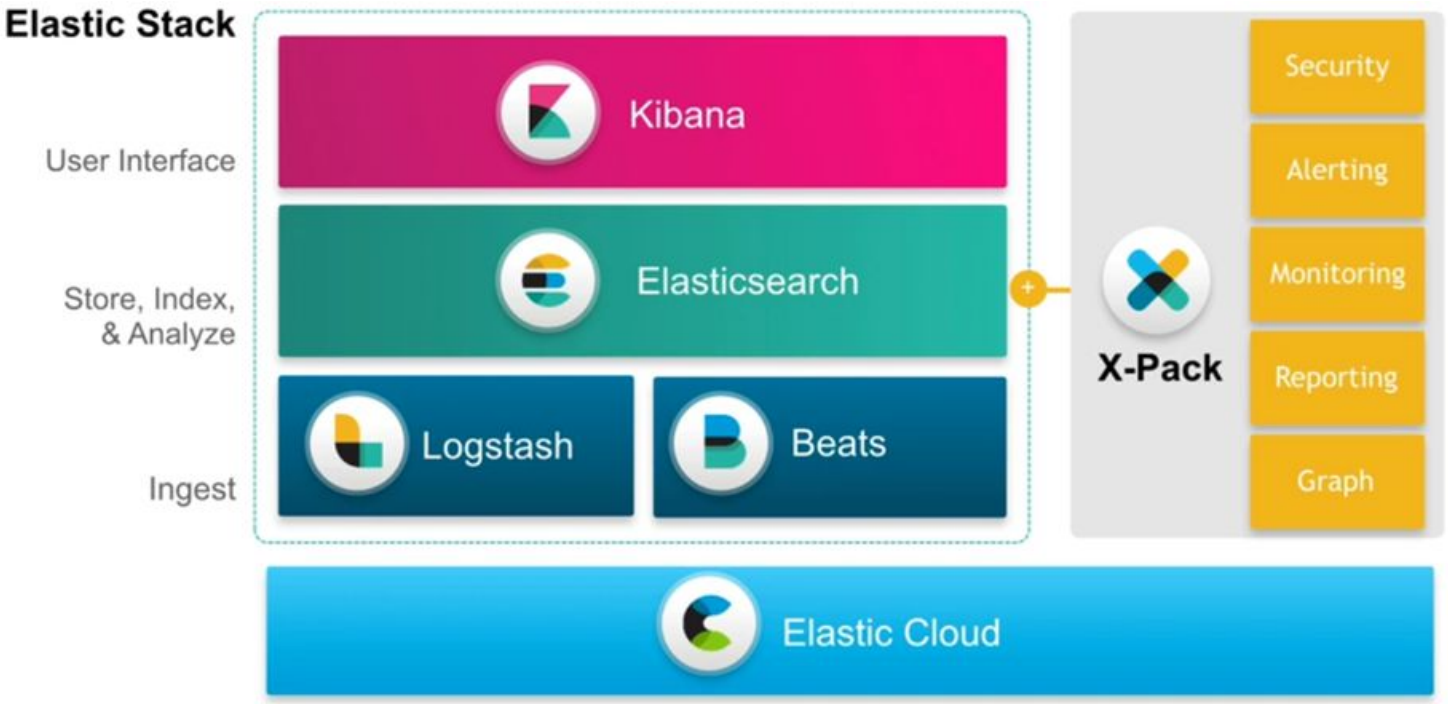




2. STACK ELK

Présentation de la stack Elastic

STACK ELK


Elastic Stack
User Interface
Store, Index, & Analyze
Ingest
Kibana
Elasticsearch
Logstash
Beats
X-Pack
Security
Alerting
Monitoring
Reporting
Graph
Elastic Cloud

STACK ELK

ELK =



- **Elasticsearch** Le moteur de recherche et stockage de la donnée
- **Logstash** L'ETL qui permet d'alimenter Elasticsearch
- **Kibana** Le module web de visualisation de la données

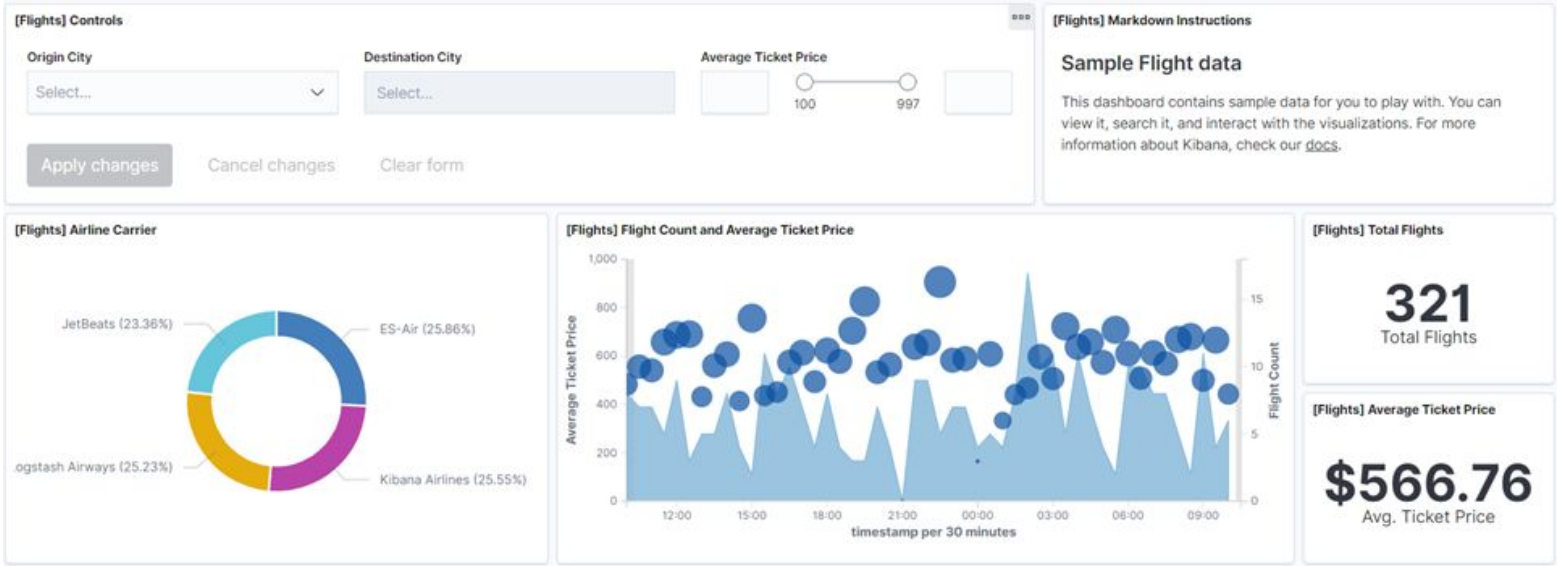
Cas d'usages :

- Exploitation / analyse de fichiers de logs volumineux
- Suivi d'un fil d'actualité Twitter
- Module de recherche dans une application
- Analyse de métriques (machines industrielles par exemple / RAM / CPU…)

Pourquoi choisir Elasticsearch?

- Grande communauté
- Très scalable (De type horizontal)
- Système flexible
- Aucune contrainte de schéma (données non structurées mais pas seulement)
- Temps réel (Délai de 1 seconde dans Kibana après indexation)

EXEMPLE DE RENDU GRAPHIQUE



VOCABULAIRE



- **Cluster :** Architecture Elasticsearch constituée de plusieurs nœuds (node)
- **Node :** Représente un serveur physique / virtuel / conteneur au sein du cluster
- **Index :** Représente une collection de documents de la même famille (équivalent d'une collection de tables SQL)
- **Type :** Détermine les champ disponibles dans l'index **(déprécié en version 7)**
- **Document :** Représente une donnée stockée dans un index (équivalent d'une ligne de table SQL)
- **Lucene :** Bibliothèque (Framework) écrit en Java qui permet d'indexer et de chercher du texte. Proposer sous licence Apache.
- **Shards :** Nombre d'instances Lucene qui travaillent sur un index particulier. **Par défaut : 1**
- **Réplicat :** Nombre de fois où la donnée est répliquée sur le cluster. **Par défaut : 1**



3. INDEX

Comprendre où et comment est stockée la donnée



AU DEBUT ETAIT LUCENE

Apache Lucene est un framework OpenSource imaginé par Doug Cutting à la fin des années 1990.

A l'époque, le web était en pleine expansion et il commençait à devenir compliqué de réaliser des recherches pertinentes, tant les volumes grandissaient. En effet, à l'époque, chercher un mot revenait à **scanner chaque mot de manière séquentielle dans une phrase, ce qui n'est pas du tout optimisé**.

Doug Cutting a donc reprit un concept que l'on retrouve encore dans nos livres d'aujourd'hui : **l'index**. L'index d'un livre permet de retrouver plus facilement des mots ou des concepts dans un ouvrage. C'est le même concept qui a été mis en œuvre avec Lucene.



QU'EST CE QU'UN INDEX?

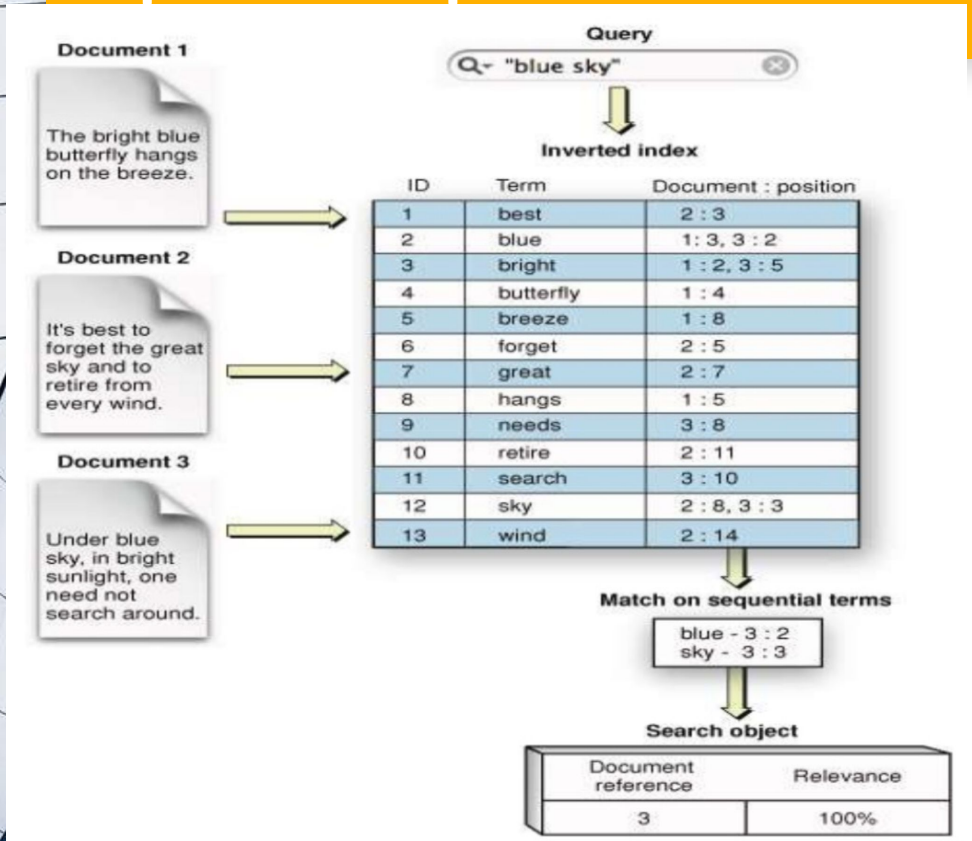
Comme vu dans le chapitre NOSQL, Elasticsearch est en vérité une base de données orientée documents.

Cela veut donc dire qu'elle stocke des documents au format JSON, comme MongoDB ou CouchDB par exemple.

Cependant, il y a une grande différence entre ces différents système de stockage de données : Dans le cas d'Elasticsearch (et donc d'Apache Lucène derrière), à chaque fois qu'un nouveau document est injecté dans la base, il est également **indexé**.

Chaque mot du document vient compléter un index inversé qui servira ensuite à accélérer les recherches de mots dans des phrases. On parle de recherche **Full-Text**.

QU'EST CE QU'UN INDEX?



Credit: https://developer.apple.com/library/mac/documentation/userexperience/conceptual/SearchKitConcepts/searchKit_basics/searchKit_basics.html

Ce schéma illustre parfaitement bien un index : les documents JSON sont découpés et chaque occurrence de terme (mot) est stockée dans un "index inversé".

De cette manière, la recherche de donnée est très aisée et très rapide.

Grâce à l'index inversé, le moteur Lucène parvient très vite à retrouver l'intégralité des documents associés au terme recherché.

LES PROPRIÉTÉS D'UN INDEX



Un index peut posséder plusieurs propriétés mais deux essentielles vont nous intéresser :

- **Number_of_shards** : Il s'agit du nombre d'instances Lucène allouées à un index. Par défaut, la valeur est de 1. Dans certains cas, il peut être utile d'augmenter cette valeur. Attention cependant : vous ne pouvez pas modifier cette valeur à chaud sur un index en lecture / écriture! Si vous souhaitez augmenter le nombre de shards pour des index existants, vous serez obligés de créer un nouvel index vierge avec le bon nombre de shards, puis de transférer les données de l'ancien vers le nouvel index ou de le passer en mode lecture seule au préalable (pour pouvoir utiliser `_split` ou `_shrink`)

- **Number_of_replicas** : Il s'agit du nombre de fois où le shard est répliqué sur le cluster. Par défaut, la valeur est positionnée à 1. Si jamais le nombre de réplicas ne paraît pas suffisant, il est toujours possible d'augmenter, à chaud, le nombre. **Attention** : plus on augmente et plus on accroît les performances (en lecture) mais plus on occupe de place et plus on consomme en écriture...

Liste des autres propriétés :

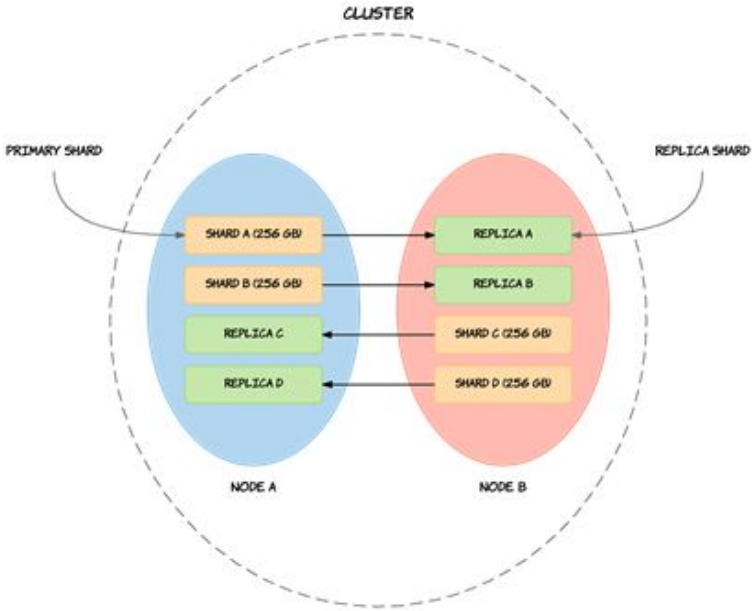
https://www.elastic.co/guide/en/elasticsearch/reference/current/index-modules.html

LE SHARDING

Un index est découpé en shards, répartis sur le cluster. Chaque document est stocké entier dans un seul shard, choisi par `hash( id) % nombre de shards primaires`. La réplication copie chaque shard primaire sur un autre nœud.

Elasticsearch repose sur ces deux concepts clefs pour :

- Garantir des temps de réponse optimisés en lecture sur les données
- Proposer une très haute tolérance aux pannes





LES TYPES DE DONNÉES

Si on ne lui précise rien du tout, Elasticsearch, au moment de l'indexation de données, parvient à "deviner" le type des données et applique un type.

Voici les types de données classiques :

- Binary / Boolean
- Keyword : champ agrégable (très utile dans Kibana)
- Long, double
- Date
- Object : un document JSON
- Flattened : un document JSON en un seul champ (mis à plat)
- Nested : un document JSON qui préserve la relation entre les sous-champs
- Range
- Ip
- Text : pour stocker du contenu qui sera découpé pour de la recherche full text
- Geo_point : Coordonnée GPS
- Geo_shape : Polygone
- Semantic_text : Recherche par sens plutôt que par mot

KEYWORD vs TEXT



Sur des champs de type texte, deux choix peuvent se poser : **Keyword** ou **Text**

- **Text** : Un champ text est analysé (Analyzer), c'est à dire qu'il est découpé en plusieurs mots, ce qui permet de réaliser des recherches full text. Idéal pour des phrases!

- **Keyword** : Un champ keyword prend aussi du texte en entrée mais ce dernier est conservé intégralement (sans être découpé en mots). Il peut servir pour réaliser des agrégations de données et il est possible de réaliser des recherches basées sur des caractères spéciaux (wildcard). Idéal pour des mots seuls!

KEYWORD vs TEXT – Synthèse



	`text`	`keyword`
Traitement à l'indexation	Analysé : découpé en mots, **passé en minuscules** par l'analyzer standard	Conservé **tel quel, à l'octet près**
Sensible à la casse ?	**Non** par défaut (`"Elasticsearch"` et `"elasticsearch"` matchent la même recherche)	**Oui** par défaut (`"Paris"`, `"paris"` et `"PARIS"` sont trois valeurs différentes)
Agrégable (`terms`, tri...)	Non directement (il faudrait activer `fielddata`, déconseillé et coûteux)	Oui, nativement — c'est fait pour ça
Recherche par phrase / mots proches	Oui (`match_phrase`, proximité, synonymes...)	Non — recherche exacte uniquement
Wildcard / recherche par motif	Limité	Oui, efficace (`Boots*`, etc.)
Exemple d'usage	Description produit, corps d'un ticket, message de log	Catégorie, statut, code pays, adresse email, référence produit

LE MAPPING

Il peut cependant arriver qu'Elasticsearch devine mal le type de données.

Par exemple, si on tente d'insérer une paire de points GPS : [-1.74699007296, 48.0591504968]

L'index va deviner le type de données suivant :

```
# Recuperation du mapping "auto detecte" par Elasticsearch :
GET /trafic_rennes/_mapping
# Les types de donnees ne sont pas coherents...
```

```
"geo_point_2d" : {
  "type" : "float"
},
"geo_shape" : {
  "properties" : {
    "coordinates" : {
      "type" : "float"
    },
    "type" : {
      "type" : "text",
      "fields" : {
        "keyword" : {
          "type" : "keyword",
          "ignore_above" : 256
        }
      }
    }
  }
},
```

LE MAPPING

Il peut cependant arriver qu'Elasticsearch devine mal le type de données.

Pour éviter ce genre de désagréments, il peut être utile de créer un mapping :

```
PUT /trafic_rennes_dupl
{
 "mappings": {
  "properties": {
   "geo_point_2d" : {"type": "geo_point"},
   "geo_shape" : {"type": "geo_shape"}
  }
 }
}
```

Ici, deux champs sont créés (geo_point_2d et geo_shape) avec leurs types respectifs (geo_point et geo_shape). De cette manière, chaque donnée insérée possédera le bon type.

LES TEMPLATES

Il est possible de créer des templates, pour qu'à chaque nouvel index créé, un certain nombre de paramètres soient automatiquement intégrés :

```
PUT _template/snmp
{
  "index_patterns": ["snmp-*"],
  "settings": {
    "number_of_replicas": 1,
    "number_of_shards": 3
  },
  "mappings": {
    "properties": {
      "gps": {"type": "geo_point"}
    }
  }
}
```

Ici, par exemple, sur chaque nouvel index commençant par "**snmp-**" seront attribués un nombre de réplicas et un nombre de shards.

Egalement, un mapping est mis en place pour un type de données précis.

LES TEMPLATES COMPOSABLES

Depuis Elasticsearch **7.8 (juin 2020)**, l'API _template est remplacée par les templates composables : des **component templates** réutilisables, assemblés dans un **index template** :

```
PUT _component_template/snmp-settings
{ "template": { "settings": {
    "number_of_shards": 1,
    "index.lifecycle.name": "snmp-policy" } } }

PUT _component_template/snmp-mappings
{ "template": { "mappings": { "properties": {
    "host.ip":          { "type": "ip" },
    "gps":              { "type": "geo_point" },
    "if.in_octets":     { "type": "long" } } } } }

PUT _index_template/snmp
{ "index_patterns": ["snmp-*"],
  "data_stream": {},
  "priority": 500,
  "composed_of": ["ecs@mappings", "snmp-settings", "snmp-mappings"] }
```

Avantages

- **Réutilisable :** un même component template sert à plusieurs index templates
- **Prévisible :** un seul template s'applique, celui de plus haute priority (fini la fusion par order)
- **Prioritaire :** l'emporte sur les _template legacy, dépréciés mais encore acceptés en 8.x et 9.x

Data_stream *: On écrit dans un nom unique, Elasticsearch crée et fait tourner les index en coulisses.*

MISE A JOUR DES TEMPLATES



Points d'attention lors de la modification d'un template :

- **Nouveaux index uniquement :** les index existants gardent leur configuration (passer par _settings, un rollover ou un _reindex)
- **Remplacement complet :** un PUT écrase tout le template, il faut renvoyer toutes les sections
- **Effet en cascade :** modifier un component template impacte tous les index templates qui l'utilisent
- **Conflits de priorité :** deux templates de même priority sur des patterns qui se chevauchent sont refusés ; viser >500 pour passer devant les templates intégrés (Elastic Agent, Beats)

```
# Vérifier le template qui sera appliqué, avant de créer l'index
POST _index_template/_simulate_index/snmp-routeurs
```



4. ELASTICSEARCH

Architecture, installation, paramétrage, mise à l'échelle



INSTALLATION DE LA STACK

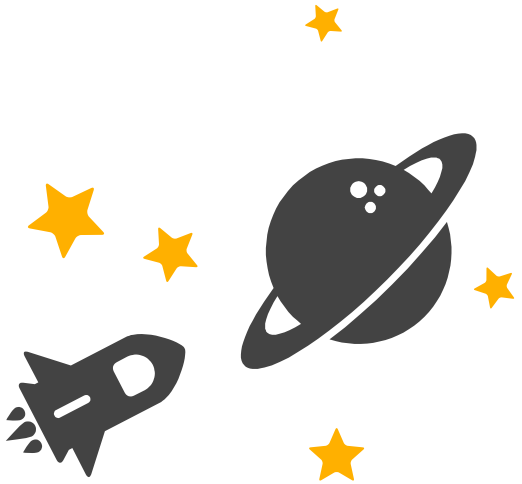
Plusieurs façons d'installer / utiliser la stack ELK :

- **Cloud**
  - Hébergé directement sur les serveurs d'Elastic (Essai gratuit de 14 jours)
  - Sur un cloud public (AWS / GCP / Microsoft)
- **On-Premise**
  - Sur Windows / Linux / MacOs : Téléchargement des sources, à dézipper et à exécuter (Attention, une JRE est indispensable)
  - Sur Linux / Unix : Installation à l'aide d'un gestionnaire de paquets (yum par exemple)
  - Docker : Elastic met à disposition des images Docker pour chacun de ses différents produits → Facilité de déploiement en mode cluster / administration

Dans le cadre de cette formation, nous mettrons en place une installation à l'aide de **Docker**.



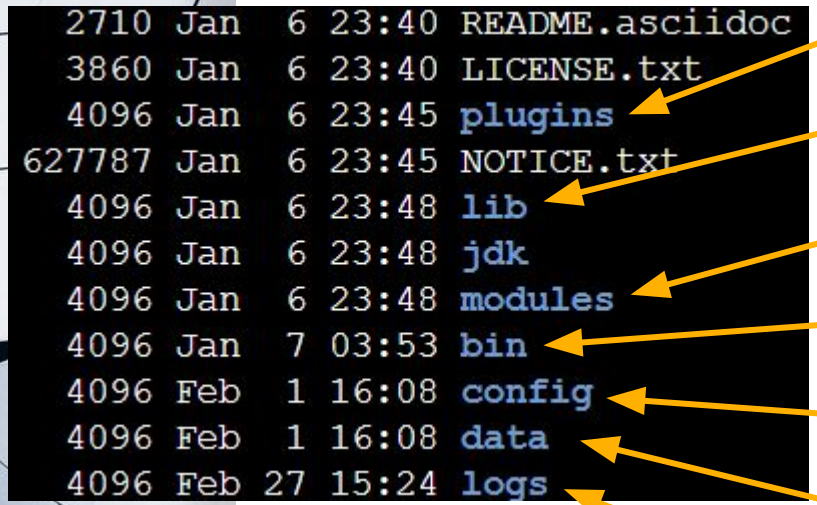
EXERCICE



Installation d’un cluster d’un noeud (Mode Single-Node)

ELASTICSEARCH : CONFIGURATION

Une fois Elasticsearch installé, voici l'arborescence typique du produit sur un noeud :



- Modules complémentaires (vide au début)
- Librairies / Dépendances Java (ne pas toucher)
- Modules installés par défaut avec Elasticsearch
- Exécutable Elasticsearch et installateur de plugins
- Fichiers de configuration du noeud
- Données stockées sur le noeud
- Fichier de logs du cluster / noeud

ELASTICSEARCH.YML

Le fichier config/elasticsearch.yml est le fichier de configuration permettant de paramétrer le noeud.

Plusieurs chapitres. Options de cluster / Noeud :

```
---------------------------------- Cluster -----------------------------------

Use a descriptive name for your cluster:

cluster.name: my-application

------------------------------------ Node ------------------------------------

Use a descriptive name for the node:

node.name: node-1

Add custom attributes to the node:

node.attr.rack: r1
```

Il est possible de spécifier un nom précis pour le cluster ou pour le noeud.

Si ce n'est pas fait, un nom est choisi au hasard pour le nom du noeud.

ELASTICSEARCH.YML

Le fichier config/elasticsearch.yml est le fichier de configuration permettant de paramétrer le noeud.

Plusieurs chapitres. Options de chemins :

```
# ---------------------------------- Paths ------------------------------------
#
# Path to directory where to store the data
# (separate multiple locations by comma):
#
path.data: /path/to/data
#
# Path to log files:
#
path.logs: /path/to/logs
```

Il est possible de changer l'emplacement où seront stockés les données (**path.data**) et les journaux de log (**path.logs**)

ELASTICSEARCH.YML

Le fichier config/elasticsearch.yml est le fichier de configuration permettant de paramétrer le noeud.

Plusieurs chapitres. Options de mémoire :

```
################################### Memory ####################################

Lock the memory on startup:

bootstrap.memory_lock: true

Make sure that the heap size is set to about half the memory available
on the system and that the owner of the process is allowed to use this
limit.

Elasticsearch performs poorly when the system is swapping the memory.
```

Il est possible d'affiner la mémoire qui sera utilisée par Java

ELASTICSEARCH.YML

Le fichier config/elasticsearch.yml est le fichier de configuration permettant de paramétrer le noeud.

Plusieurs chapitres. Options de réseau :

```
---------------------------------- Network -----------------------------------

Set the bind address to a specific IP (IPv4 or IPv6):

network.host: 192.168.0.1

Set a custom port for HTTP:

http.port: 9200

For more information, consult the network module documentation.
```

Par défaut, un noeud Elasticsearch n'est pas visible depuis l'extérieur. Il convient alors de toucher au paramètre **network.host** pour le rendre visible

Le port par défaut d'écoute est le 9200 sur du http

ELASTICSEARCH.YML

Le fichier config/elasticsearch.yml est le fichier de configuration permettant de paramétrer le noeud.

Plusieurs chapitres. Options de mise en cluster :

```
---------------------------------- Discovery -----------------------------------

Pass an initial list of hosts to perform discovery
 when this node is started:
The default list of hosts is ["127.0.0.1", "[::1]"]

iscovery.seed_hosts: ["host1", "host2"]

Bootstrap the cluster using an initial set of master-eligible nodes:

luster.initial_master_nodes: ["node-1", "node-2"]

For more information, consult the discovery and cluster
 formation module documentation.
```

Dès lors que l'on souhaite créer un cluster, il sera nécessaire de toucher aux paramètres ***discovery.seed_hosts*** et ***cluster.initial_master_nodes***.

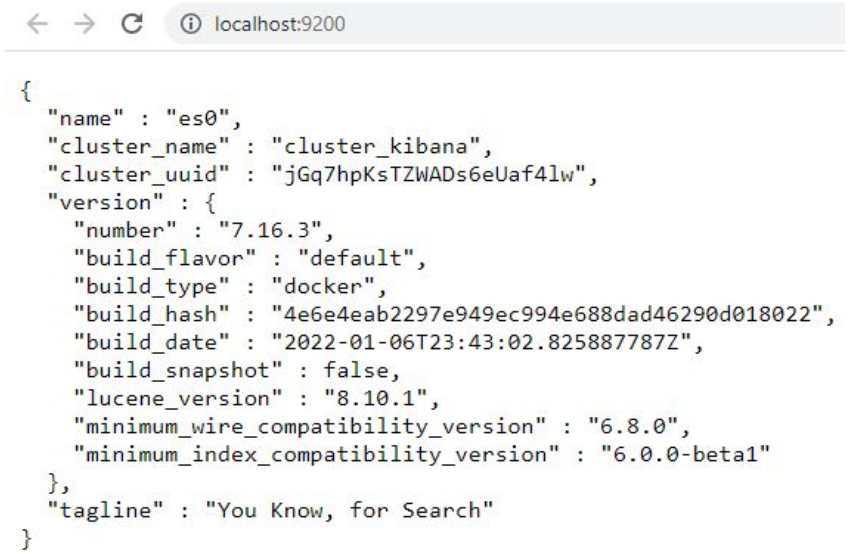
Voir ci-après : ***Mise en cluster***

LANCEMENT D'ELASTICSEARCH

Se positionner dans le répertoire et lancer la commande suivante :

```
bin/elasticsearch
```

Pour vérifier que tout fonctionner, tenter d'accéder à l'URL http://<IP>:9200

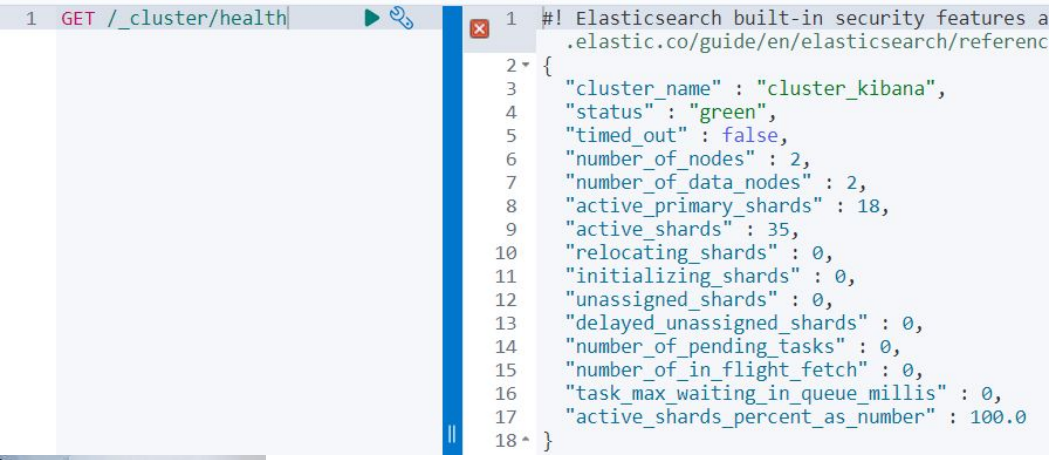




5. LES API

Connaître les API utiles à Elasticsearch

_cluster/health



État de santé global du cluster :

Les 3 niveaux de santé :

- **Green** : Les données sont accessibles et sécurisées. Cela veut dire que tous les shards ont été correctement répliqués sur plusieurs nœuds (en fonction du nombre de réplicas choisis)
- **Yellow** : Les données sont accessibles mais non sécurisées : des shards ne sont pas répliqués et il peut y avoir un risque de perte de données, si un nœud venait à couper. Pour voir les shards non assignés, se référer à la valeur « **unassigned_shards** »
- **Red** : La donnée n'est plus accessible : tous les nœuds sur laquelle les shards avaient été alloué sont hors service

_cat/nodes

Cet appel API permet d'avoir la liste des nœuds qui composent le cluster, de voir les rôles attribués à chaque nœud, ainsi que d'identifier qui est le master du cluster :

```
GET /_cat/nodes?v
```

```
#! Elasticsearch built-in security features are not enabled. Without authentication, your
  anyone. See https://www.elastic.co/guide/en/elasticsearch/reference/7.16/security-minir
ip         heap.percent ram.percent cpu load_1m load_5m load_15m node.role   master name
172.29.0.2           63          48   5    0.06    0.06     0.10 cdfhilmrstw -      es0
172.29.0.4           57          48   4    0.06    0.06     0.10 cdfhilmrstw *      es1
```

Ici, c'est le noeud es1 qui est le master (*) du cluster.

Il est aussi possible de voir les rôles de chaque noeud (voir chapitre suivant)

_cat/indices

L'API _cat/indices permet de lister tous les index de la plateforme :

```
GET /_cat/indices?v
```

```
#! Elasticsearch built-in security features are not enabled. Without authentication, your cluster could be accessible to anyone
  .elastic.co/guide/en/elasticsearch/reference/7.16/security-minimal-setup.html to enable security.
health status index                           uuid                   pri rep docs.count docs.deleted store.size pri.store.size
green  open   .geoip_databases                 b2cXaBH5Qi-e6m-rnh2ELA   1   1         45            4     88.2mb         45.7mb
green  open   .kibana_7.16.3_001               8sw4i5ZcQyegtBhPv-O7Ag   1   1        235            3      6.8mb          3.4mb
green  open   twitter_sentiment               oglJXDH5SN6lQubm4RMuKg   1   1       1917            0      2.4mb       1006.7kb
green  open   library                         SpZc2KIDSga_yiIw-NTtMA   1   0          5            0      5.7kb          5.7kb
green  open   .apm-custom-link                27vZiGJTR_y41aDg2XQUgQ   1   1          0            0       452b           226b
green  open   kibana_sample_data_ecommerce    KuVflRngRJGcl_BokBFujA   1   1       4675            0      8.9mb          4.4mb
green  open   .apm-agent-configuration        FOmfSHMQRlqklOsXKzlVOw   1   1          0            0       452b           226b
green  open   .kibana_task_manager_7.16.3_001 ItiBMpAYShuFSMfsmCVD2g   1   1         17        20703      5.2mb          2.8mb
green  open   kibana_sample_data_logs         yDA41mZYTuOsXCLFHmQ9Ew   1   1      14074            0     15.4mb          7.6mb
green  open   .async-search                   sr8VZgh_Q6C4CV61eoJjhQ   1   1          0            0      510b           255b
green  open   kibana_sample_data_flights      VzrTxm_FQ3y29-H8MTAuiA   1   1      13059            0     12.1mb           6mb
green  open   .tasks                          93e7PaNySe-xkDPDnoXJdg   1   1         12            0     92.9kb         70.1kb
```

Cela vous permet notamment d'obtenir :

- L'état de santé de vos index
- La taille qu'occupe l'index sur le cluster

<INDEX>/_settings

Cette API permet de voir les propriétés d'un index, notamment le nombre de réplicas et le nombre de shards :

```
GET twitter_sentiment/_settings
```


```
#! Elasticsearch built-in security features are nc
  ://www.elastic.co/guide/en/elasticsearch/referer
{
  "twitter_sentiment" : {
    "settings" : {
      "index" : {
        "routing" : {
          "allocation" : {
            "include" : {
              "_tier_preference" : "data_content"
            }
          }
        },
        "number_of_shards" : "1",
        "provided_name" : "twitter_sentiment",
        "creation_date" : "1643731934107",
        "number_of_replicas" : "1",
        "uuid" : "oglJXDH5SN6lQubm4RMuKg",
        "version" : {
          "created" : "7160399"
        }
      }
    }
  }
}
```

_cat/shards

L'API _cat/shards permet de savoir sur quels nœuds et combien pèsent chaque shard lié à un index :

```
GET /_cat/shards/twitter*?v
```

```
#! Elasticsearch built-in security features are not enabled. Without
  ://www.elastic.co/guide/en/elasticsearch/reference/7.16/security-m
index             shard prirep state   docs    store ip          node
twitter_sentiment 0     r      STARTED 1917    1.4mb 172.29.0.4 es1
twitter_sentiment 0     p      STARTED 1917 1006.7kb 172.29.0.2 es0
```

Cela permet notamment rapidement de voir si des shards ne sont pas assignés, ce qui peut conduire à un état de santé du cluster dégradé :

```
GET /_cat/shards/twitter*?v
```

```
#! Elasticsearch built-in security features are not enabled. Without auth
  ://www.elastic.co/guide/en/elasticsearch/reference/7.16/security-minima
index             shard prirep state      docs    store ip          node
twitter_sentiment 0     r      STARTED    1917    1.4mb 172.29.0.4 es1
twitter_sentiment 0     p      STARTED    1917 1006.7kb 172.29.0.2 es0
twitter_sentiment 0     r      UNASSIGNED
twitter_sentiment 0     r      UNASSIGNED
twitter_sentiment 0     r      UNASSIGNED
```

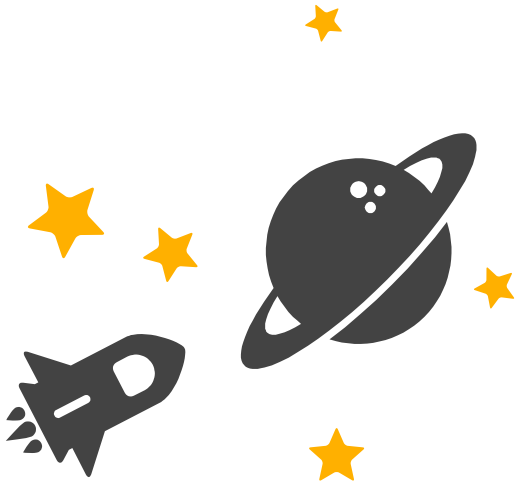


6. ARCHITECTURE

Mise en cluster / Failover / Rôles d’un noeud / Election



EXERCICE



Mise en place d’un cluster et compréhension du sharding

LES RÔLES D'UN NOEUD



Par défaut, un noeud possède tous les rôles possibles. Voici les rôles de base à comprendre :

- **Master-Eligible** (M) : Ce rôle permet au noeud d'être "master-eligible". Cela ne signifie donc pas qu'il sera forcément Master du cluster mais pourrait l'être, en fonction du résultat de l' élection (voir après)
- **Data** (D) : Le noeud stocke de la donnée et peut opérer des opérations en lecture / écriture / recherche sur la donnée
- **Ingest** (I) : Le noeud sera capable d'appliquer des pipelines d'ingestion de données (sera vu ensuite dans la partie Losgtash / Alimentation)
- **ML** (L) : (Payant) Ce type de noeud sera capable d'entraîner des modèles de Machine Learning

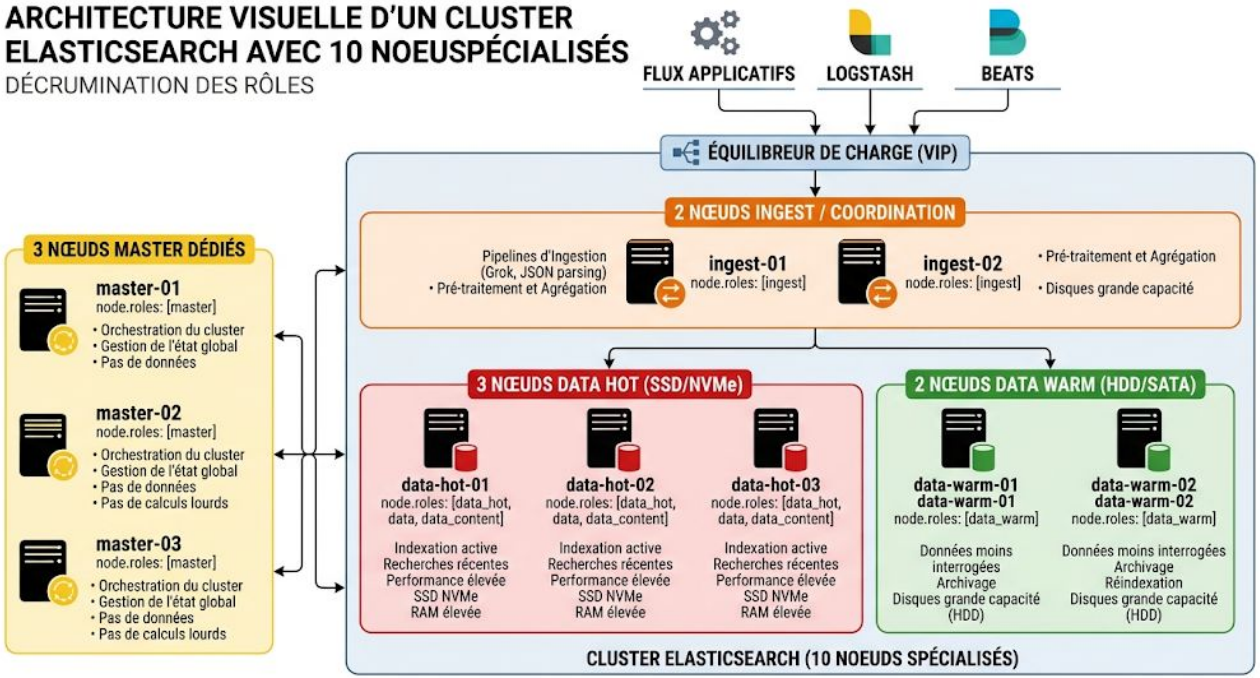
D'autres rôles possibles :

- Voting_only, data_content, data_hot, data_warm, data_cold, data_frozen, etc.

https://www.elastic.co/guide/en/elasticsearch/reference/current/modules-node.html

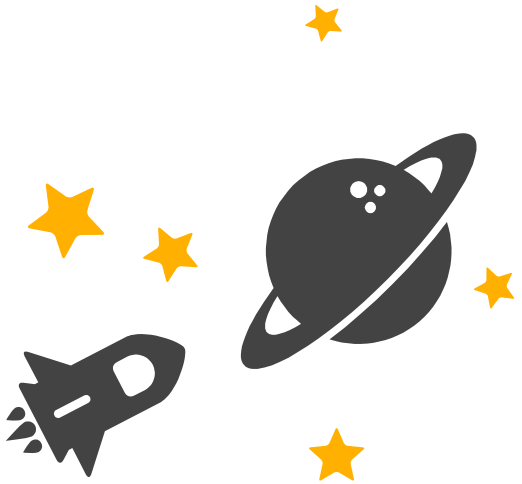
EXEMPLE D'ARCHITECTURE CIBLE

Exemple d'architecture cible :





EXERCICE



Test de failover : Tuer le master d’un cluster

PRINCIPE D'ÉLECTION D'UN MASTER



Lorsque le cluster élit un nouveau noeud pour endosser le rôle de master, tous les noeuds qui possèdent ce rôle participent au vote.

Cependant, il n'est pas utile d'obtenir les votes de tous les noeuds avec le rôle master. La moitié seulement suffit : il s'agit du **quorum**.

Pour qu'une élection puisse se conclure, le nombre de votes reçus doit être supérieur ou égal au **quorum** qu'il est possible de calculer comme suit :

Quorum = [Nombre de noeuds avec le rôle master / 2] + 1

PRINCIPE D'ÉLECTION D'UN MASTER



Si un cluster ne contient que deux noeuds Master-Eligible, le résultat du quorum est le suivant :

Quorum = (2 / 2) + 1 = 2

Cela signifie qu'il faudrait 2 noeuds Master-Eligible pour procéder au vote, ce qui n'est pas possible si l'un des noeuds venait à défaillir.

Si un cluster contient 3 noeuds Master-Eligible, tout va bien : si le master vient à défaillir, il reste suffisamment de noeuds dans le Quorum pour procéder à une élection et choisir un nouveau master :

Quorum = (3 / 2) + 1 = 2

C'est pour cette raison qu'il est de coutume de dire qu'un minimum de 3 noeuds est recommandé pour assurer un failover et la résilience de la donnée.



7. SIZING

Comment dimensionner? Combien de noeuds, de shards?



GUIDE DE DIMENSIONNEMENT

Lors de la mise en place d'un projet sur Elasticsearch, se posent souvent deux questions :

- Combien de noeuds mettre dans le cluster?
- Combien de shards allouer à mes index?

En vérité, ces deux questions sont intimement liées à la volumétrie que ces index vont stocker à terme, ainsi qu'à l'architecture choisie.

Parfois, il n'est pas facile de savoir à l'avance les volumes finaux. Il peut alors être intéressant d'expérimenter (**POC**) sur un périmètre représentatif de la donnée, qui permettrait ensuite de réaliser des projections, en fonction aussi de la durée de rétention de la donnée.



GUIDE DE DIMENSIONNEMENT

Question 1 à se poser : quelle taille va faire mon index?

Si on prend l'exemple de données de logs (au format texte) générées par une application, il est intuitivement presque impossible d'estimer la volumétrie finale qui sera contenue dans l'index.

Avant, on avait coutume de créer un index par jour mais cela rendait certains calculs compliqués.

Maintenant, on utilise le cycle de vie de la donnée (ILM) pour mettre des tailles maximales d'index (par défaut : 50 Go par shard primaire). Quand un shard primaire dépasse les 50 Go, un nouvel index est créé : c'est ce qu'on appelle le Rollover.

Note : le rollover et ILM seront détaillés à la fin du support



GUIDE DE DIMENSIONNEMENT

Question 2 à se poser : combien de shards par index?

Si on ne définit rien, le nombre de shards automatiquement alloué à un index sera de 1. Concrètement, cela signifie qu'une instance Lucène sera attribuée à l'index.

Dans les anciennes versions (avec la logique d'un index par jour), il fallait calculer à l'avance le nombre de shards à allouer. Maintenant, avec les ILM et le Rollover, c'est plus simple. On attribue 1 shard par index (valeur par défaut) ou, au maximum le nombre de noeuds Hot (vu précédemment dans la partie Rôles)

Le nombre total de shards devient le résultat d'un calcul :

```
volume par jour * rétention / taille cible x (1 + réplicas)
```



GUIDE DE DIMENSIONNEMENT

Question 3 à se poser : quel niveau de granularité?

Attention également à bien choisir le niveau de granularité de mes index.

Pour que le stockage et la recherche restent optimales, il ne faut pas que les index soient trop petits.

Il est bien plus préférable d'avoir un index par mois qui contient 20 Go pour 1 shard plutôt que 30 index de 600 Mo pour 1 shard.

Sur un mois :

Dans le premier scénario, 1 shard est mobilisé pour 20 Go de donnée et, dans le second, 30 shards sont alloués pour 600 Mo de données. Ceci est également résolu par le Rollover!



GUIDE DE DIMENSIONNEMENT

Question 4 à se poser : combien de noeuds sur le cluster?

Au delà des considérations liées à la discrimination des noeuds (master / ingest / data / etc.) et qui peuvent permettre déjà de déterminer un nombre de noeuds, une autre limite est à prendre en compte.

En effet, un noeud Elasticsearch ne peut pas accueillir plus de 1000 shards par défaut. C'est une limite de sécurité qu'il est possible de lever (**cluster.max_shards_per_node**). C'est finalement le disque qui dimensionne le cluster :

`Volume * rétention * (1 + réplicas) / 0,85` (seuil d'alerte disque)

La mise en place des ILM et du Rollover permet donc de dimensionner plus facilement les cluster!



8. INGESTION DE DONNÉES

Logstash / Agents Beats



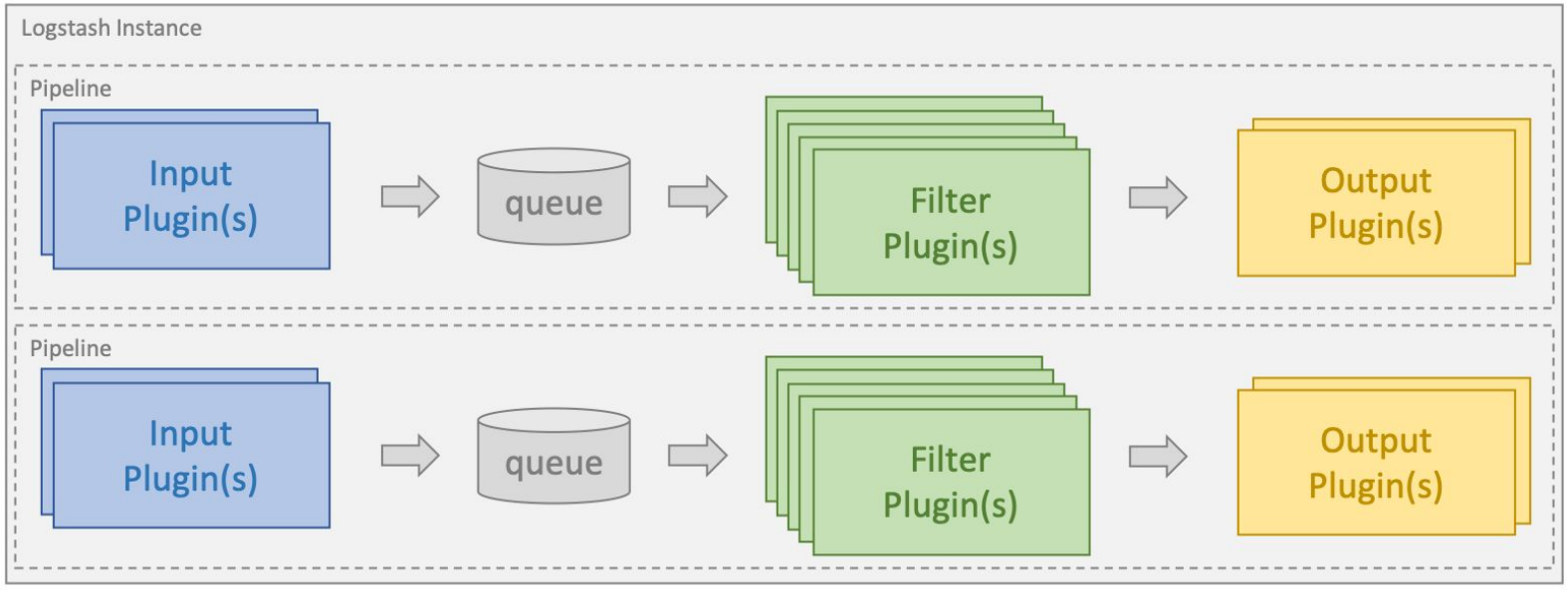
LOGSTASH

Parmi les différents produits capables d'alimenter Elasticsearch, il y a Logstash.

- **Logstash** : un moteur de collecte et de traitements (ETL) des données grâce à de nombreux plug-ins
- Logstash facilement configurable pour collecter, traiter et transférer les données dans un grand nombre d'architectures variées

Le traitement est organisé en un ou plusieurs pipelines. Dans chaque pipeline, un ou plusieurs plug-ins reçoivent ou collectent des données qui sont alors placées dans une file d'attente interne. Par défaut, cette dernière est petite et enregistrée en mémoire (RAM) mais il est possible de la configurer pour l'agrandir et l'enregistrer sur disque de manière permanente, afin d'améliorer la fiabilité et la résilience.

PIPELINE


Logstash Instance
Pipeline
Input Plugin(s)
queue
Filter Plugin(s)
Output Plugin(s)
Pipeline
Input Plugin(s)
queue
Filter Plugin(s)
Output Plugin(s)

PIPELINE (1/3)

Les pipelines Logstash sont créés à partir d'un ou plusieurs fichiers de configuration. Avant toute chose, nous commencerons par décrire les différentes options disponibles. Les répertoires décrits dans cette section peuvent varier en fonction du mode d'installation et du système d'exploitation, et sont spécifiés dans la documentation (disponible sur Internet).

- **Un seul Pipeline avec un seul fichier de configuration**

La méthode la plus simple pour démarrer Logstash consiste à demander au système de créer un pipeline, à partir d'un fichier de configuration, défini grâce au paramètre de ligne de commande -f.

PIPELINE (2/3)

Les pipelines Logstash sont créés à partir d'un ou plusieurs fichiers de configuration. Avant toute chose, nous commencerons par décrire les différentes options disponibles. Les répertoires décrits dans cette section peuvent varier en fonction du mode d'installation et du système d'exploitation, et sont spécifiés dans la documentation (disponible sur Internet).

- **Un seul Pipeline avec une collection de fichiers de configuration**

Logstash peut aussi être configuré de manière à utiliser tous les fichiers d'un répertoire spécifique comme fichiers de configuration. Cette option peut être paramétrée grâce au fichier logstash.yml ou en intégrant le chemin d'un répertoire dans la ligne de commande, à l'aide du paramètre de ligne de commande -f. Il s'agit de l'option par défaut si vous installez Logstash en tant que service.

Lorsqu'un répertoire est spécifié, tous les fichiers de ce répertoire sont concaténés par ordre lexicographique, puis analysés sous la forme d'un seul fichier de configuration. Les données provenant de toutes les entrées sont alors traitées par l'ensemble des filtres et envoyées à toutes les sorties, sauf si vous utilisez des conditions pour contrôler le flux.

PIPELINE (3/3)

Les pipelines Logstash sont créés à partir d'un ou plusieurs fichiers de configuration. Avant toute chose, nous commencerons par décrire les différentes options disponibles. Les répertoires décrits dans cette section peuvent varier en fonction du mode d'installation et du système d'exploitation, et sont spécifiés dans la documentation (disponible sur Internet).

- **Plusieurs pipelines**

Pour utiliser plusieurs pipelines dans Logstash, vous devez modifier les fichiers pipelines.yml fournis avec Logstash. Pour ce faire, vous devez accéder au répertoire des paramètres. Il contient des fichiers de configuration et des paramètres de configuration pour tous les pipelines pris en charge par cette instance Logstash.

L'utilisation de plusieurs pipelines permet de séparer des flux logiques différents. Cela permet de réduire considérablement la complexité et la quantité de conditions utilisées. Ainsi, il est plus facile de régler et de préserver la configuration. Les données qui passent dans le pipeline deviennent alors de plus en plus homogènes. Cela permet d'améliorer les gains de performance, car les plug-ins de sortie peuvent être utilisés plus efficacement.

CONFIGURATION - INPUT

Un fichier de configuration Logstash est constitué de 3 parties :

- **Input** : La liste des sources de données en entrée du traitement
- **Filter** : Les traitements opérés sur la donnée (calculs, etc.)
- **Output** : Précise où seront envoyées les données à l'issue du traitement

Input : Plusieurs possibilités :

- Csv
- Twitter
- Beats
- Etc.

```
input {
  beats {
    port => 5044
    include_codec_tag => false
  }
}
```

CONFIGURATION - FILTER

La partie "***filter***" permet de traiter la donnée (pour réaliser des calculs, découper la donnée, etc.) :

```
filter{
  grok {
    match => {"message" => '%{TIMESTAMP_ISO8601:date_log}%{SPACE}\|
  }
  date {
    match => ["date_log", "yyyy-MM-dd HH:mm:ss"]
    target => "dt_date_log"
  }
}
```

CONFIGURATION - OUTPUT



La partie "***output***" permet de préciser où seront écrites les données. Plusieurs possibilités :

- Elasticsearch
- Logstash
- Stdout (sortie standard)
- etc.

```
output {
    elasticsearch {
        hosts => "es-demo-log-noeud:9200"
        index => "demo_logs"
        user => "elastic"
        password => "tQErpj3RyxEa9EbfbZHG"
    }
    stdout { codec => json }
}
```

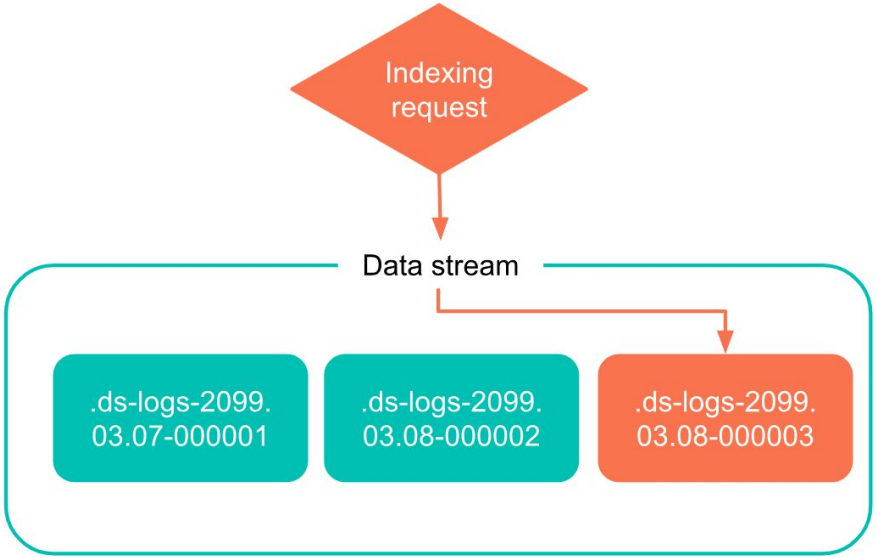
CONFIGURATION - INDEX vs DATA STREAM

Les datastreams sont plus récents dans l'offre Elastic :

Les datastreams sont constitués de 1 ou plusieurs index masqués, auto-générés.

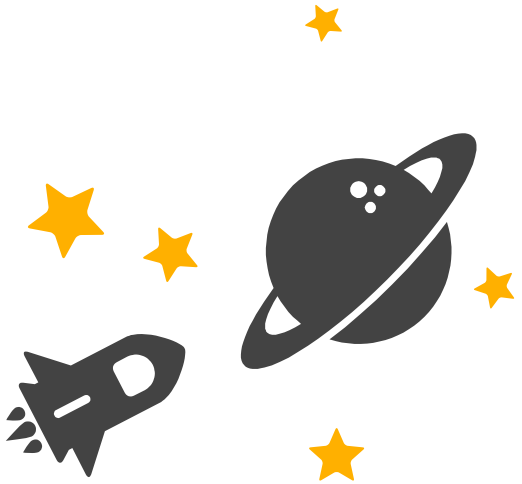
Les datastreams sont pensés pour les cas d'usages où la donnée est rarement mise à jour.

C'est donc le format parfait pour les logs.





EXERCICE



Création de quelques pipeline Logstash d'alimentation

PIPELINES MODULAIRES

Il pourrait être intéressant, dans Logstash, de pouvoir réutiliser des parties de scripts d'un pipeline à l'autre.

Par exemple, la fonction suivante transforme une chaîne en date et on souhaiterait l'utiliser plusieurs fois dans plusieurs pipeline, sans avoir besoin de la recopier intégralement dans chaque script :

```
filter {
 # Partie pour l'ESB :
 if ([type_interface] == "ESB") {
  # Conversion timestamp (epoch) vers date
  date {
   match => [ "timestamp", "UNIX", "UNIX_MS"]
   timezone => "Europe/Paris"
   target => "dt_date_log"
  }
 }

 # Partie pour l'ETL :
 if ([type_interface] == "ETL") {
  # Conversion timestamp (epoch) vers date
  date {
   match => [ "eventDate", "YYYY-MM-dd HH:mm:ss.SSS"]
   timezone => "Europe/Paris"
   target => "dt_date_log"
  }
 }
}
```

PIPELINES MODULAIRES



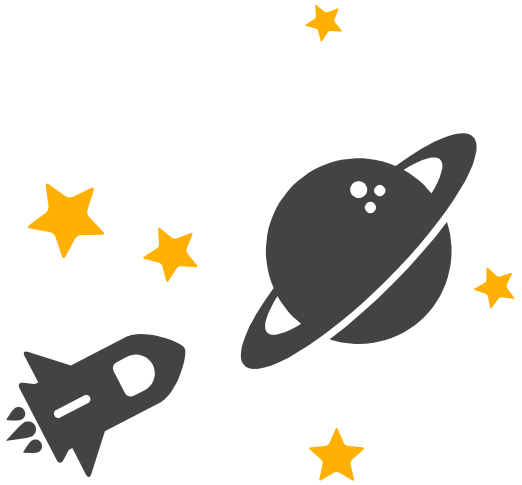
Se rendre sur le site suivant pour comprendre le principe des pipelines “modulaires” :

https://www.elastic.co/fr/blog/how-to-create-maintainable-and-reusable-logstash-pipelines

```
- pipeline.id: my-pipeline_1
  path.config: "<path>/{01_in,01_filter,02_filter,01_out}.cfg"
- pipeline.id: my-pipeline_2
  path.config: "<path>/{02_in,02_filter,03_filter,01_out}.cfg"
```



EXERCICE



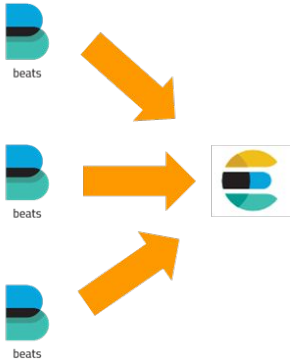
Création d’un pipeline modulaire

LOGSTASH vs BEATS

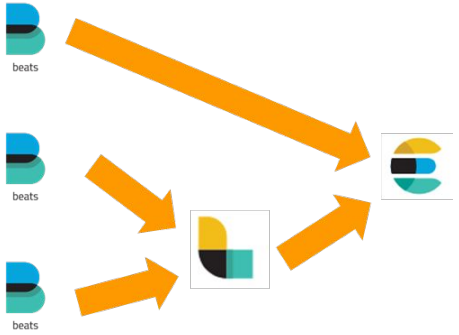
Logstash : outil puissant dans lequel il est possible de réaliser des transformations, des calculs, etc.

Son défaut : il est gourmand en ressources RAM ! Pour parer à ça, Elastic a mis au point la suite **Beats** qui est constituée d'agents légers (beaucoup moins puissants que Logstash mais moins gourmands en ressources), qu'il est possible d'installer un peu partout pour envoyer de la donnée vers Elasticsearch.

<u>Deux façons d'envoyer de la données avec Beats :</u>



Envoi **<u>sans</u>** transformations (ou alors dans Elasticsearch)



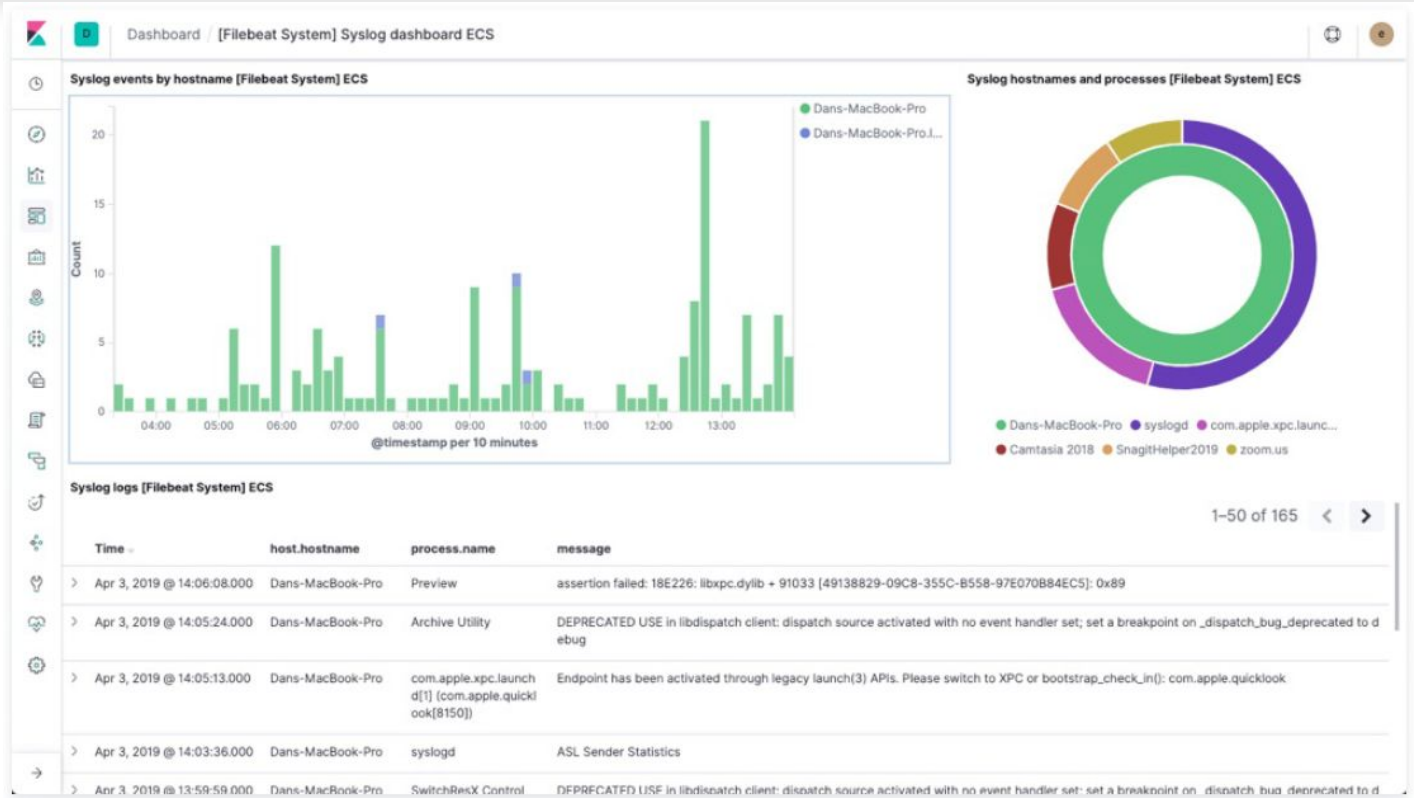
Envoi **<u>avec</u>** transformations dans Logstash

TYPES D'AGENTS DISPONIBLES

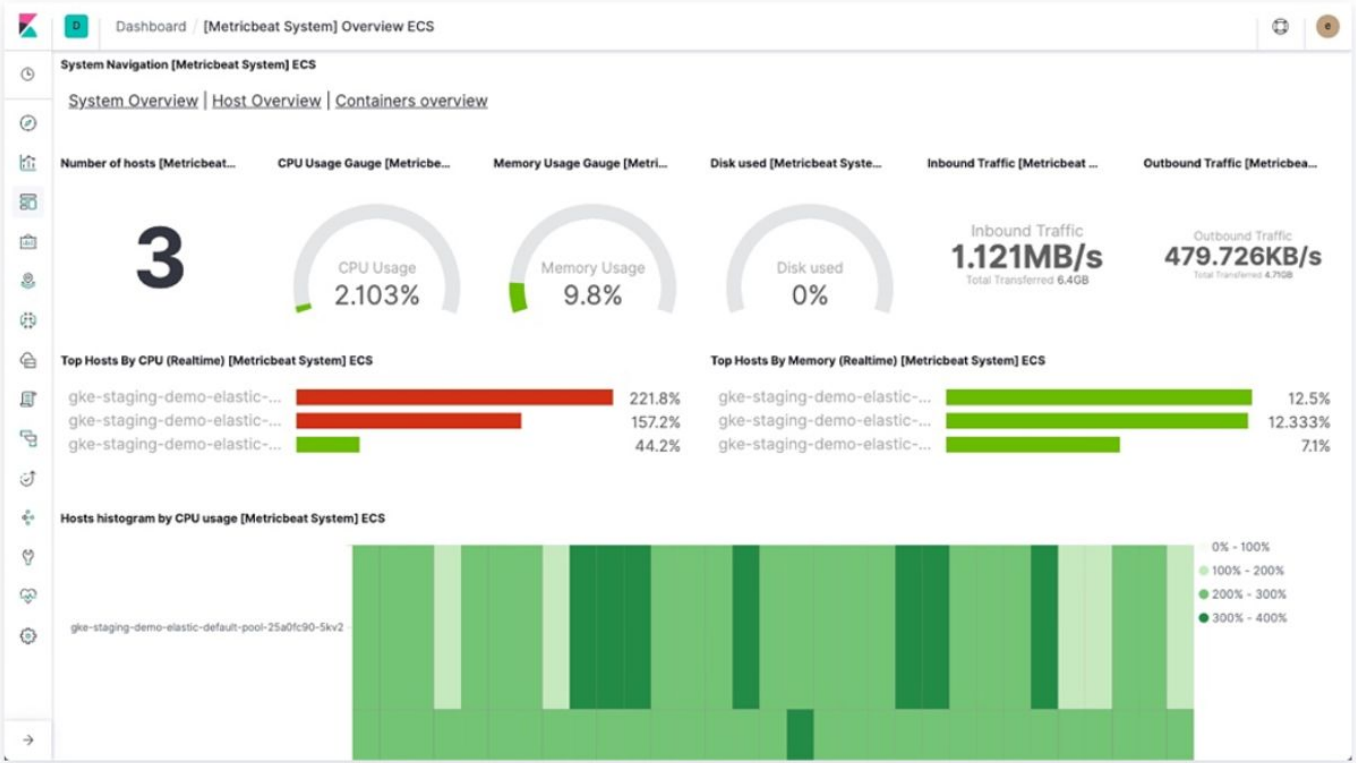


Filebeat	Agent de transfert de logs (fichiers plats)
Metricbeat	Agent de transfert de métriques issues de services (Apache, NGINX, Windows, etc.)
Packetbeat	Agent de transfert de données réseaux (paquets, etc.)
Winlogbeat	Agent de transfert de logs d'événements Windows
Auditbeat	Agent de transfert de logs d'audit Linux (auditd)
Heartbeat	Agent de surveillance de disponibilité
Functionbeat	Agent de transfert de logs cloud (FaaS : Function As A Service)

BEATS : FILEBEAT

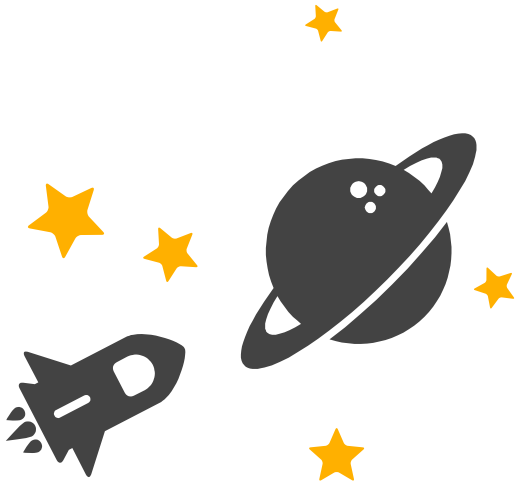


BEATS : METRICBEAT





EXERCICE



Mise en oeuvre de Metricbeat

METRICBEAT (UBUNTU)

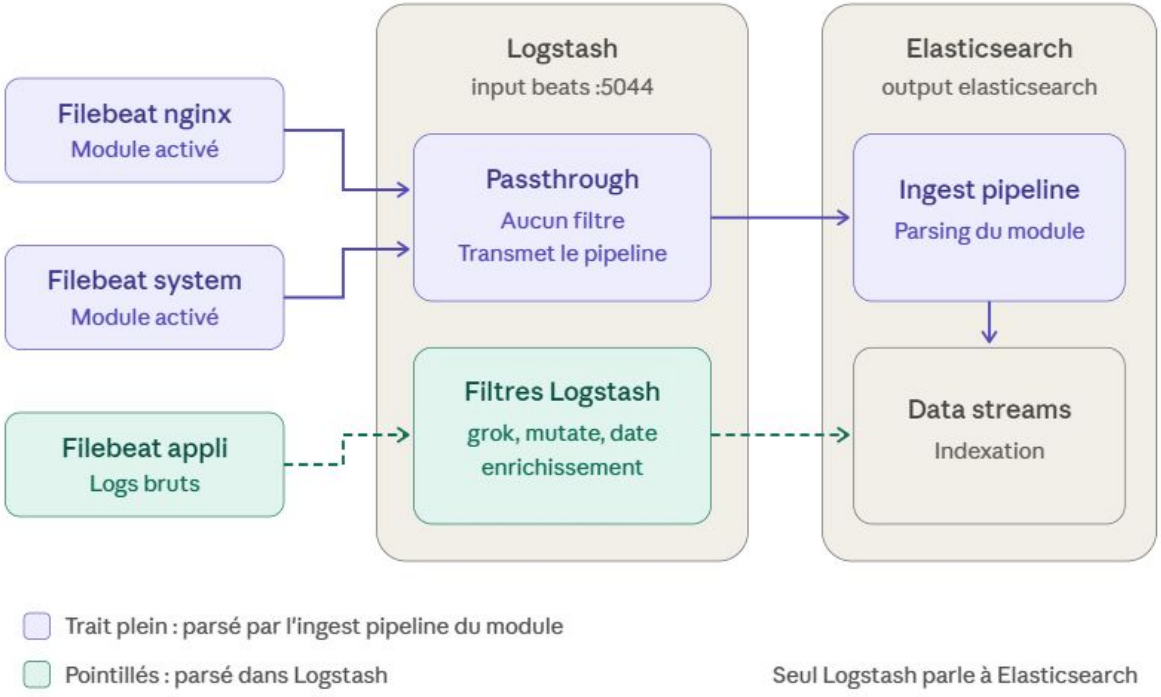


Exercice :

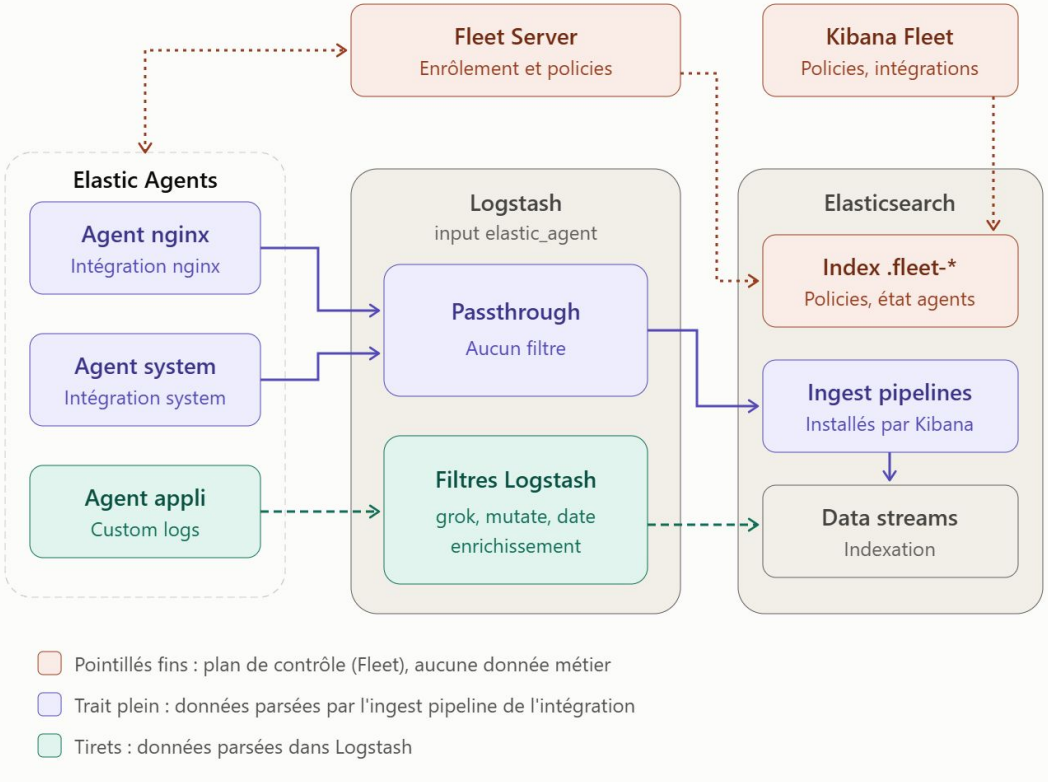
Collecter des informations système (RAM / CPU / Disque / ec.) d'un poste Ubuntu et les stocker dans Elasticsearch pour analyse.

1. Télécharger Metricbeat et dézipper sur un poste Ubuntu
2. Modifier le fichier ***metricbeat.yml*** (si nécessaire)
3. Ouvrir un terminal et lancer les commandes suivantes :
   a. ./metricbeat modules enable linux
   b. ./metricbeat setup (pour créer les dashboards dans Kibana)
   c. ./metricbeat -e (Pour lancer Metricbeat)

LOGSTASH EN MODE HYBRIDE

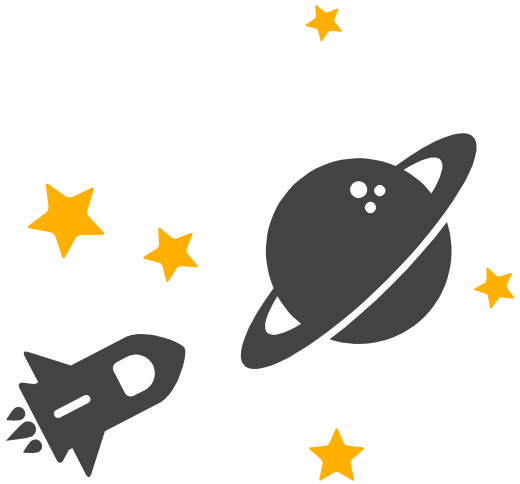


ET AVEC FLEET





EXERCICE



Ingestion de données Apache avec Filebeat

TP 1 : FILEBEAT SANS MODULE



Exercice :

Parser les données d'un fichier access.log d'Apache

1. Télécharger Metricbeat et dézipper sur un poste Windows
2. Modifier le fichier ***filebeat.yml***
3. Ouvrir une invite de commande Windows et lancer les commandes suivantes :
   a. ./filebeat -e (Pour lancer Filebeat)

Quels sont les avantages / inconvénients de cette méthode?

TP 1 : FILEBEAT SANS MODULE

Corrigé :

Contenu du fichier **filebeat.yml** :

```
filebeat.inputs:

# Each - is an input. Most options can be set at the input level, so
# you can use different inputs for various configurations.
# Below are the input specific configurations.

# filestream is an input for collecting log messages from files.
- type: filestream

  # Change to true to enable this input configuration.
  enabled: true

  # Paths that should be crawled and fetched. Glob based paths.
  paths:
    - /home/user/elastic/data/*.logs
    #- c:\programdata\elasticsearch\logs\*
```

TP 2 : FILEBEAT AVEC LE MODULE APACHE



Exercice :

Parser les données d'un fichier access.log d'Apache

1. Télécharger Metricbeat et dézipper sur un poste Windows
2. Modifier le fichier ***filebeat.yml*** (si nécessaire)
3. Ouvrir une invite de commande Windows et lancer les commandes suivantes :
   a. ./filebeat modules enable apache
   b. Modifier le fichier modules.d\apache.yml
   c. ./filebeat -e (Pour lancer Filebeat)

Vous voyez la différence?

TP 2 : FILEBEAT AVEC LE MODULE APACHE



Corrigé :

Contenu du fichier modules.d/apache.yml

```
- module: apache
  # Access logs
  access:
    enabled: true

    # Set custom paths for the log files. If left empty,
    # Filebeat will choose the paths depending on your OS.
    var.paths: ["/home/user/elastic/data/*.logs"]

  # Error logs
  error:
    enabled: false
```

INGEST PROCESSORS

Si vous disposez de noeuds avec le rôle Ingest (I), il est alors possible de mettre en place des Ingest Processors. Ces processus se déclencheront à chaque fois qu'une nouvelle donnée sera ajoutée dans Elasticsearch et permettront de transformer la donnée au besoin.

Exemple :

Tentons d'insérer le document suivant dans Elasticsearch.

Le champ dt_insertion sera vu comme étant un champ texte.

Or il pourrait être pratique de demander à le convertir en champ de type date

```
POST dossier_client/_doc/
{
  "dt_insertion": "05/05/2022 17:14:12",
  "message": "Test d'envoi de données dans Elastic",
  "utilisateur": {
    "prenom": "Vincent",
    "age": 28
  }
}
```

INGEST PROCESSORS

Pour cela, nous allons créer le pipeline suivant :

```
PUT _ingest/pipeline/UTC_to_french_timestamp
{
  "processors": [
    {
      "date" : {
        "field" : "dt_insertion",
        "target_field": "dt_insertion",
        "formats" : ["dd/MM/yyyy HH:mm:ss"],
        "timezone" : "Europe/Paris"
      }
    }
  ]
}
```

INGEST PROCESSORS

Il est possible de tester le pipeline avant de l'utiliser réellement en production :

```
POST _ingest/pipeline/UTC_to_french_timestamp/_simulate
{
  "docs": [
    {
      "_source": {
        "dt_insertion": "05/05/2022 17:14:12"
      }
    }
  ]
}
```



```
{
  "docs" : [
    {
      "doc" : {
        "_index" : "_index",
        "_type" : "_doc",
        "_id" : "_id",
        "_source" : {
          "dt_insertion" : "2022-05-05T17:14:12.000+02:00"
        },
        "_ingest" : {
          "timestamp" : "2022-06-28T06:51:11.610383202Z"
        }
      }
    }
  ]
}
```

INGEST PROCESSORS



Enfin, il est possible d'utiliser le pipeline lors de l'indexation de nouvelles données, pour que le noeud puisse traiter la donnée avant insertion :

```
POST dossier_client/_doc/?pipeline=UTC_to_french_timestamp
{
  "dt_insertion": "05/05/2022 17:14:12",
  "message": "Test d'envoi de données dans Elastic",
  "utilisateur": {
    "prenom": "Vincent",
    "age": 28
  }
}
```



9. LA PERTINENCE

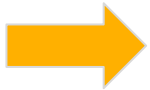
Comprendre le mécanisme de pertinence

CALCUL DE LA PERTINENCE ET SCORING

Exemple de recherche dans Elasticsearch :

Ici, on cherche dans l'index "library" tous les livres dont le titre contient le mot "quick" :

```
GET /library/books/_search
{
 "query": {
   "match": {
     "title": "quick"
   }
 }
}
```




```
{
  "took": 68,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": 3,
    "max_score": 0.6330254,
    "hits": [
      {
        "_index": "library",
        "_type": "books",
        "_id": "3",
        "_score": 0.6330254,
        "_source": {
          "title": "The quick brown fox jumps over the quick dog",
          "price": 8,
          "colors": [
            "red",
            "blue"
          ]
        }
      },
      {
        "_index": "library",
        "_type": "books",
        "_id": "1",
        "_score": 0.6103343,
        "_source": {
          "title": "The quick brown fox",
          "price": 5,
          "colors": [
            "red",
            "green",
            "blue"
```

CALCUL DE LA PERTINENCE ET SCORING

Notion de « ***Relevance*** » (pertinence) pour l'attribution du score par Elasticsearch :

https://www.elastic.co/guide/en/elasticsearch/guide/current/relevance-intro.html

Eléments de prise en compte de la pertinence :

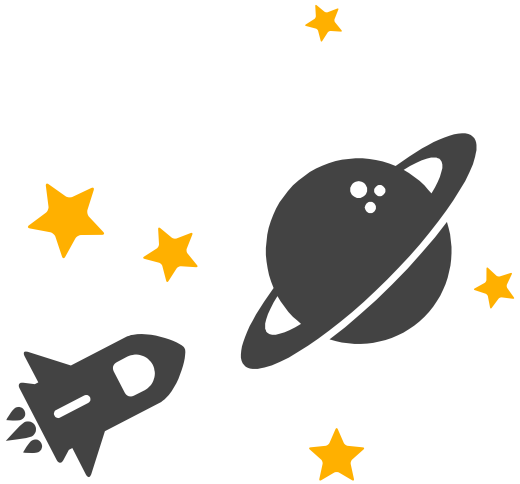
- **TF** (**T**erm **F**requency) : plus un terme recherché est présent dans un champ et plus il sera pertinent : aura un score élevé
- **IDF** (**I**nverse **D**ocument **F**requency) : plus un terme est présent dans l'index et moins il aura de poids dans le calcul du score (mots tels que « le », « la », etc.)
- **FLN** (**F**ield-**L**ength **N**orm) : plus une phrase est longue et plus la probabilité d'y rencontrer un terme augmente, rendant mécaniquement sa pertinence moins élevée

Si le score obtenu ne vous parait pas clair, Elasticsearch peut vous expliquer comment ce dernier a été calculé

```
GET /_search
{
  "explain": true,
  "query": {
   "match_phrase": {
     "title": "quick dog"
   }
  }
}
```



EXERCICE



Comprendre le calcul de la pertinence et expérimenter le langage DSL

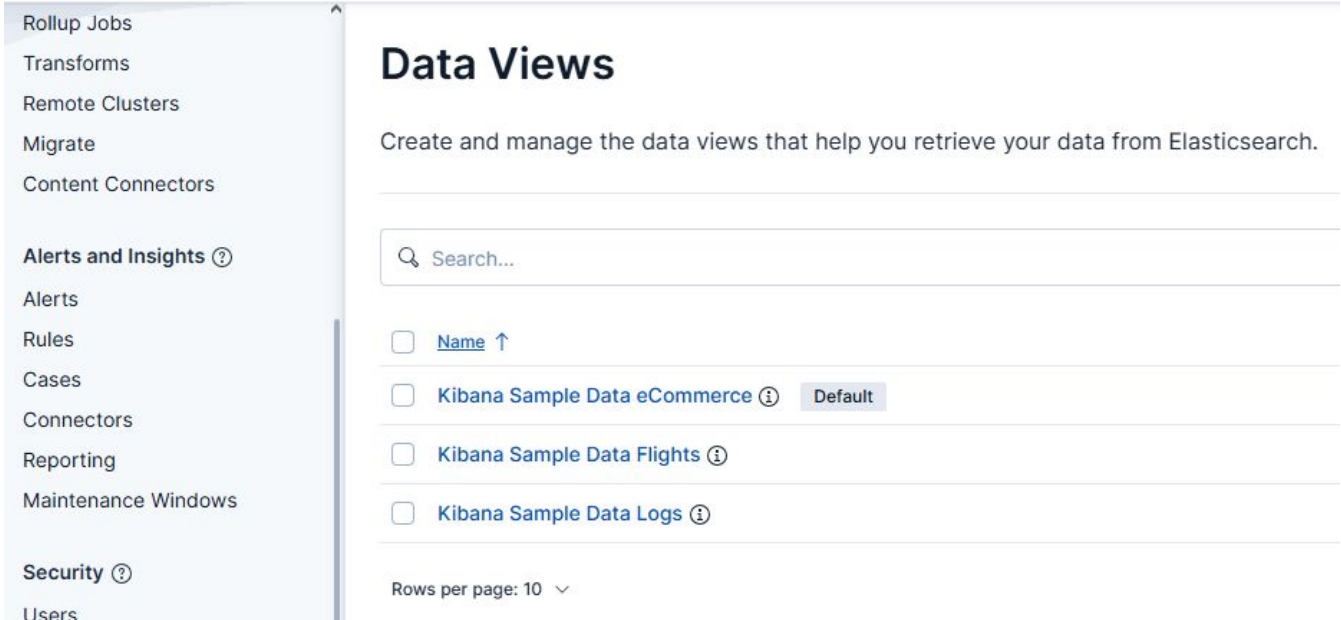


10. KIBANA

Découverte, visualisation et analyse de données

INDEX PATTERN (DATA VIEWS EN 8)

Avant de pouvoir visualiser de la donnée dans Kibana, il est indispensable de créer des Index Pattern. Ces derniers sont gérés dans le menu « **Stack Management > Index patterns (ou Data Views en v8)** » :



INDEX PATTERN (DATA VIEWS EN 8)



Un index pattern permet d'associer un ensemble d'index à un seul et même nom d'index, en utilisant le caractère spécial « * »

Exemple :

Vous avez peut-être des index hebdomadaires de données :

- my_application_logs_202101
- my_application_logs_202102
- my_application_logs_202103
- Etc.

Pour pouvoir tous les visualiser de manière consolidée dans Kibana, nous allons donc créer un index pattern :

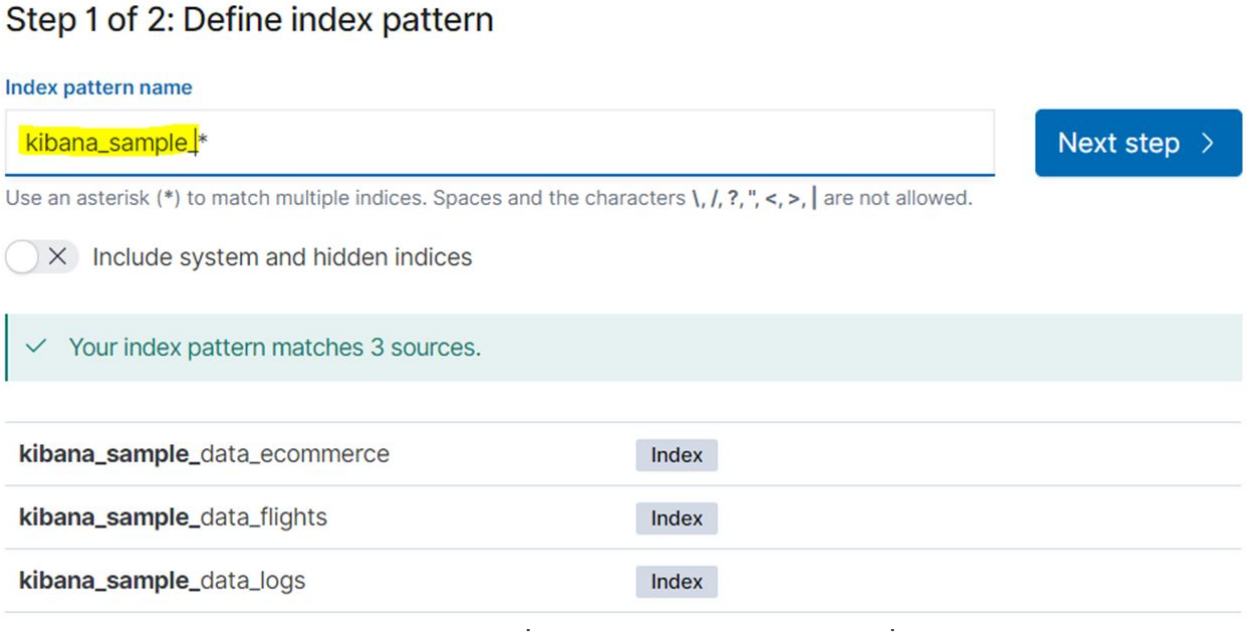
- my_application_logs*

Cet index pattern permettra de voir tous les index en lien avec le nom choisi.

Attention : cela suppose un bon nommage de vos index!

INDEX PATTERN (DATA VIEWS EN 8)

Dans l'exemple ci-dessous, nous tentons de créer un index pattern **kibana_sample_*** :



Kibana

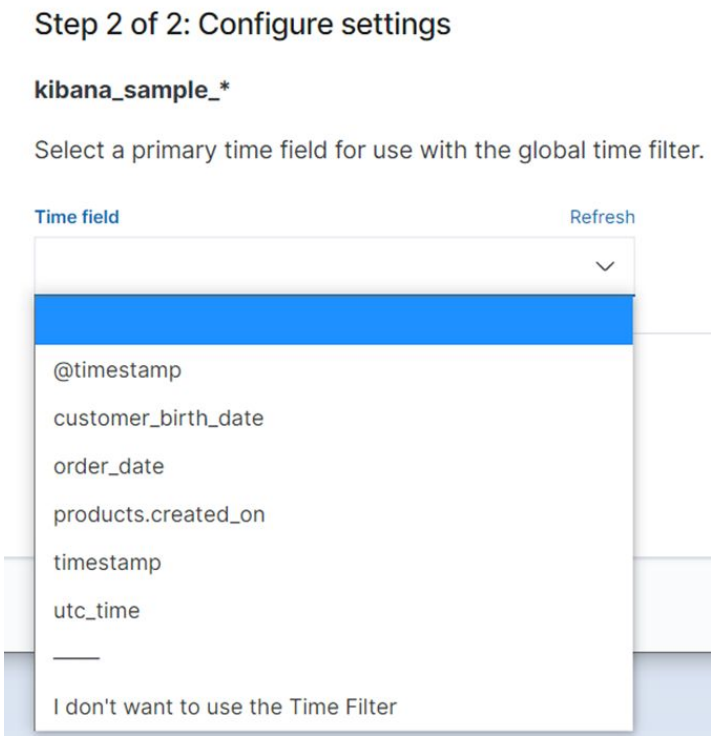
INDEX PATTERN (DATA VIEWS EN 8)

Dans l'exemple ci-dessous, nous tentons de créer un index pattern **kibana_sample_*** :

Maintenant, il faut choisir quelle sera la date « principale » de l'index pattern.

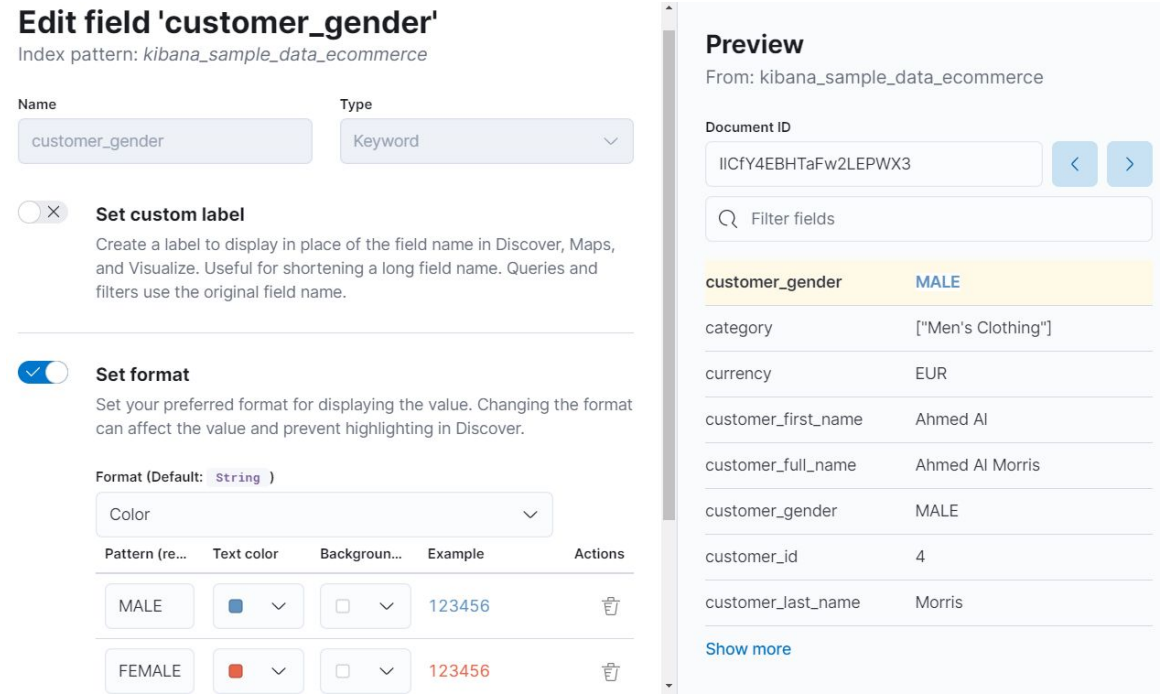
C'est la date sur laquelle les opérations de recherche dans le temps s'effectueront.

Vous pouvez aussi ne pas renseigner de date mais vous ne pourrez pas, dans le menu Découverte, filtrer simplement sur l'aspect temps.



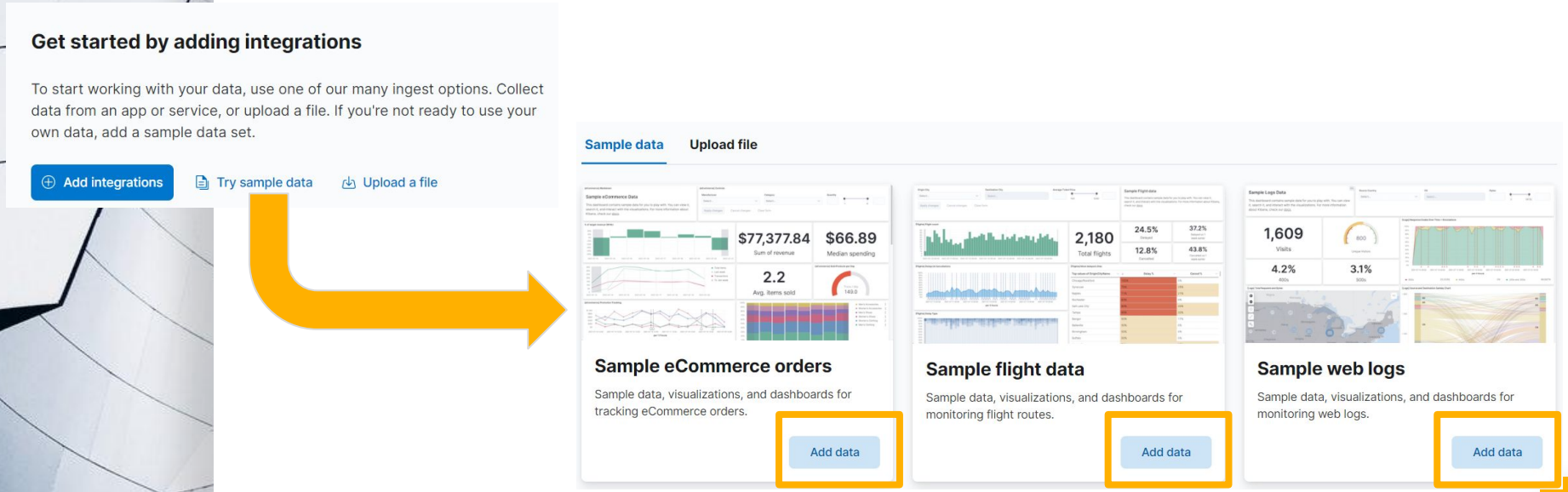
INDEX PATTERN (DATA VIEWS EN 8)

Il est également possible de modifier l'index pattern pour ajouter des couleurs selon des valeurs de données (par exemple) :

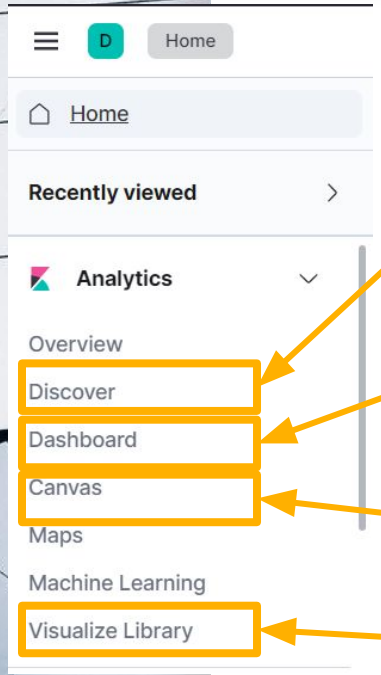


AJOUT DE DONNÉES SAMPLE

Pour tester, il est possible d'ajouter de la donnée de démonstration :



MENU KIBANA : ANALYTICS



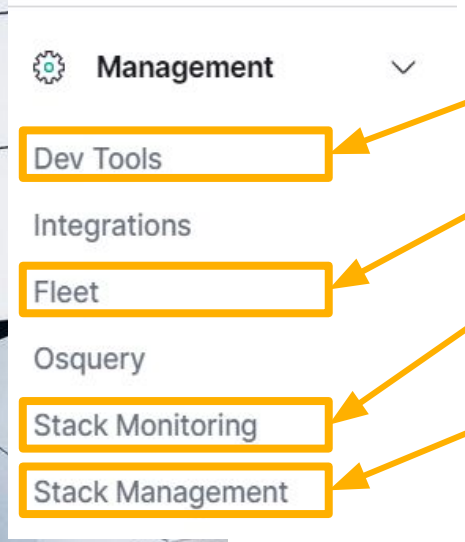
Discover : le menu pour voir la donnée brute, réaliser de la recherche dessus et découvrir son jeu de données

Dashboard : pour pouvoir consulter / créer des tableaux de bords à partir d'une bibliothèque de visualisations

Canvas : pour créer des présentations à la PowerPoint

Visualize Library: pour consulter / créer des visualisations qui serviront ensuite dans des tableaux de bord

MENU KIBANA : MANAGEMENT



Dev Tools : pour interroger les données dans le cluster à l'aide du langage DSL

Fleet : pour monitorer les Elastic Agent

Stack Monitoring : pour voir l'état de santé du cluster / des index

Stack Management : pour gérer les index pattern, les espaces, la sécurité, etc.

DISCOVER

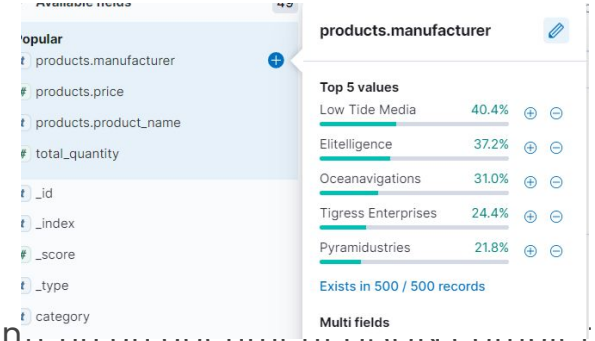
Menu discover :
C'est ici que tout commence. C'est l'endroit parfait pour découvrir sa donnée, réaliser des filtres, des recherches, etc.



DISCOVER

Menu discover : Découverte de la donnée

- Possibilité de réaliser des recherches en KQL, en Lucene ou en ES|QL
- Possibilité de jouer sur l'axe temps (en fonction de l'axe temporel choisi dans l'Index Pattern précédemment créé)
- Possibilité de sauvegarder ses recherches (pour créer des visualisations dessus par exemple)
- Possibilité de faire un peu de profiling de données, comme ceci :



- Possibilité de voir le contenu du document JSON complet

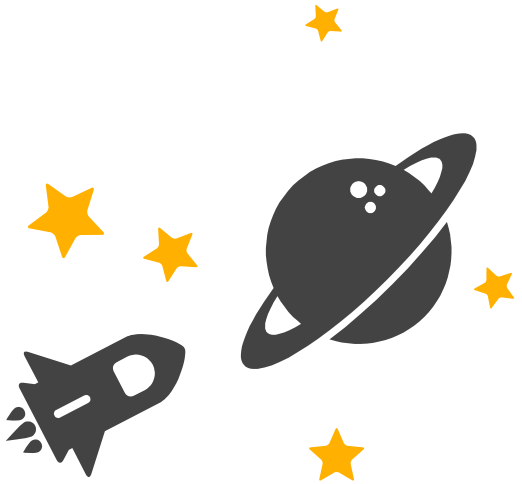
DISCOVER : Query DSL vs KQL vs ES|QL



	Query DSL	KQL | Lucene	ES|QL
Usage typique	Requêtes JSON complexes, agrégations avancées, utilisées par les applications	Barre de recherche Kibana (Discover, Lens classique)	Langage complet : filtrage + transformation + agrégation + (depuis peu) jointures
Où on l'utilise	API REST, Dev Tools Console	Discover, Lens, alerting	Dev Tools Console (`POST _query`), Discover (mode ES|QL), Lens (visualisations ES|QL), API `_query`
Syntaxe	JSON	Expression simple type moteur de recherche	Pipeline façon SQL/PRQL avec `
Peut agréger/transformer les données ?	Oui (agrégations), mais verbeux	Non (filtrage uniquement)	Oui, nativement, dans la même requête que le filtrage
Courbe d'apprentissage	Élevée	Très faible	Faible à moyenne (proche de SQL)



EXERCICE



Réaliser des recherches dans Discover

LUCENE vs KQL (1 / 2)



Opération	KQL	Lucene
Recherche libre (plusieurs termes)	`dark light`	`dark light`
ET / OU	`orange and (dark or light)`	`orange AND (dark OR light)`
Recherche sur un champ précis	`color : orange`	`color:orange`
Recherche d'une phrase exacte	`"our planet"`	`"our planet"`
Termes réservés en tant que valeurs	`"and" "or" xor` (entre guillemets)	opérateurs toujours en MAJUSCULES pour être interprétés comme tels
Wildcard en fin/milieu de mot	`dark*`, `d*k`	`dark*`, `d*k`
Wildcard sur le nom du champ	`user.address.* : fakestreet`	non supporté
Comparaison numérique/date	`price >= 42 and price < 100`	`price:>=42 AND price:<100`

LUCENE vs KQL (2 / 2)



Opération	KQL	Lucene
Plage fermée (range)	non supporté (utiliser deux comparaisons)	`price:[4000 TO 5000]`
Négation de recherche	`not city_name : Paris`	`NOT city_name : Paris`
Plage ouverte	non supporté	`price:[4000 TO *]`
Existence d'un champ	`destination : *`	`_exists_:destination`
Recherche sur objets/champs imbriqués (nested)	`products:{ name:pencil and price > 10 }`	non supporté
Expressions régulières	non supporté	`mail:/mailbox\.org$/`
Recherche floue (fuzzy, tolère les fautes de frappe)	non supporté	`user:maria~`
Boost (pondération de la pertinence)	non supporté	`title:elasticsearch^2`
Autocomplétion des champs/valeurs dans la barre Kibana	oui	non

DISCOVER - TP

Suivre le TP : https://github.com/vincent2mots/elk/blob/main/Kibana/TP%20Discover/README.md

🔎 Travaux Pratiques : Recherches avec Discover

Ce dépôt contient une série d'exercices pratiques pour maîtriser le menu **Discover** de Kibana : recherche KQL, Lucene, ES|QL, filtres, sauvegarde de recherches et profiling de champs — en utilisant les jeux de données d'exemple `kibana_sample_data_logs`, `kibana_sample_data_ecommerce` et `kibana_sample_data_flights`.

(Même esprit que le dépôt `Kibana/TP Lens` : un TP = un objectif + une configuration à reproduire.)

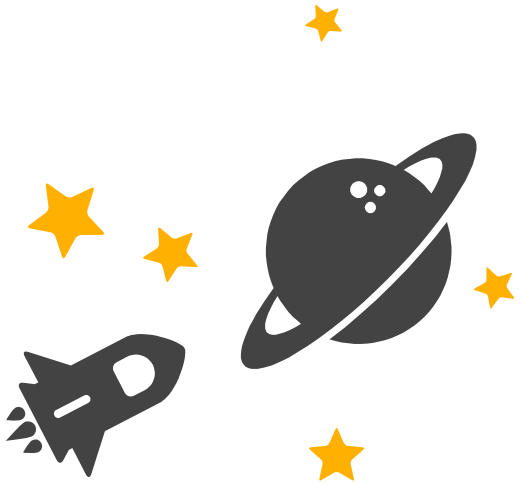
🟢 Niveau 1 : KQL — les bases

TP 1 : Recherche simple par mot-clé

- **Objectif** : Trouver toutes les requêtes ayant transité par le navigateur Chrome.
- **Configuration** :
  - Data view : `kibana_sample_data_logs`.
  - Barre de recherche (KQL) : `agent : *Chrome*`
  - *Astuce* : utiliser l'autocomplétion (commencer à taper `agent` puis `:`) pour découvrir les champs disponibles sans avoir à les connaître par cœur.



EXERCICE



Créer de nouveaux champs calculés à la volée (Runtime Fields)



RUNTIME FIELDS

Les “**Runtime Fields**” sont des champs particuliers venus remplacer les “Scripted Fields” que l’on trouvait auparavant dans la partie “Index Patterns”.

Ces champs dynamiques vont permettre de créer des mesures, des calculs qui n’avaient pas été pensés lors de l’alimentation dans les index.

Exemple :

Dans le jeu de données **kibana_sample_data_ecommerce**, nous aimerions ajouter un nouveau champ “**time_of_day**” qui permettrait d’extraire l’heure du champ “**order_date**”

Cela pourrait par exemple permettre ensuite de filtrer facilement dans les visuels pour ne prendre que les données du jour dont les heures sont comprises entre 8h du matin et 17h.

RUNTIME FIELDS



Il est possible d'ajouter ce champ à l'aide d'une requête DSL, comme ceci :

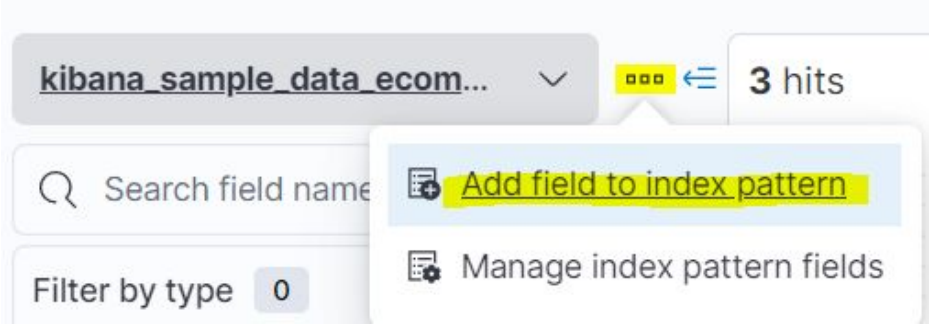
```
PUT /kibana_sample_data_ecommerce/_mapping
{
 "runtime": {
  "time_of_day": {
   "type": "long",
   "script": {
    "source": "emit(LocalDateTime.ofInstant(Instant.ofEpochMilli(doc['order_date'].value.millis), ZoneId.of('Europe/Paris')).getHour())"
   }
  }
 }
}
```



Attention : les dates sont par défaut stockées au format UTC. Il faut donc prévoir un script prenant en compte le fuseau horaire.

RUNTIME FIELDS

Ou bien directement depuis le menu Discover de Kibana :

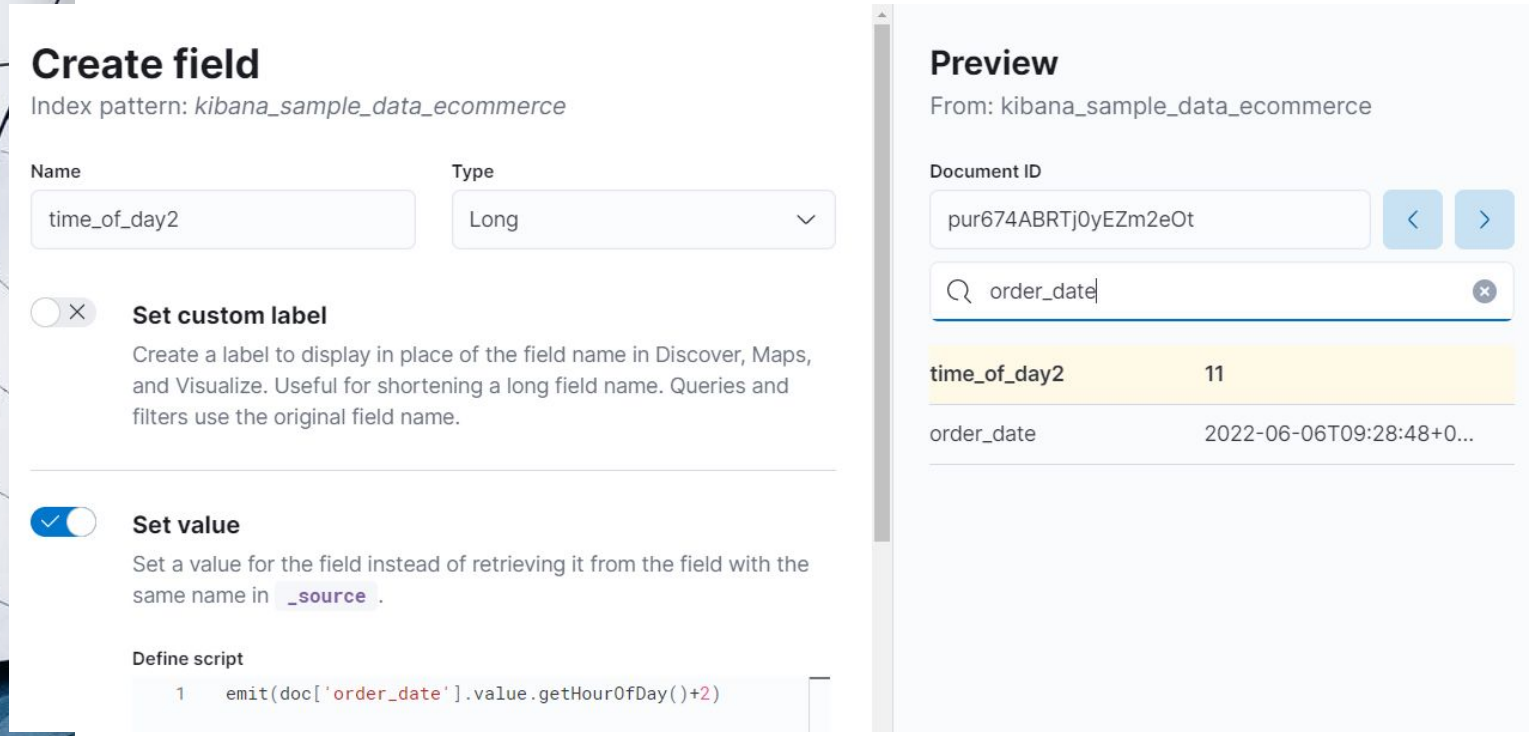


Nous allons reprendre la formule suivante :

```
emit(LocalDateTime.ofInstant(Instant.ofEpochMilli(doc['order_date'].value.millis),
ZoneId.of('Europe/Paris')).getHour())
```

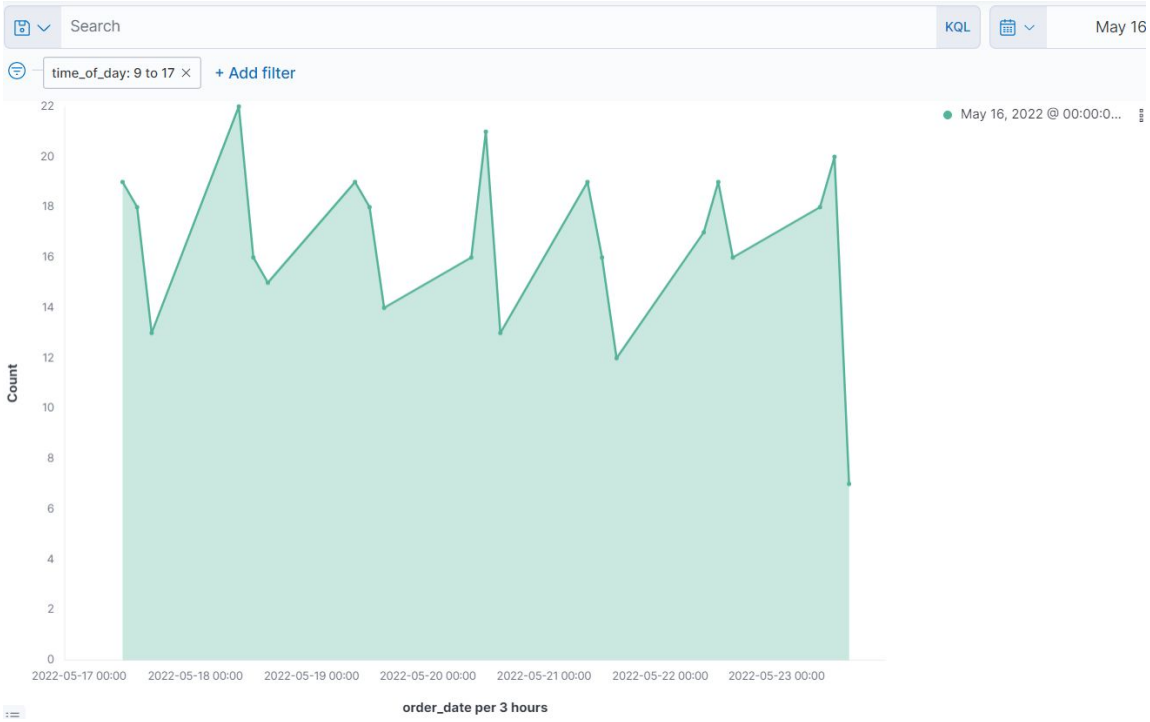
RUNTIME FIELDS

Ou bien directement depuis le menu Discover de Kibana :



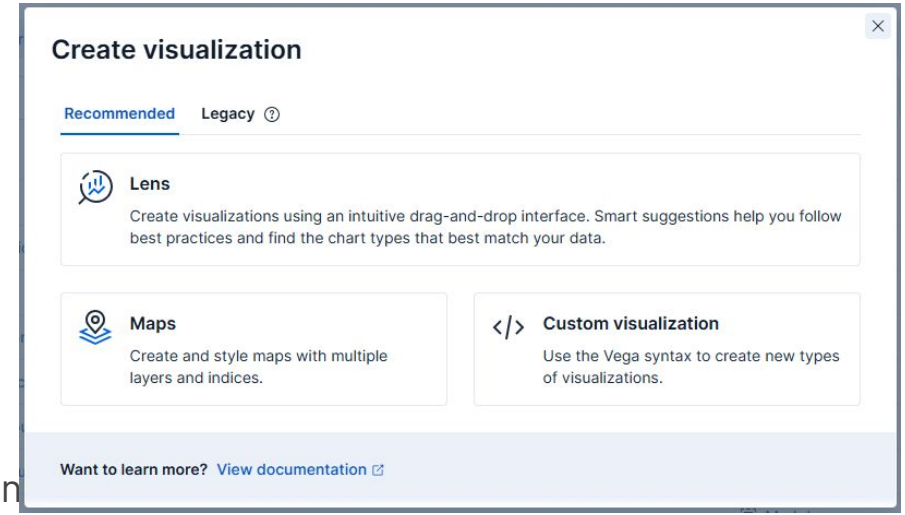
RUNTIME FIELDS

Il ne reste plus qu'à l'utiliser dans des filtres :



VISUALIZE LIBRARY

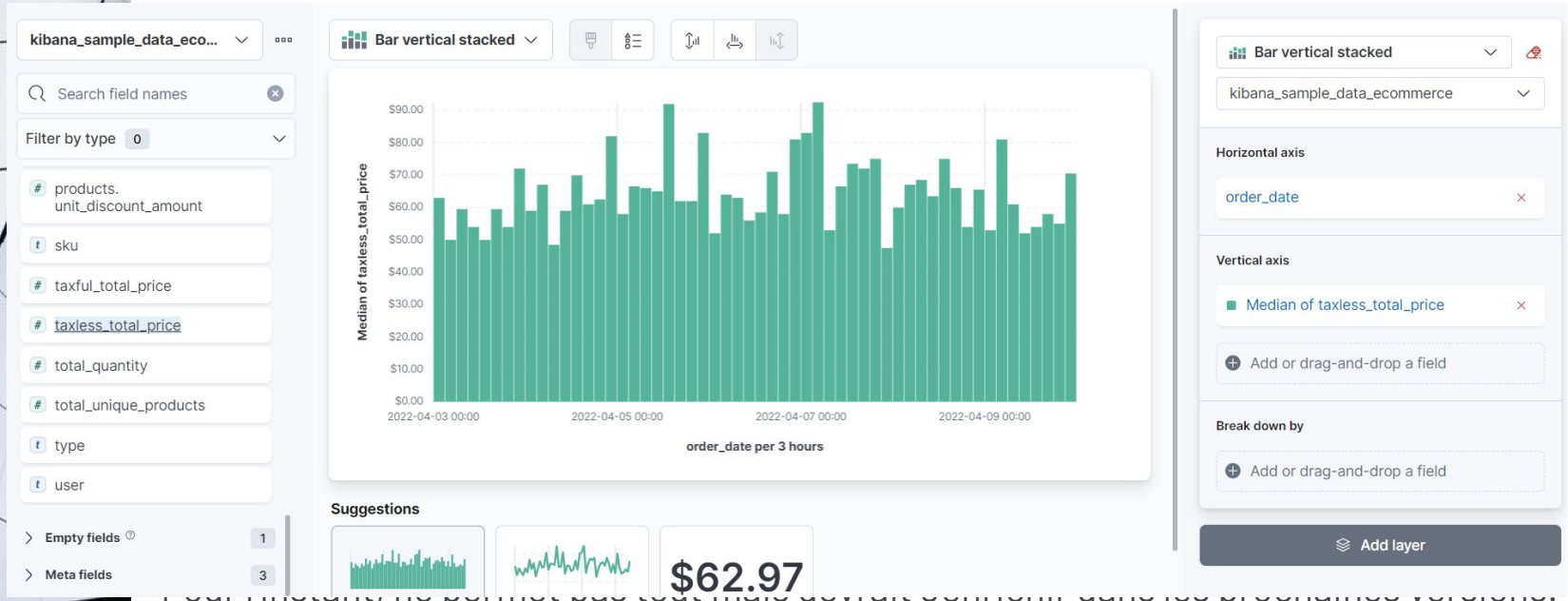
Le menu Visualize Library va vous permettre de créer des visualisations, de plusieurs manière différentes :



En V8 : Il reste essen déprécié dans quelques versions

VISUALIZE LIBRARY : LENS

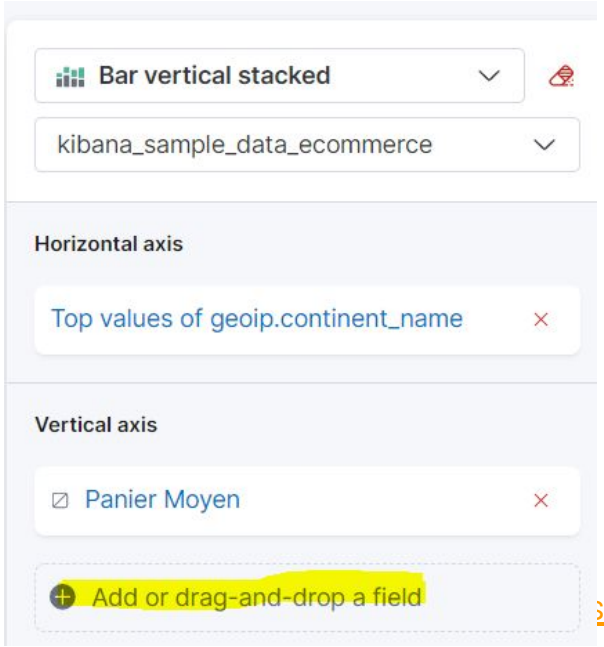
Lens est arrivé en version 7.10 et propose une interface simplifiée pour créer des visualisations :

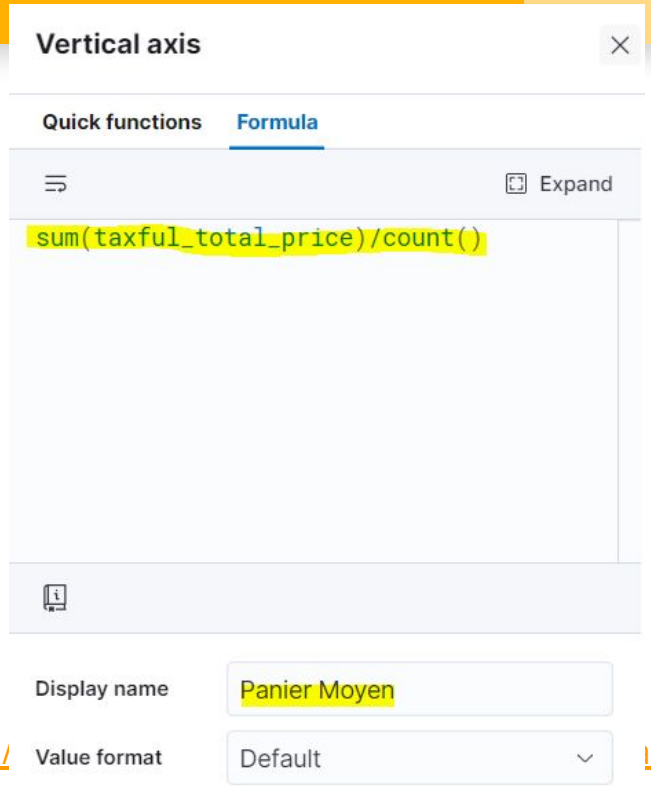


Pour l'instant, il ne permet pas tout mais devrait s'enrichir dans les prochaines versions.

VISUALIZE LIBRARY : LENS

Lens permet l'ajout de calculs à la volée :

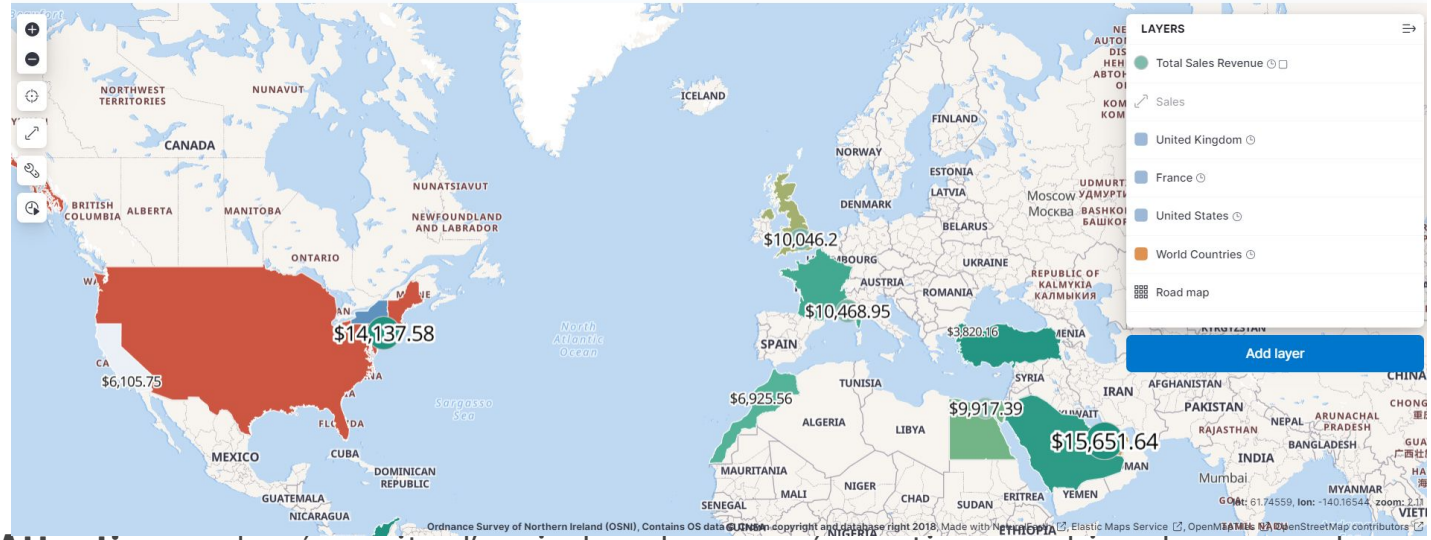




stic.co/guide/en/ ıs

VISUALIZE LIBRARY : MAPS

Maps s'appuie sur Elastic Map Service et permet de créer de la cartographie multicouches :

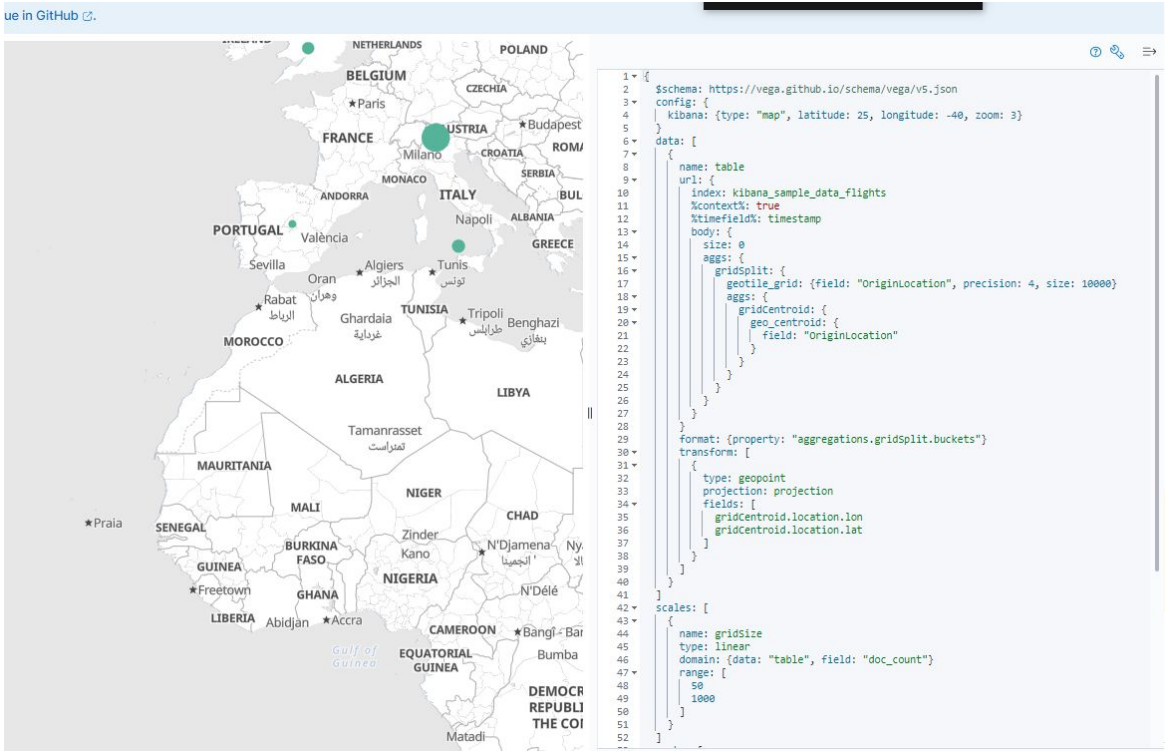


Attention : cela nécessite d'avoir des champs géospatiaux, ou bien des noms de zones géographiques (ou via des GeoJSON)

VISUALIZE LIBRARY : CUSTOM

Custom permet de créer des visualisations basées sur le langage Vega Lite.

Cela permet de créer des visualisations qui n'existent pas par défaut dans Kibana.





MARKDOWN

Syntaxe markdown :

- **#** : Permet de créer un en-tête avec plusieurs niveaux (en fonction du nombre de dièses). Attention de bien laisser un espace après les dièses
- **texte** : Permet de mettre en gras le texte
- *texte* : Permet de mettre en italique
- ***texte*** : Permet de mettre en gras et en italique
- **>** : Permet de mettre un bloc de type citation. Peut s'imbriquer dans d'autres citations (**>>**)
- **!** : Permet de mettre une image en lien (**![titre](lien)**)
- `texte` : Permet d'introduire du code
- ******* : Insère une ligne horizontale
- **[Titre](Lien)** : Permet d'introduire des liens (idéal pour rediriger vers d'autres dashboards dans Kibana!)
- **<**texte@gmail.com**>** : Permet d'introduire des URL / adresses mail

MARKDOWN

Exemple de bloc texte

Il est possible de mettre des Liens vers des ressources extérieures.



Contacter notre administrateur@gmail.com

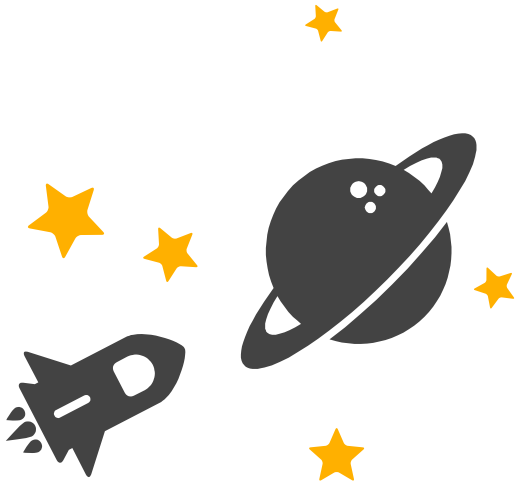
Data | Options

Markdown Help

```
### Exemple de bloc texte
***
Il est possible de mettre des [Liens](http://google.com) vers des ressources extérieures.
![image]
(https://images.contentstack.io/v3/assets/bltefdd0b53724fa2ce/blt5d10f3a91df97d15/620a9ac8849cd4
22f315b83d/logo-elastic-vertical-reverse.svg)
***
Contacter notre <administrateur@gmail.com>
```



EXERCICE



Se constituer un ensemble de visualisations avec Lens

TP LENS

Se rendre ici et suivre les différents TP proposés :
https://github.com/vincent2mots/elk/tree/main/Kibana/TP%20Lens

🎓 Travaux Pratiques : Visualisation avec Kibana Lens

Ce dépôt contient une série d'exercices pratiques pour maîtriser l'outil **Lens** dans Kibana, en utilisant le jeu de données d'exemple `kibana_sample_data_ecommerce`.

🛒 Niveau 1 : Fondamentaux & Métriques simples

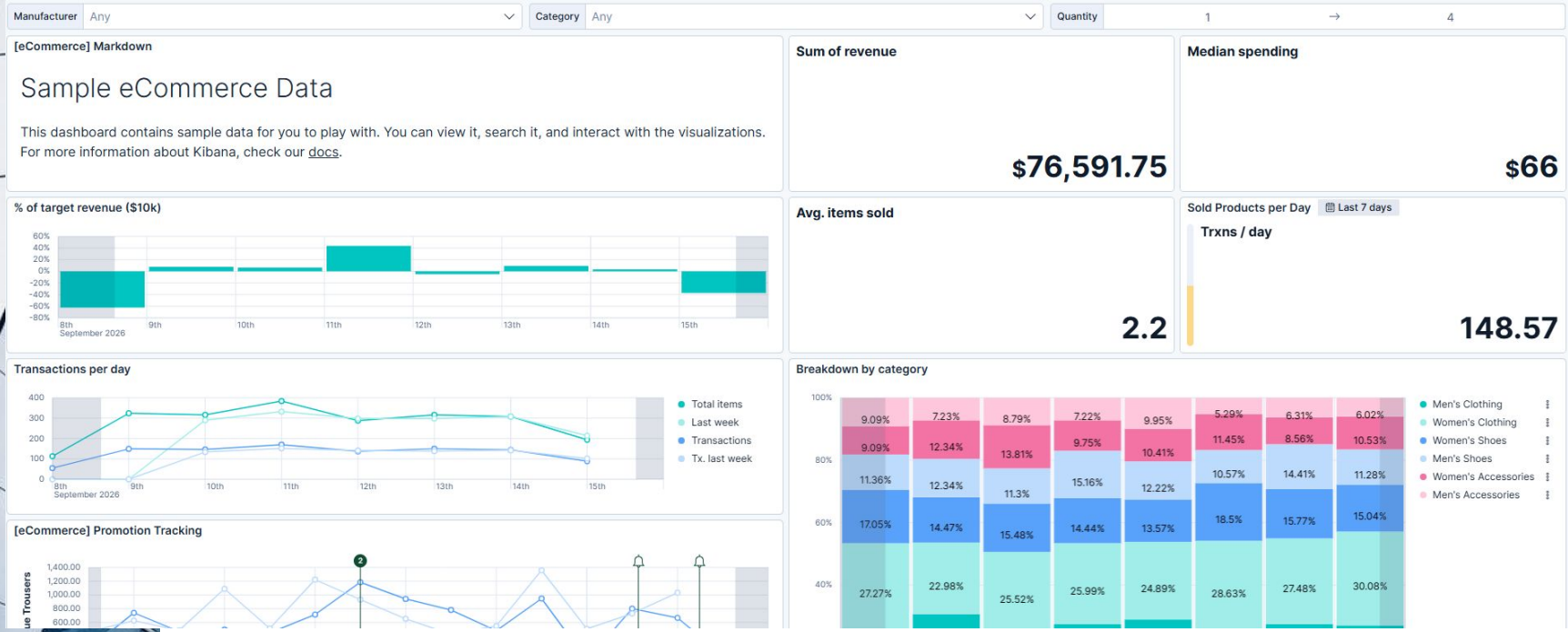
TP 1 : Volume d'activité (Metric)

- **Objectif** : Afficher le nombre total de commandes.
- **Configuration** :
  - Utiliser le type de visualisation **Metric**.
  - Mesure : `Count`.
  - *Optionnel* : Ajouter une comparaison temporelle (**Time Shift**) de 1 semaine pour voir l'évolution.

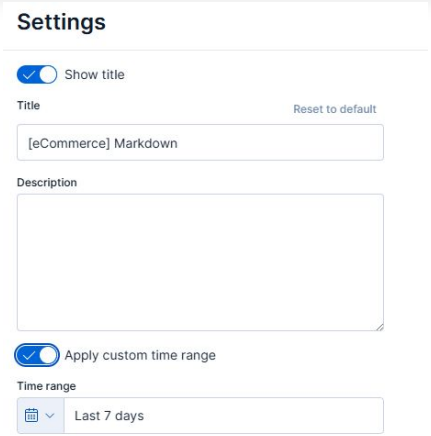
TP 2 : Répartition du CA par Sexe (Donut)

DASHBOARD

Un dashboard est un ensemble de visualisations consolidées dans un même endroit :

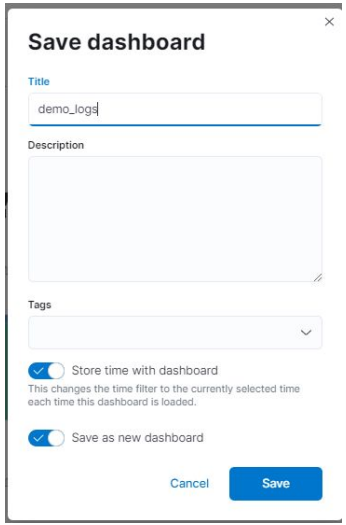


DASHBOARD



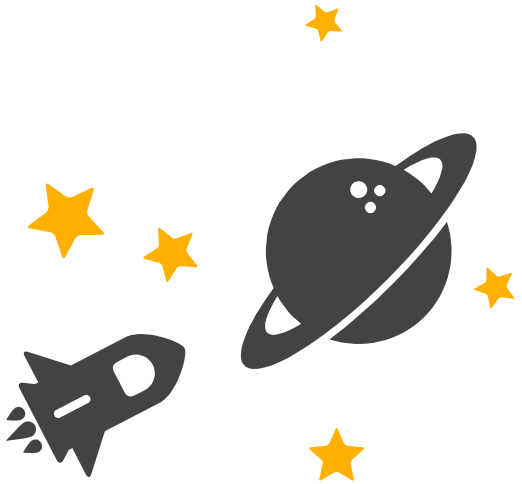
Il est possible de dé-synchroniser l'axe temps pour les composants.

A l'enregistrement, il est possible d'enregistrer le temps dans le tableau de bord. Idéal pour créer des tableaux de bord glissants.





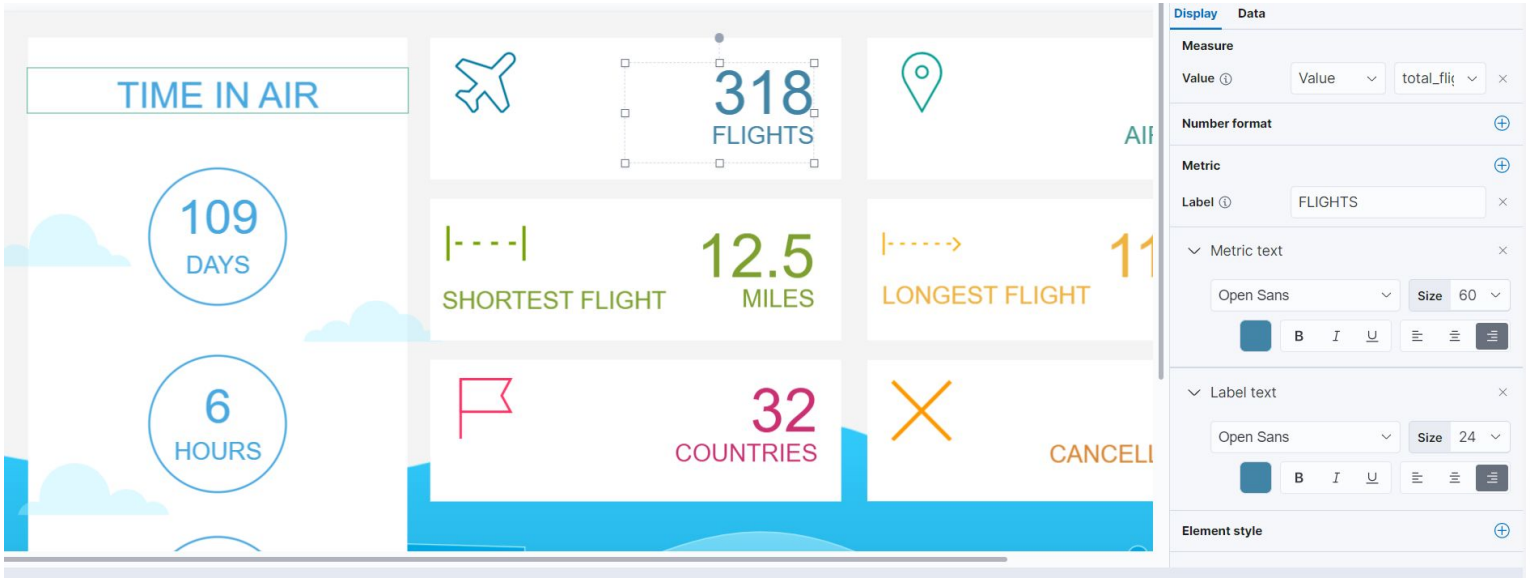
EXERCICE



Créer ses premiers dashboards

CANVAS

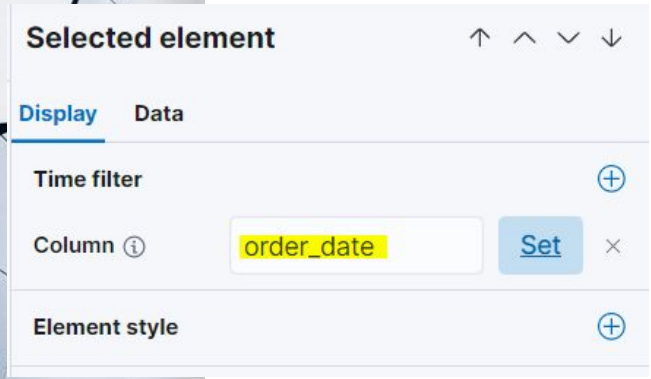
Parfois, il peut être utile d'ajouter davantage d'interactivité dans un tableau de bord. Pour cela, il peut être utile d'utiliser le menu canvas pour se créer ses propres **présentations**. Ces derniers sont notamment faits pour réaliser des présentations animées des données.

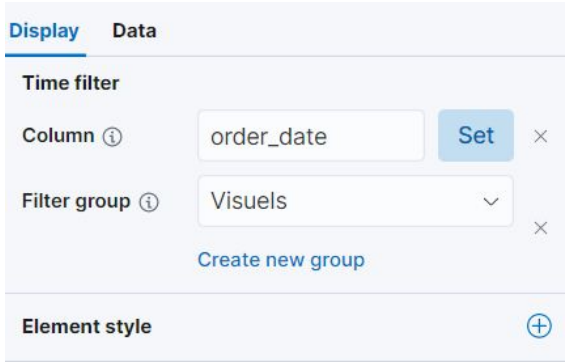


CANVAS : LES FILTRES

Les filtres vont automatiquement fonctionner pour les visuels directement créés dans Canvas, pour peu que ce soit le bon champ temporel qui soit paramétré dans le filtre (Remplacer la valeur par défaut **@timestamp** par votre propre champ temporel).

N'hésitez pas à créer des groupes de filtres pour pouvoir ensuite lier des visualisations issues de Kibana avec le filtre temporel.





CANVAS : LES FILTRES

Quand un visuel issu de Kibana est ajouté, il est filtré sur les 15 dernières minutes par défaut.

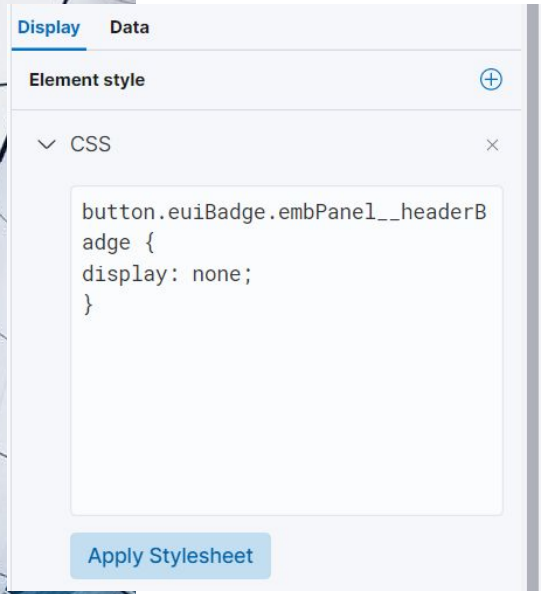
Pour le lier au groupe de filtre précédemment créé, il faut modifier l'expression et ajouter le visuel dans un filtre, comme ceci :

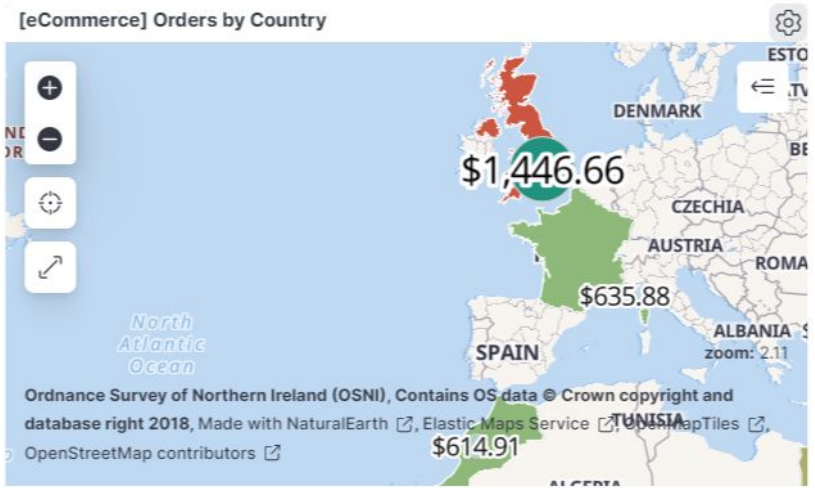
```
1  filters "Visuels"
2  | savedMap id="2c9c1f60-1909-11e9-919b-ffe5949a18d2" center={mapCenter lat=45.88578 lon=-15.07605 zoom=2.11} timerange={timerange from="now-1y/d" to="now"}
3  | render
```

Attention néanmoins : les deux filtres agissent de sorte à afficher les valeurs communes entre les deux filtres (celui de la présentation et celui du visuel)

CANVAS : LES FILTRES

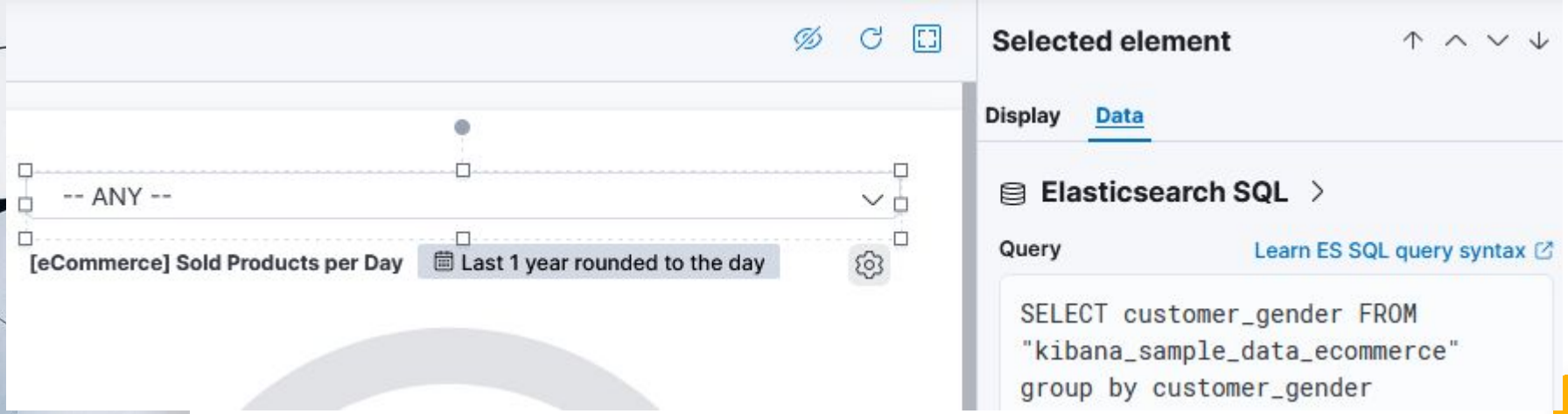
Enfin, il est possible de jouer sur le code CSS du visuel pour masquer la barre de filtre de temps :





CANVAS : LES FILTRES

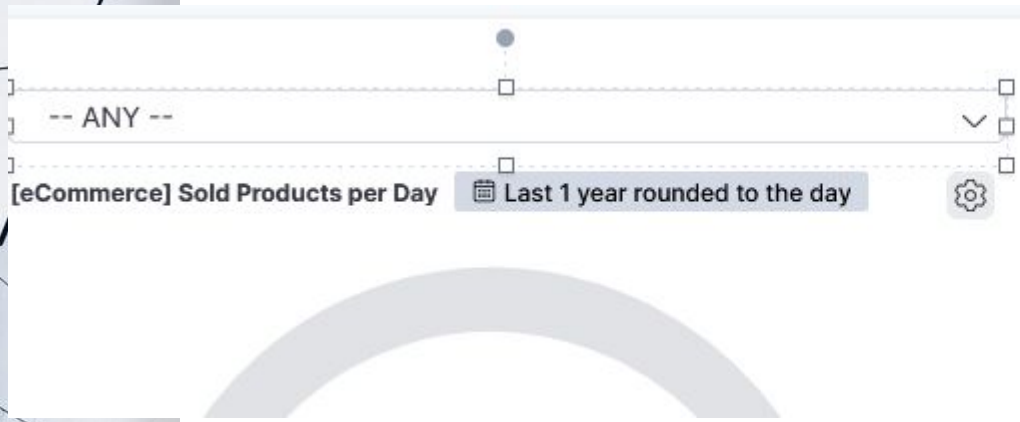
Il est également possible d'ajouter des filtres sous forme de listes déroulantes, comme ceci :

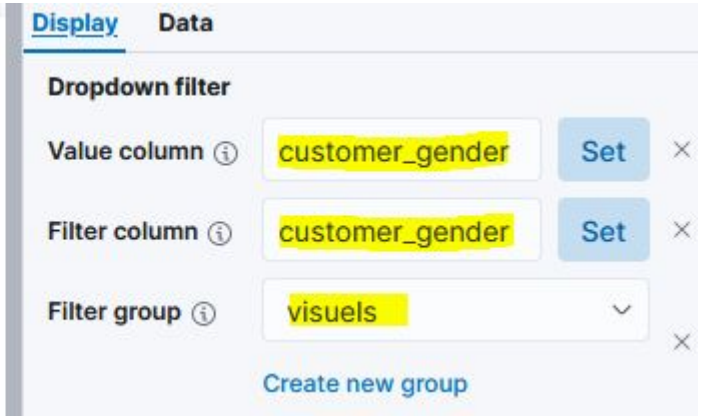


CANVAS : LES FILTRES

Pour que cela fonctionne, il faut bien penser à changer les valeurs dans la partie “Display”.

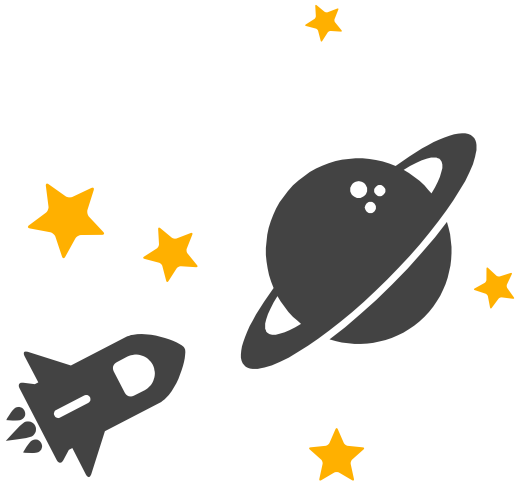
Il est également possible d’ajouter ce filtre au groupe de visuels précédemment créé







EXERCICE



Créer une présentation

DATA : TRANSFORMS

Elasticsearch est une base de données NOSQL orientée document (JSON) et, de ce fait, chaque document doit être vu comme étant indépendant les uns des autres.

Cependant, il peut arriver que la donnée soit interdépendante, comme avec le jeu de données ci-dessous :

```
POST /colis/_bulk
{ "index": {"_id": 1}}
{ "num_colis": "colis_995", "date": "2022-11-18T10:34:00+00:00", "event": "01_Demande créée"}
{ "index": {"_id": 2}}
{ "num_colis": "colis_995", "date": "2022-11-18T10:35:00+00:00", "event": "02_Pris en charge"}
{ "index": {"_id": 3}}
{ "num_colis": "colis_995", "date": "2022-11-19T17:45:00+00:00", "event": "03_Expédié"}
{ "index": {"_id": 4}}
{ "num_colis": "colis_995", "date": "2022-11-20T20:57:00+00:00", "event": "04_Livré"}
{ "index": {"_id": 5}}
{ "num_colis": "colis_996", "date": "2022-11-18T07:32:00+00:00", "event": "01_Demande créée"}
{ "index": {"_id": 6}}
{ "num_colis": "colis_996", "date": "2022-11-25T10:22:00+00:00", "event": "02_Pris en charge"}
{ "index": {"_id": 7}}
{ "num_colis": "colis_996", "date": "2022-11-28T12:15:00+00:00", "event": "03_Expédié"}
```



DATA : TRANSFORMS

Dans cet exemple, on voit des colis avec des dates et des statuts :

- Demande créée
- Pris en charge
- Expédié
- Livré

Il y a donc clairement un lien entre les documents et cela pose des soucis pour répondre à certaines questions orientées métier.

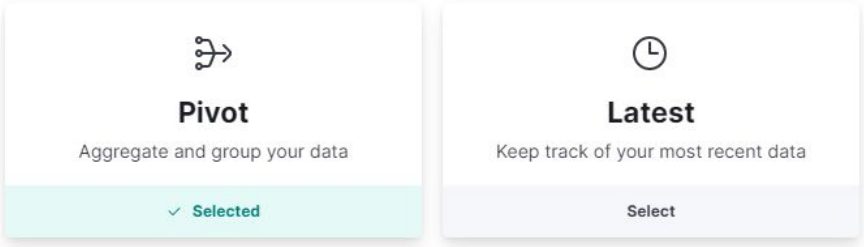
Voici quelques exemples de questions qu'il serait légitime de se poser mais que cette modélisation risque de nous empêcher de résoudre :

- Comment identifier facilement mes colis expédiés mais non livrés?
- Quel est le délai, en jours, entre la prise en charge d'un colis et son expédition?

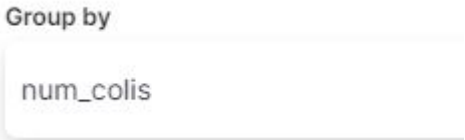
Ces deux exemples de questions supposent un lien fort entre les documents.

DATA : TRANSFORMS

Créer une transformation de type Pivot :



Concernant la partie **Group By**, il faut choisir le champ **num_colis** :



Cette agrégation est de type "**terms**" et fait référence au champ "**num_colis.keyword**".

DATA : TRANSFORMS

Pour les agrégations, plusieurs ont été créées :

- **01_Demande créée** : agrégation de type "**filter**", avec la requête "**term**" sur la valeur "**01_Demande créée**". Cette agrégation permet de savoir si le colis a été créé dans le système.
- **02_Pris en charge** : agrégation de type "**filter**", avec la requête "**term**" sur la valeur "**01_Pris en charge**" avec deux sous agrégations :
  - event.cardinality : le colis est-il pris en charge?
  - date.max : à quelle date le colis a-t-il été pris en charge?
- **03_Expédié** : agrégation de type "**filter**", avec la requête "**term**" sur la valeur "**03_Expédié**" avec deux sous agrégations :
  - event.cardinality : le colis est-il expédié?
  - date.max : à quelle date le colis a-t-il été expédié?
- **04_Livré** : agrégation de type "**filter**", avec la requête "**term**" sur la valeur "**04_Livré**". Cette agrégation permet de savoir si le colis a été livré au destinataire.
- **Date_demande** : agrégation de type "**min**",sur le champ **date**. Cette agrégation permet de trouver la date principale de la donnée (qui servira pour la data view ensuite)

DATA : TRANSFORMS

En synthèse :



DATA : TRANSFORMS

Le résultat obtenu dans l'aperçu (suite à une réorganisation des colonnes) :

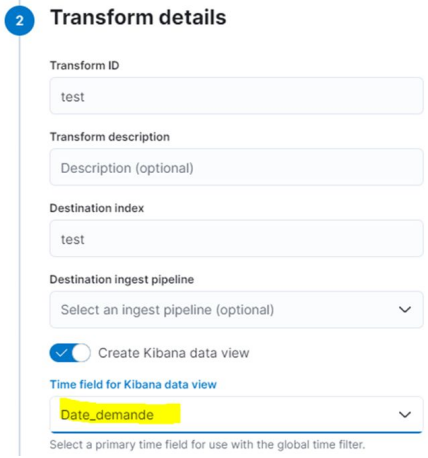
Transform preview

Columns Sort fields

num_colis	01_Demande créée	02_Pris en charge.dat...	02_Pris en charge.eve...	03_Expédié.date.max	03_Expédié.event.car...	04_Livré	Date_demande
colis_995	1	November 18th 2022, 11...	1	November 19th 2022, 18...	1	1	November 18th 2022, 11...
colis_996	1	November 25th 2022, 11...	1	November 28th 2022, 13...	1	0	November 18th 2022, 08...

Avec cette nouvelle vision de la donnée, il va être possible de répondre aux deux cas d'usages précédents!

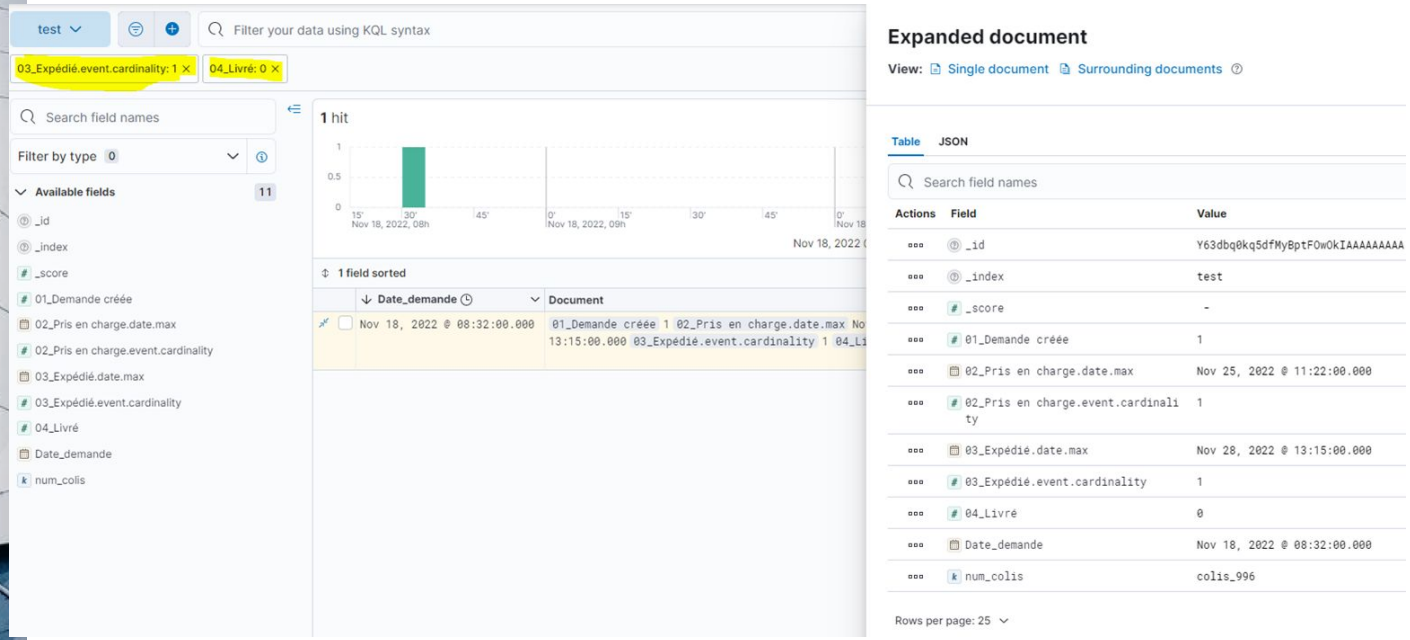
Finir la transformation maintenant :



DATA : TRANSFORMS : CAS D'USAGE 1

Quels sont mes colis expédiés mais non livrés?

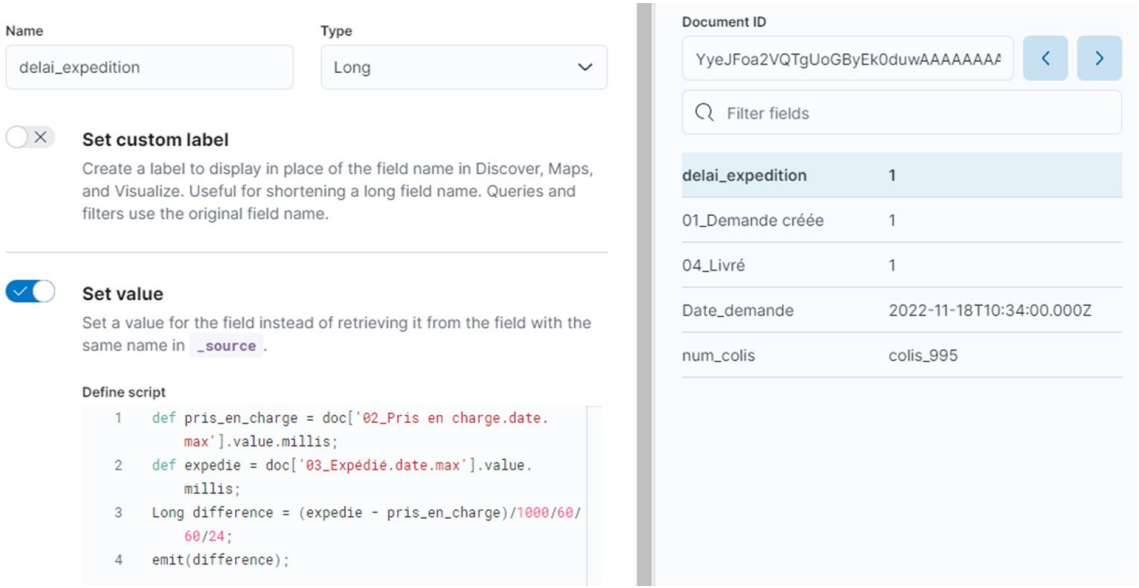
Il est maintenant très simple d'y répondre à l'aide de filtres :



DATA : TRANSFORMS : CAS D'USAGE 2

Quel est le délai, en jours, entre la prise en charge et l'expédition d'un colis?

Pour pouvoir répondre à cette question, il faut d'abord ajouter un Runtime Field qui permet justement de calculer ce délai :



DATA : TRANSFORMS : CAS D'USAGE 2

Quel est le délai, en jours, entre la prise en charge et l'expédition d'un colis?

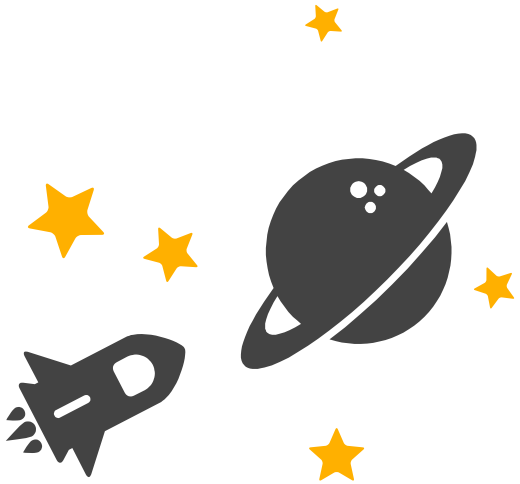
Une fois que ce nouveau champ "**delai_expedition**" existe, il est très simple de surveiller ce délai dans Discover :

Columns 1 field sorted

Date_demande	num_colis	02_Pris en charge.date.max	03_Expédié.date.max	delai_expedition
Nov 18, 2022 @ 11:34:00.000	colis_995	Nov 18, 2022 @ 11:35:00.000	Nov 19, 2022 @ 18:45:00.000	1
Nov 18, 2022 @ 08:32:00.000	colis_996	Nov 25, 2022 @ 11:22:00.000	Nov 28, 2022 @ 13:15:00.000	3



EXERCICE



Créer une transformation de données



11. ALERTING

Mise en place des alertes

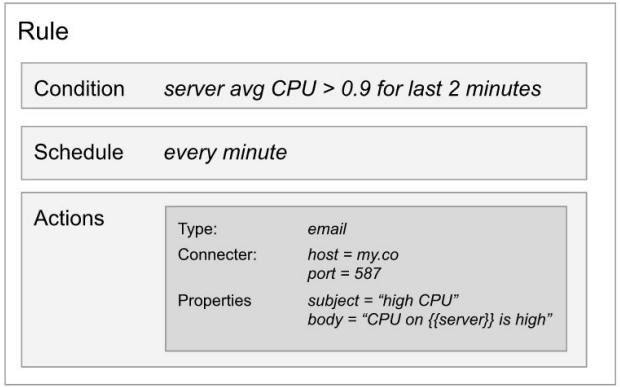
PRINCIPE GENERAL

L'alerting va exécuter de manière **planifiée** des tests sur de la donnée et va déclencher des **actions** définies par des **règles**. Les **connecteurs** permettent de préciser ce que les actions vont faire.

Une règle est constituée de 3 éléments :

- Une **condition** : ce que l'on souhaite détecter
- Une **planification** : définir la périodicité de test de la règle
- Une (ou des) **action**(s) : ce qu'il se passe quand la condition est détectée

Par exemple, si on souhaite envoyer un mail quand la moyenne de consommation de la CPU sur les deux dernières minutes dépasse les 90% :



MISE EN PLACE



L'alerting est activé par défaut dans Kibana mais peut nécessiter une certaine configuration pour pouvoir fonctionner.

Il faut commencer, côté Kibana, par générer une clef d'encryption

```
bin/kibana-encryption-keys generate
```

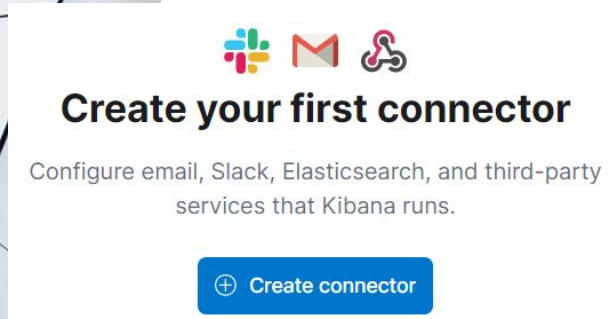
Cette clef sera ensuite installée, côté Kibana, dans le fichier kibana.yml :

```
xpack.encryptedSavedObjects.encryptionKey: 7caf9d09d24fee77dcf8916d1b708bcd
xpack.reporting.encryptionKey: ce8b256b1aa7108b6cf896c4eeefbb4c
xpack.security.encryptionKey: 26aed80f53756c66c79417ce3df42666
```

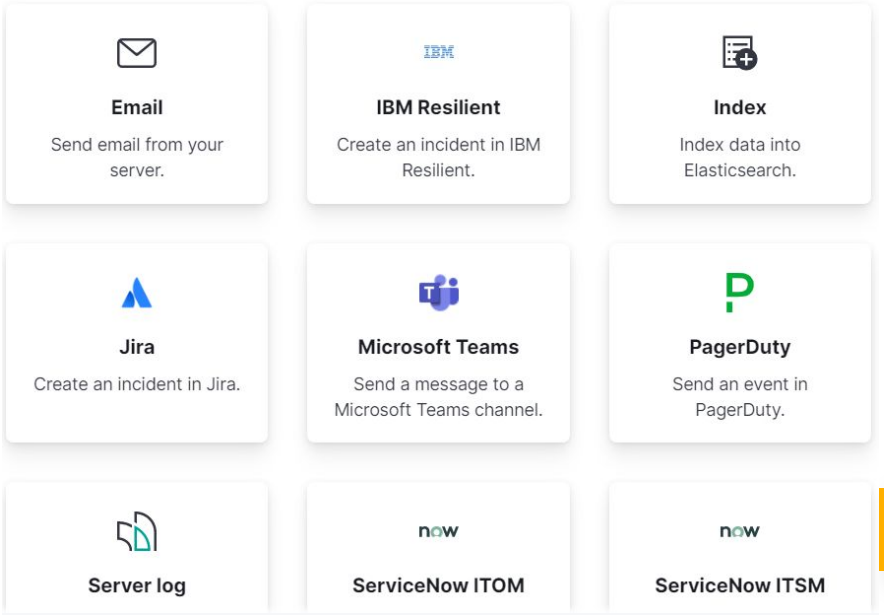
Il est bien sûr possible de stocker les token dans un keystore côté Kibana, afin que les clefs ne soient pas lisibles telles quelles dans le fichier Kibana.yml

CRÉATION DE CONNECTEURS

Il est possible de créer des connecteurs auxquelles les alertes pourront envoyer des messages :

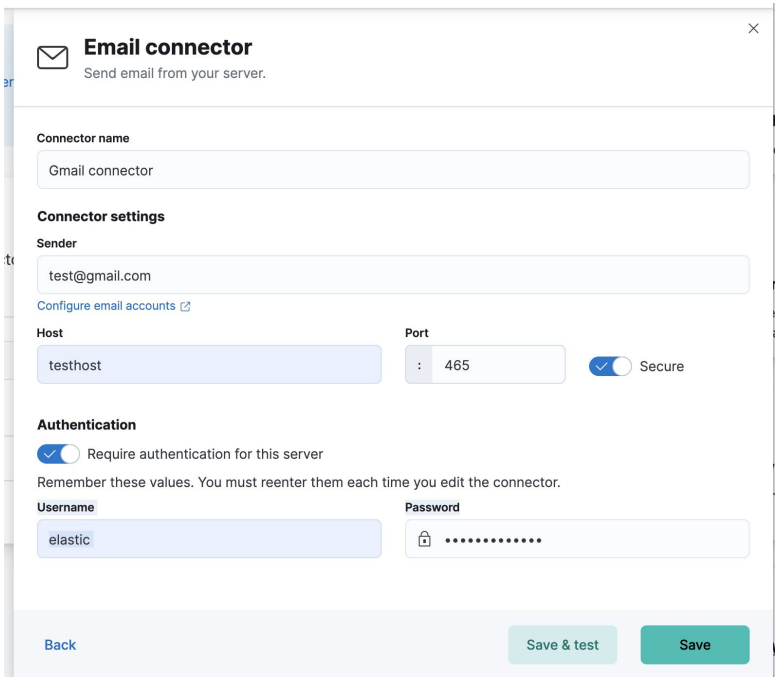






CRÉATION DE CONNECTEURS

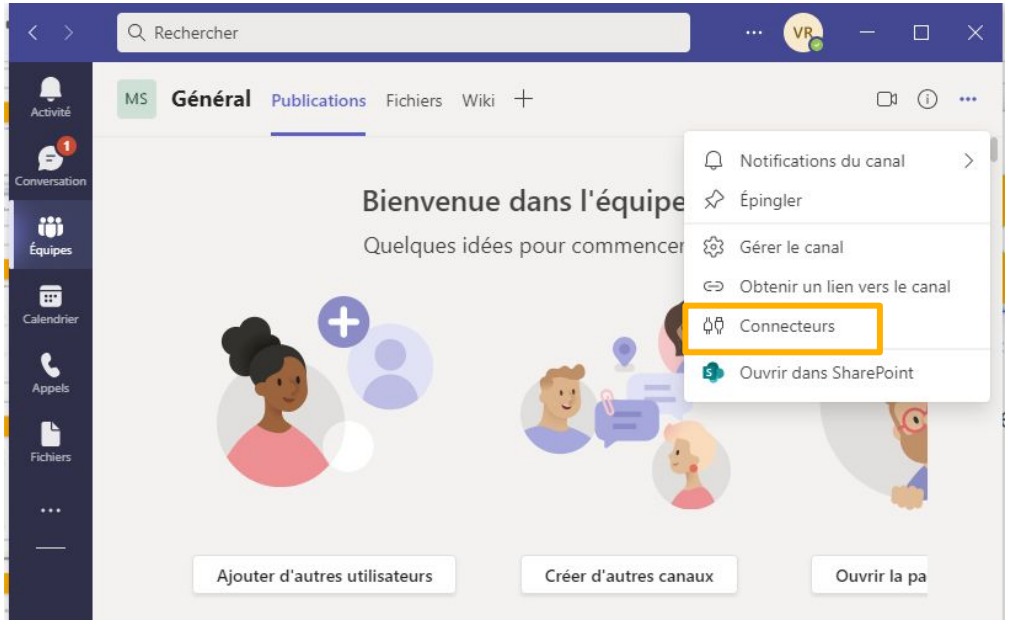
Exemple de création d'un connecteur de type "Email" :



CRÉATION DE CONNECTEURS

Exemple de création d'un connecteur de type "Teams".

1ère étape, dans Teams, il faut créer l'URL Webhook qui va servir pour Kibana :



CRÉATION DE CONNECTEURS

Exemple de création d'un connecteur de type "Teams".

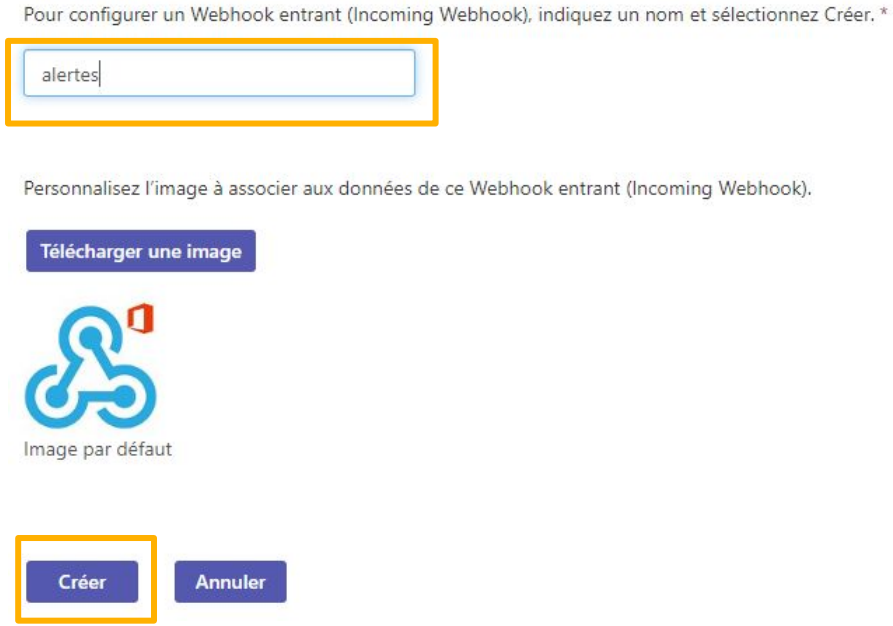
1ère étape, dans Teams, il faut créer l'URL Webhook qui va servir pour Kibana :



CRÉATION DE CONNECTEURS

Exemple de création d'un connecteur de type "Teams".

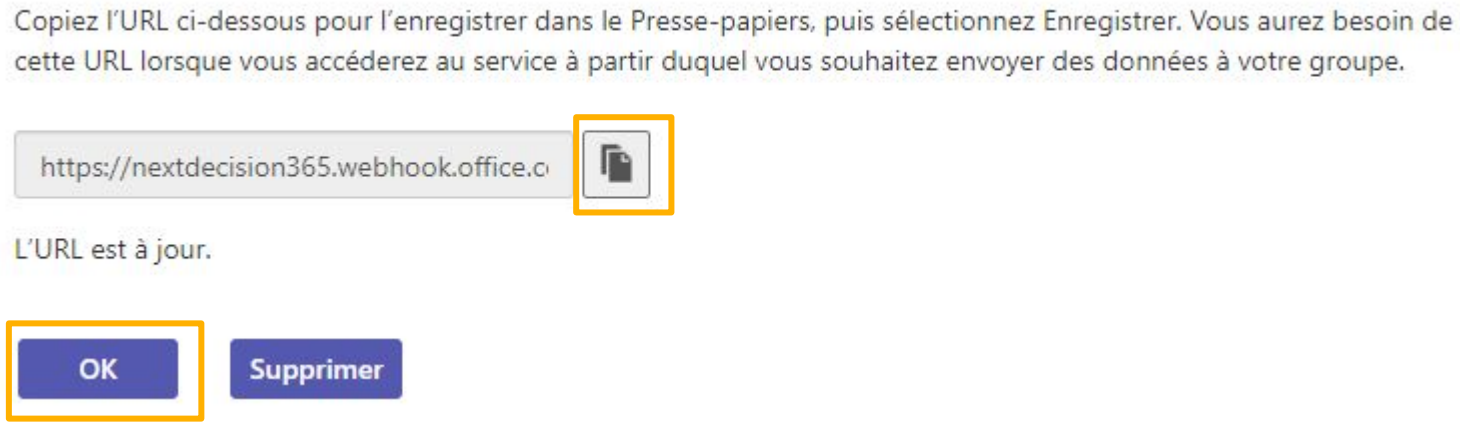
1ère étape, dans Teams, il faut créer l'URL Webhook qui va servir pour Kibana :



CRÉATION DE CONNECTEURS

Exemple de création d'un connecteur de type "Teams".

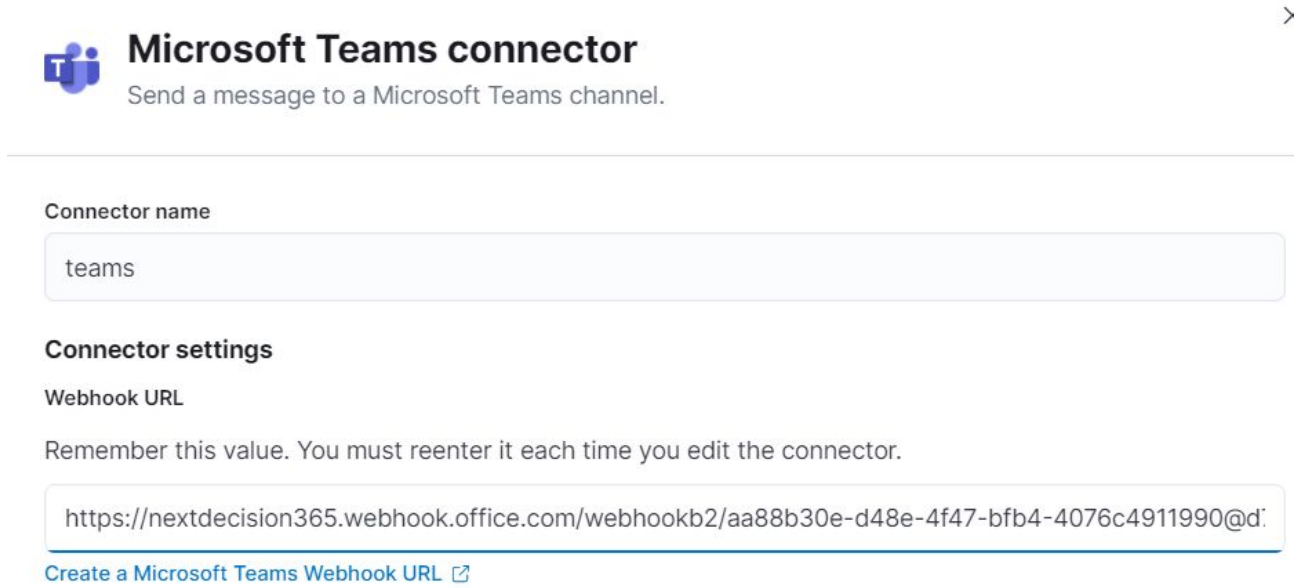
1ère étape, dans Teams, il faut créer l'URL Webhook qui va servir pour Kibana :



CRÉATION DE CONNECTEURS

Exemple de création d'un connecteur de type "Teams".

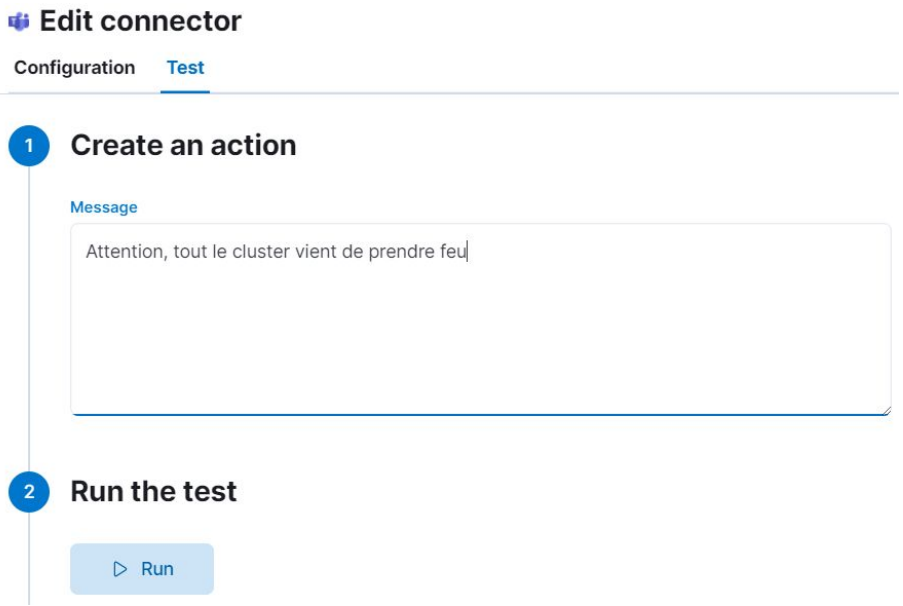
Maintenant que l'URL est connue, il est possible de créer le connecteur dans Kibana :



CRÉATION DE CONNECTEURS

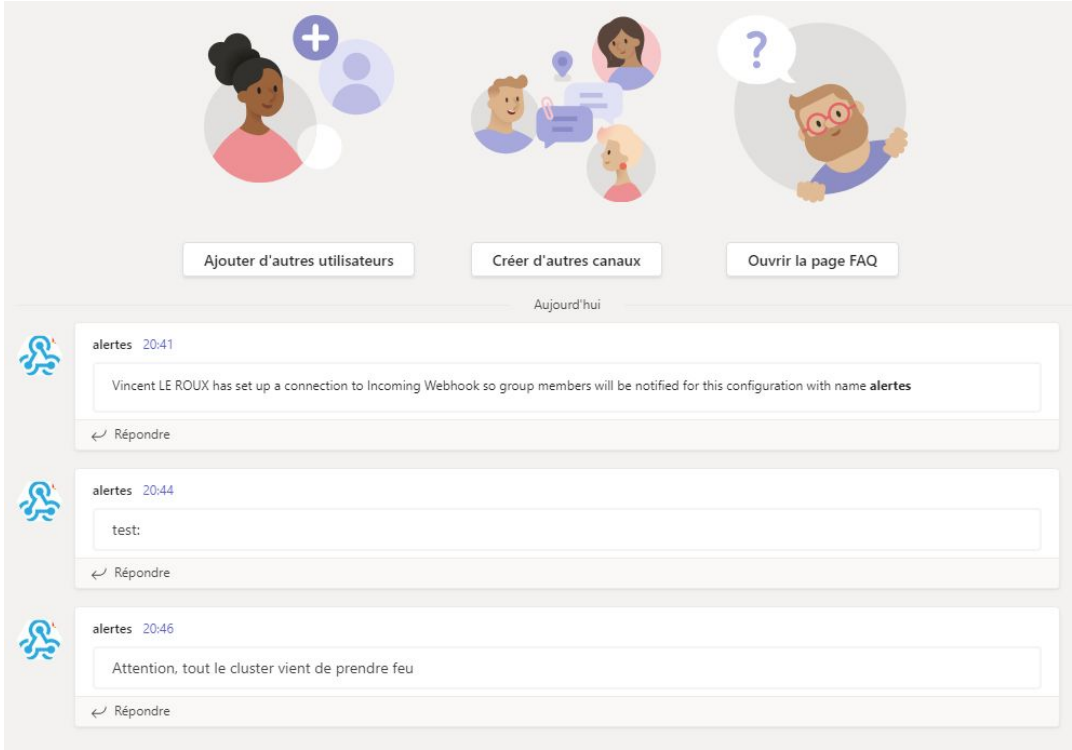
Exemple de création d'un connecteur de type "Teams".

Test du connecteur :



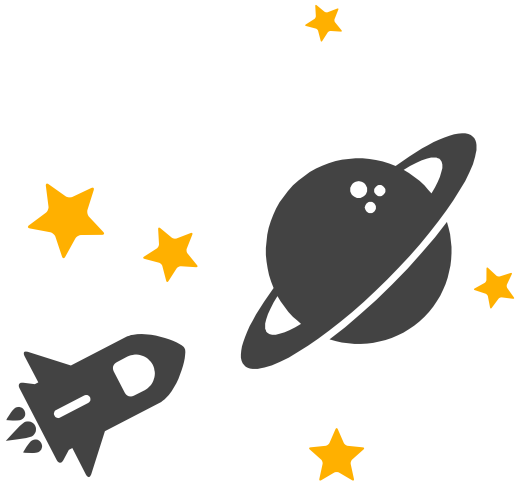
CRÉATION DE CONNECTEURS

Exemple de création d'un connecteur de type "Teams".





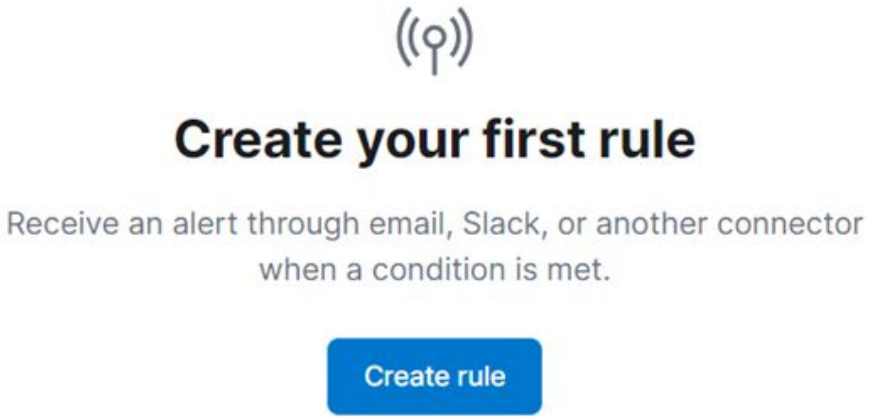
EXERCICE



Créer une alerte

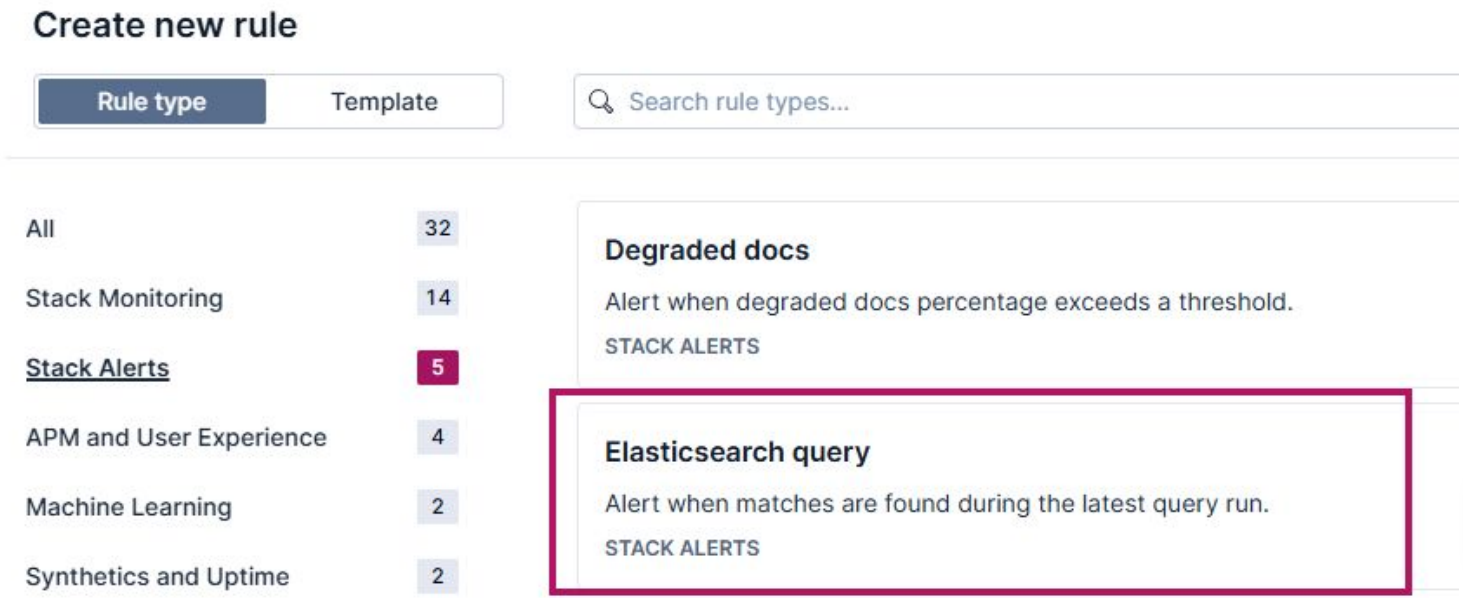
CRÉATION D'UNE RÈGLE

Dans le menu "Stack Management" > "Rules and Connectors", créer votre première règle :



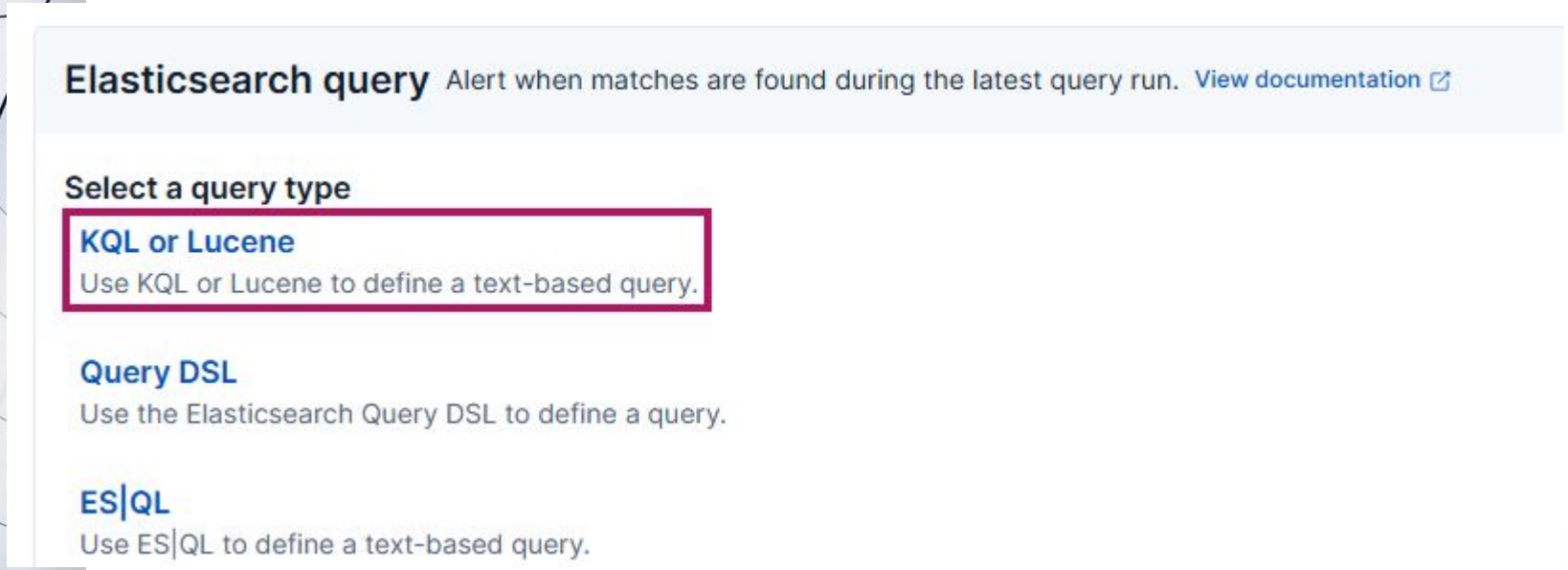
CRÉATION D'UNE RÈGLE

Elastic propose plusieurs règles pré-configurées. Ici, nous allons choisir la règle Stack Rules > Elasticsearch Query :



CRÉATION D'UNE RÈGLE

On choisit ensuite le langage de recherche :



CRÉATION D'UNE RÈGLE

On fait le filtre et on définit les règles de détection :



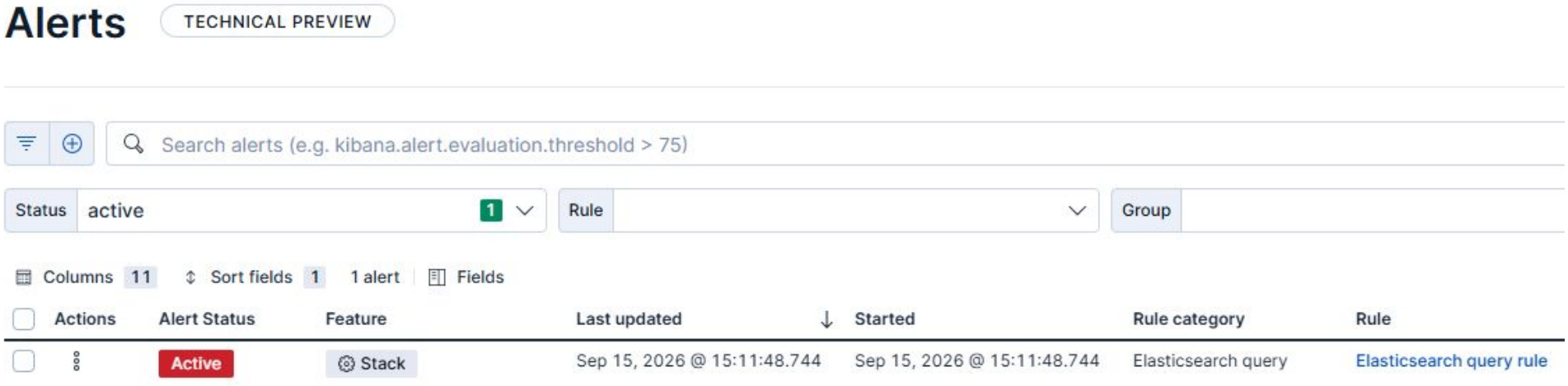
CRÉATION D'UNE RÈGLE

Puis on définit la fréquence d'exécution de la règle :



CRÉATION D'UNE RÈGLE

On peut voir ensuite toutes les alertes actives :



“RULES AND CONNECTORS” VS WATCHER

Dans Kibana, un second module permet de créer des alertes : Watcher.

Ce module est plus ancien que “Rules and Connectors”. Voici les grandes différences entre ces deux modules :

- Watcher permet plus de choses, avec notamment la possibilité d’intervenir sur le document JSON créé directement
- Rules and Connectors est plus simple d’accès pour les utilisateurs
- Rules and Connectors propose déjà des règles toutes prêtes à l’emploi
- Rules and Connector est déclenché par Kibana alors que les watcher sont lancées par Elasticsearch



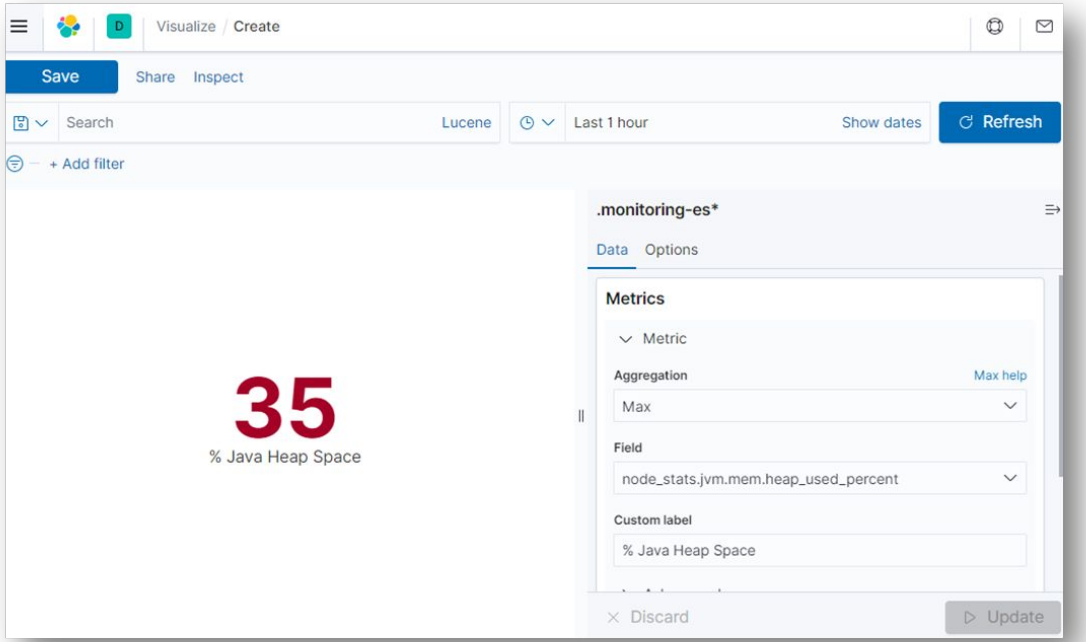


12. REPORTING

Partager ses propres tableaux de bords

REPORTING

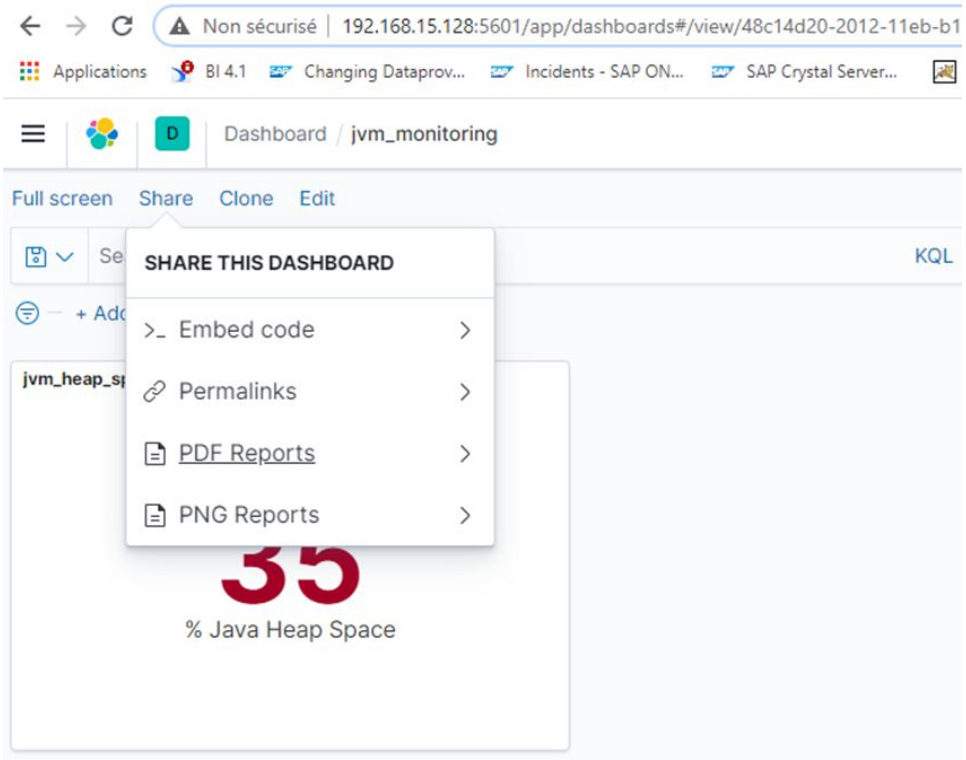
Commencer par créer un dashboard à distribuer :



REPORTING

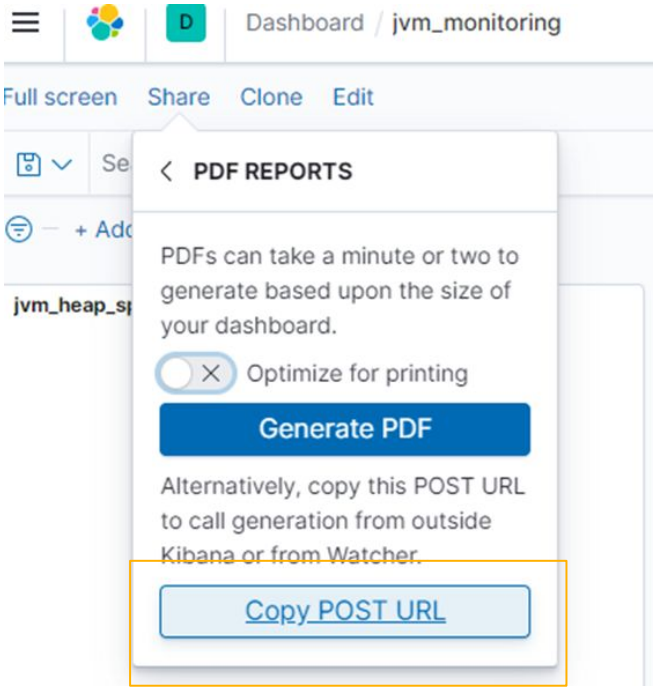
Il est possible, sur le dashboard, de le partager :

- Avec un lien direct
- En PDF
- En PNG



REPORTING

En PDF, il est possible de demander à générer le PDF de suite (cela se fera en tâche d'arrière plan), ou bien de copier l'URL qui permet la génération du PDF (utile pour embarquer dans une alerte de type watcher) :



REPORTING

Pour intégrer une pièce jointe dans le mail envoyé, il faut passer par la création de l'alerte au format DSL :

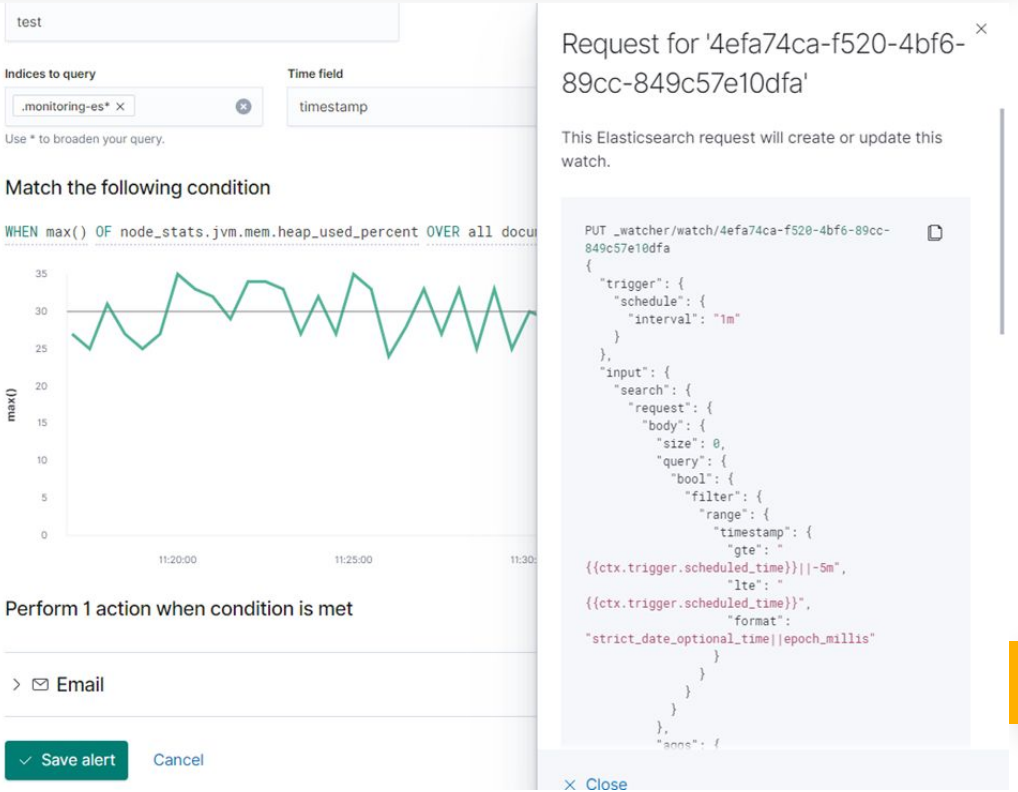
```
PUT _watcher/watch/jvm_heap_space
{
  "trigger" : {
    "schedule": {
      "interval": "1h"
    }
  },
  "actions" : {
    "email_admin" : {
      "email": {
        "to": "vincent.leroux@nextdecision.fr",
        "subject": "Warning : JVM HEAP SPACE",
        "attachments" : {
          "jvm_heap_space.pdf" : {
            "reporting" : {
              "url": "http://192.168.15.128:5601/api/reporting/generate
              /printablePdf?jobParams
              =%28browserTimezone%3AEurope%2FParis%2Clayout%3A%28dimensions%3A%28
              height%3A288%2Cwidth%3A1280%29%2Cid%3Apreserve_layout%29%2CobjectTy
              pe%3Adashboard%2CrelativeUrls%3A%21%28%27%2Fapp%2Fdashboards%23%2Fv
              iew%2F48c14d20-2012-11eb-b1e4
              -e33108b9fbf8%3F_g%3D%28filters%3A%21%21%28%29%2CrefreshInterval%3A
              %28pause%3A%21%21f%2Cvalue%3A10000%29%2Ctime%3A%28from%3Anow
              -1h%2Cto%3Anow%29%29%26_a%3D%28description%3A%21%27%21%27%2Cfilters
              %3A%21%21%28%29%2CfullScreenMode%3A%21%21f%2Coptions%3A%28hidePanel
              Titles%3A%21%21f%2CuseMargins%3A%21%21t%29%2Cquery%3A%28language%3A
              kuery%2Cquery%3A%21%27%21%27%29%2CtimeRestore%3A%21%21t%2Ctitle%3Aj
              vm_monitoring%2CviewMode%3Aview%29%27%29%2Ctitle%3Ajvm_monitoring%2
              9",
              "retries":40,
              "interval":"15s"
            }
          }
        }
      }
    }
  }
}
```

```
{
  "_id" : "jvm_heap_space",
  "_version" : 1,
  "_seq_no" : 740,
  "_primary_term" : 3,
  "created" : true
}
```

REPORTING

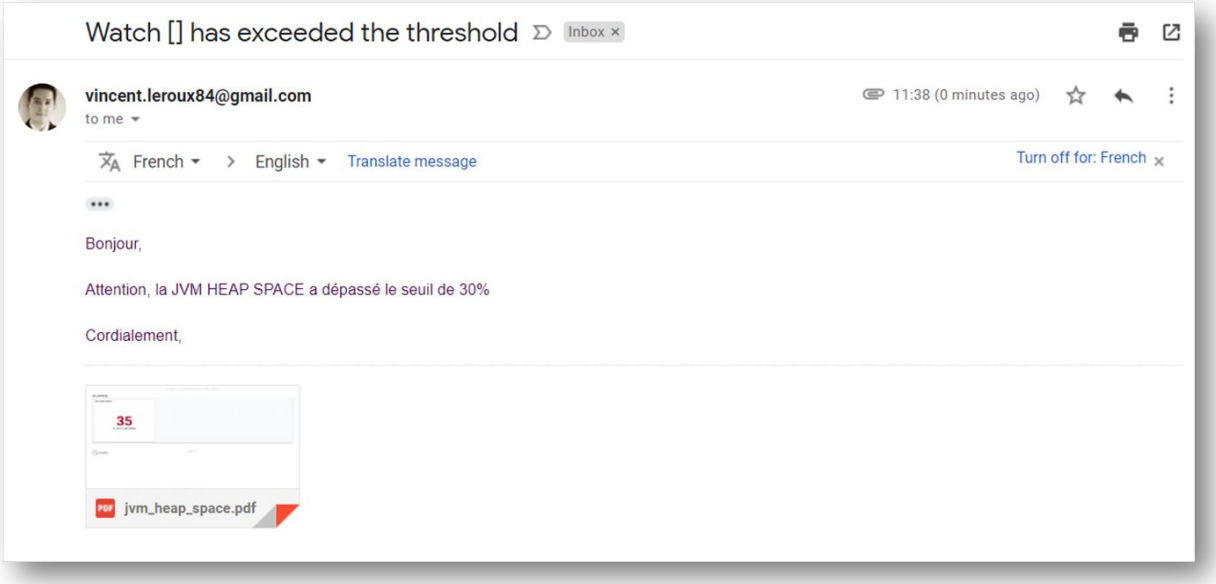
Sur une alerte de type Watcher, il est possible, via l'interface graphique, de retrouver le script de création de l'alerte :

(Pratique pour obtenir facilement la requête DSL!)



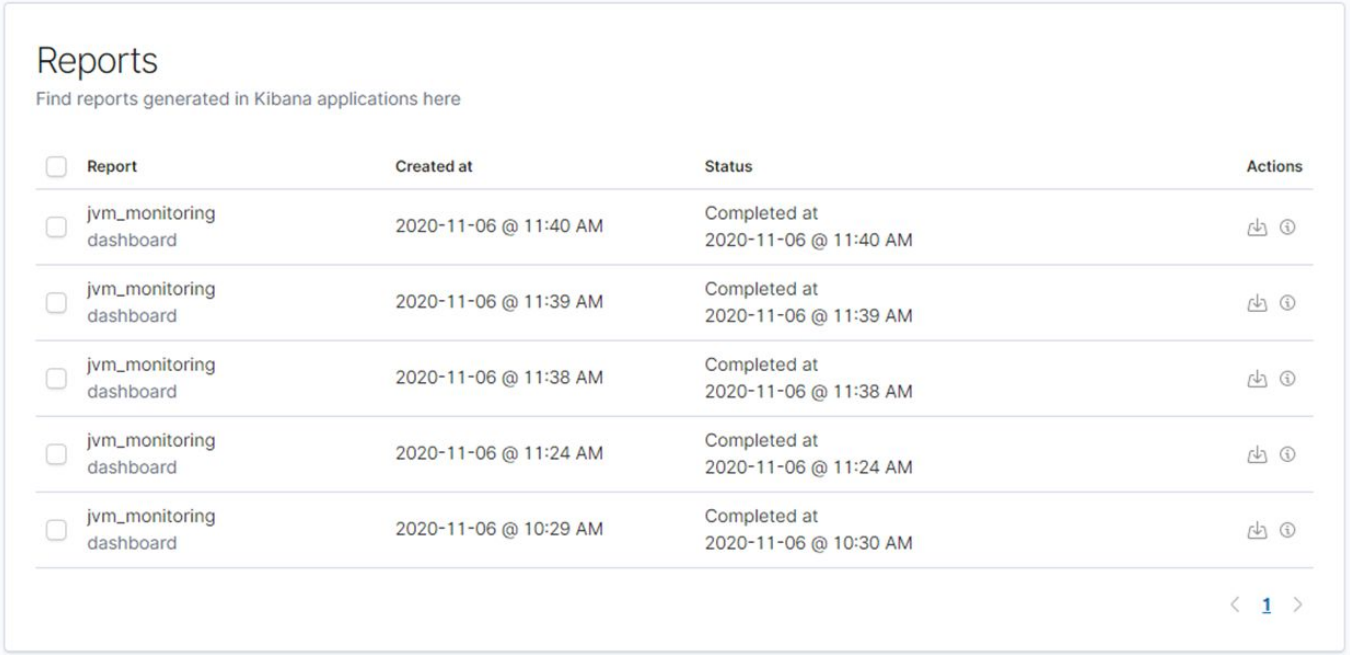
REPORTING

Résultat obtenu :



REPORTING

Il est ensuite possible de voir les rapports qui ont été générés, soit de manière planifiée, soit suite à une alerte :



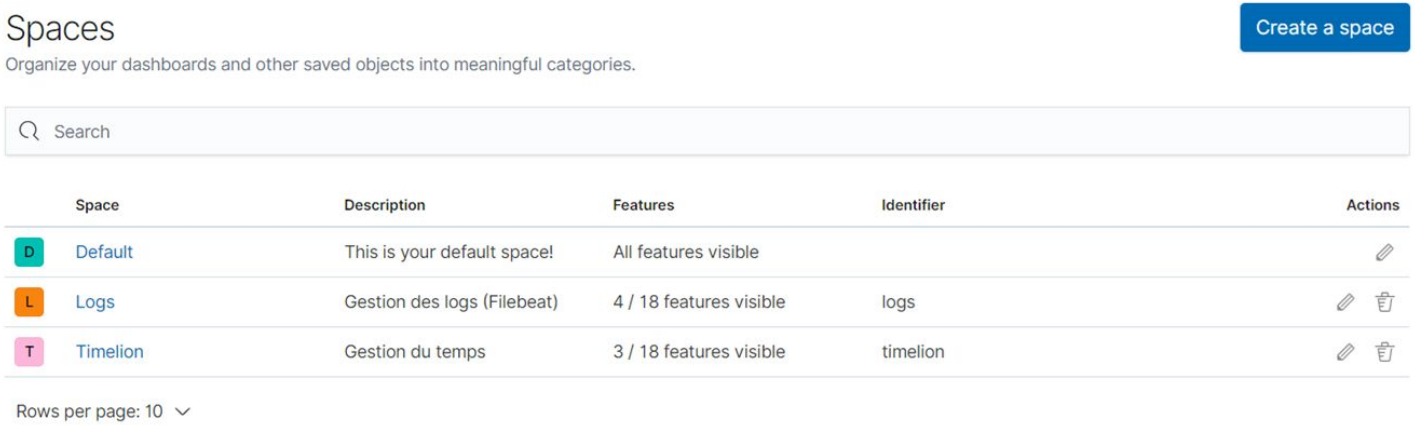


13. GESTION DES ESPACES

Discrimination par projets

GESTION DES ESPACES

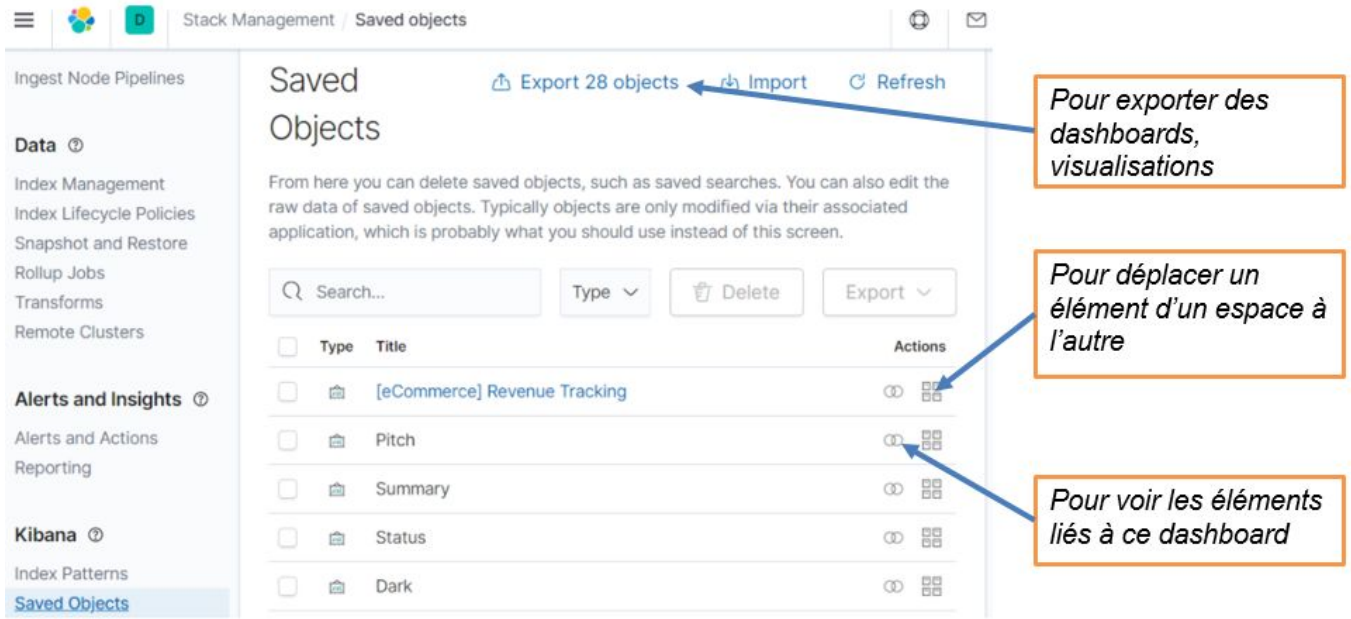
Par défaut, le menu D montre toutes les applications possibles disponibles dans Kibana. Avec l'aide des espaces, il devient alors possible de cloisonner les applications selon des espaces plus logiques :



Cela permet d'avoir une vision plus claire des différentes briques, le tout découpé par domaine fonctionnel ou selon l'attribution de droits à des groupes d'utilisateurs.

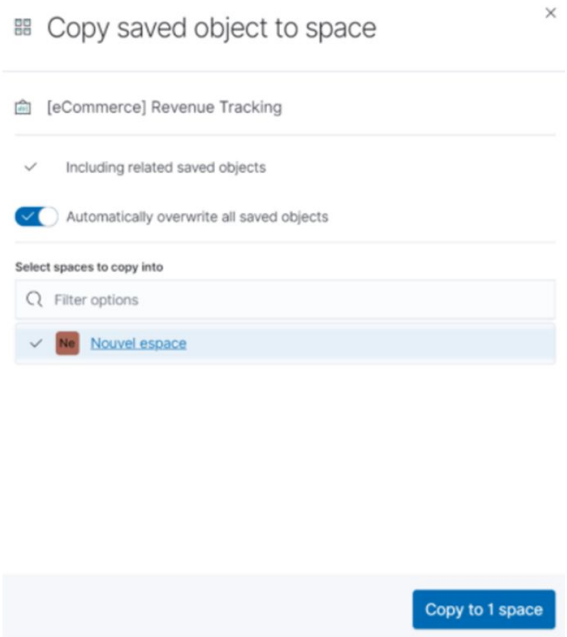
DÉPLACER DES OBJETS / SAUVEGARDER

Depuis le menu « Stack Management > Saved objects », il est possible de réaliser des actions sur les dashboards / visualisations :



DÉPLACER DES OBJETS / SAUVEGARDER

Depuis le menu « Stack Management > Saved objects », il est possible de réaliser des actions sur les dashboards / visualisations :



Il devient alors possible de déplacer un dashboard (ou une visualisation) d'un espace de travail à un autre

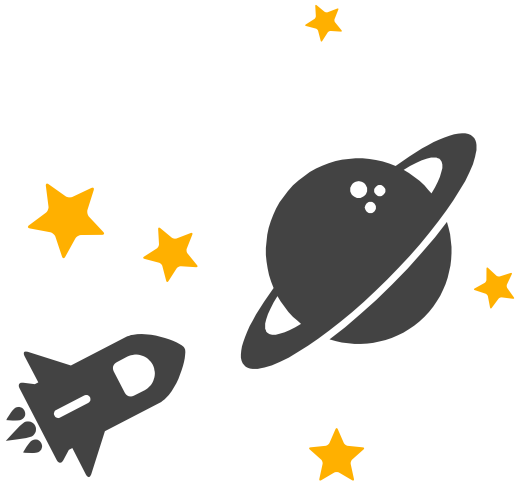


14. DATA ANALYSE

Graphes et Machine Learning



DATA ANALYSE

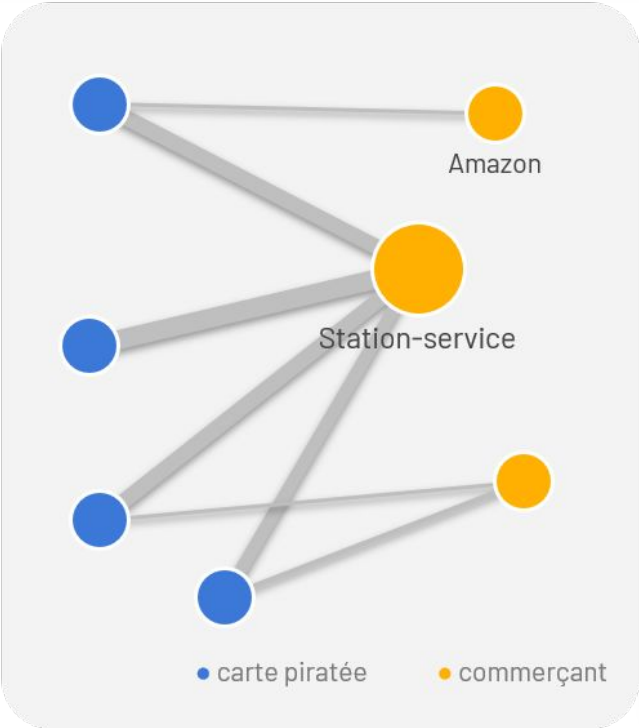


Les Graphes

GRAPH : LE PRINCIPE

Graph n'est pas une base orientée graphes (comme Neo4j). Rien n'est stocké en nœuds et relations : Graph **déduit** les relations à partir de documents ordinaires, au moment de la requête.

- **Sommet (vertex) :** une valeur d'un champ : une IP, un client, une marque...
- **Lien (edge) :** deux valeurs présentes ensemble dans les mêmes documents
- **Poids :** la significativité du lien, pas sa fréquence brute





POPULAIRE OU SIGNIFICATIF ?

Graph s'appuie sur l'agrégation **`significant_terms`** : elle compare un **premier plan** (les documents de la requête) à un **arrière-plan** (tout l'index).

Exemple : part de chaque commerçant dans les transactions

Amazon

Fréquence brute élevée

Cartes piratées	**30 %**
Tout l'index	**28 %**

Autant présent partout : populaire, mais n'apprend rien.

Station-service

Fréquence brute faible

Cartes piratées	**12 %**
Tout l'index	**0,01 %**

Sur-représentée ×1 200 : c'est le point de compromission.

COMMENT LE GRAPHE SE CONSTRUIT



1

Requête

sélectionne les documents de départ

category : "Women's Shoes"

2

Échantillon

les documents les plus pertinents

100 par shard par défaut

3

Sommets

valeurs sur-représentées de chaque champ

significant_terms

4

Liens

valeurs liées à chaque sommet

co-occurrence

5

Extension

on repart des nouveaux sommets

bouton Expand = 1 saut

Graph ne lit pas toutes les données : il travaille sur un échantillon. Un lien absent ne prouve pas l'absence de relation.

L'API _GRAPH/EXPLORE



```
POST kibana_sample_data_ecommerce/_graph/explore
{
  "query": { "match": {
      "category": "Women's Shoes" } },
  "controls": {
      "use_significance": true,
      "sample_size": 500 },
  "vertices": [
    { "field": "manufacturer.keyword",
      "size": 5, "min_doc_count": 3 } ],
  "connections": { "vertices": [
    { "field": "category.keyword",
      "size": 5, "min_doc_count": 3 } ] }
}
```

`query`
le premier plan : les documents de départ

`controls`
significativité et taille de l'échantillon

`vertices`
les champs des sommets de départ

`connections`
les champs des sommets liés

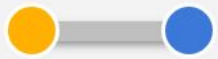
Réponse
weight = score de significativité ; depth = saut ; doc_count = documents communs

LES RÉGLAGES DANS KIBANA

Graph > Settings : ces cinq réglages changent complètement le résultat

Réglage	Rôle	Effet en pratique
Significant links	Activer la significativité	Désactivé : ce qui est populaire. Activé : ce qui est caractéristique
Certainty	Documents minimum pour un lien (3)	Évite les liens fondés sur une coïncidence
Sample size	Documents analysés par shard	Plus grand = plus complet, mais plus lent
Diversity field	Plafond de documents par valeur	Un robot ou un client très actif n'écrase plus l'échantillon
Max terms per hop	Sommets ajoutés par extension	Garde le graphe lisible

INTERPRÉTER UN GRAPHE



Lien épais

Association forte et significative entre deux valeurs



Communauté

Groupe isolé : clients aux mêmes achats, IP qui visent les mêmes URL



Hub

Sommet très connecté : point commun, commerçant compromis, serveur C2



Bruit

Valeur sans intérêt (« GET », « Mozilla ») : à exclure avec Block

Toujours vérifier : un clic sur un lien ouvre les documents correspondants dans Discover.



LES PRÉCAUTIONS

1. **Un échantillon, pas un inventaire**

   Le résultat peut varier, et un lien absent ne prouve rien.

2. **Une corrélation n'est pas une cause**

   Graph formule des hypothèses ; Discover les confirme ou les infirme.

3. **Des champs agrégeables**

   keyword, ip, nombres. Un champ text ne peut pas servir de sommet : retour au mapping.

4. **Une licence Platinum ou Enterprise**

   Graph n'est pas disponible en Basic.



CAS D'USAGE

Fraude

`carte / commerçant`

Trouver le point de compromission commun aux cartes piratées

Recommandation

`produit / produit`

« Ceux qui ont acheté X ont aussi acheté Y »

Sécurité

`IP / URL / user-agent`

Isoler une campagne d'attaque

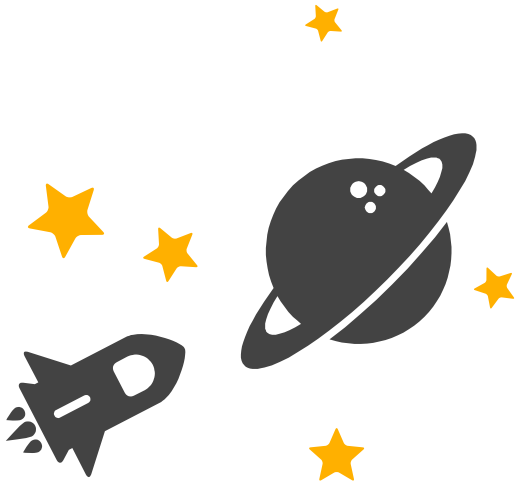
Analyse de logs

`erreur / serveur / version`

Remonter à la cause d'un incident



DATA ANALYSE



Le Machine Learning

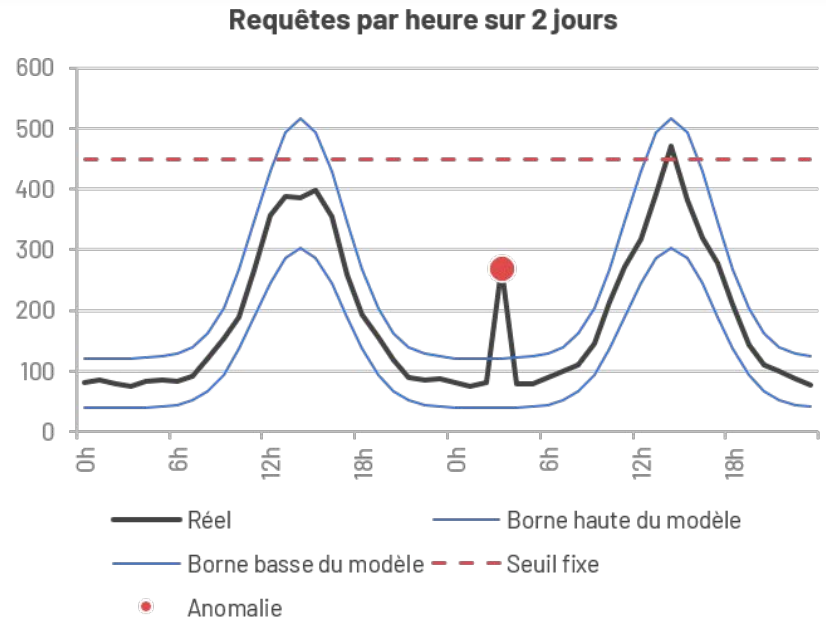
LE PRINCIPE : APPRENDRE LE NORMAL

Non supervisé : aucune étiquette, aucun seuil à fixer.

Le modèle apprend le **comportement normal** d'une série temporelle : tendance, cycles quotidiens et hebdomadaires.

Il signale ce qui est **statistiquement improbable** par rapport à ce qu'il a appris.

Un seuil fixe se trompe dans les deux sens ; le modèle s'adapte à l'heure et au jour.



● **3h :** 270 requêtes, sous le seuil mais hors du modèle

LES BRIQUES D'UN JOB



1

Index source

logs, métriques...

`logs-*`

▶

2

Datafeed

lit l'index et alimente le job

`historique puis temps réel`

▶

3

Job

le modèle : détecteurs, influenceurs

`sur un nœud ml`

▶

4

Résultats

scores stockés dans un index

`.ml-anomalies-*`

bucket_span : la taille des tranches de temps analysées (15 min par défaut)

Détecteur : une fonction appliquée à un champ : count, mean(bytes), rare(process)...

Influenceur : un champ qui aide à expliquer l'anomalie : l'IP, le pays, l'hôte

LES FONCTIONS DE DÉTECTION



Famille	Fonctions	Exemple de question
Volume	`count, high_count, low_count`	Le nombre de requêtes s'effondre-t-il ?
Métrique	`mean, max, sum, high_mean…`	Le temps de réponse moyen explose-t-il ?
Cardinalité	`distinct_count, high_distinct_count`	Une IP visite-t-elle beaucoup trop d'URL ?
Rareté	`rare, freq_rare`	Un processus jamais vu apparaît-il ?
Temps	`time_of_day, time_of_week`	Un utilisateur se connecte-t-il à 3h du matin ?
Géographie	`lat_long`	Une connexion vient-elle d'un lieu inhabituel ?

Les préfixes **`high_`** et **`low_`** ne signalent que les hausses ou que les baisses.

DÉCOUPER L'ANALYSE

Trois manières de découper un détecteur par les valeurs d'un champ



`partition_field`

Un modèle par valeur

Chaque serveur a sa propre normale. Idéal pour comparer des entités qui n'ont rien à voir.

partition : host.name



`by_field`

Une série par valeur

Chaque code HTTP est suivi séparément, mais les scores sont normalisés ensemble.

by : http.response.status_code



`over_field`

Comparé aux pairs

Chaque entité est comparée à ses pairs, pas à son propre passé.

over : source.ip



LES ASSISTANTS DE CRÉATION

1 Single metric

Une seule série : le volume global de logs

2 Multi-metric

Plusieurs détecteurs et un découpage par partition

3 Population

Chaque entité comparée à ses pairs (over)

4 Catégorisation

Regroupe les messages de logs en motifs, puis compte chaque motif

5 Rare

Valeurs rarement ou jamais vues

6 Avancé

Configuration complète, JSON compris

Configurations fournies : des jobs prêts à l'emploi pour les intégrations et les jeux de démo.



BUCKET_SPAN ET APPRENTISSAGE

15 min

bucket_span par défaut

Trop court : du bruit. Trop long : les pics courts disparaissent dans la moyenne.

Semaines

pour un cycle hebdo

Le modèle doit voir plusieurs cycles avant de les reconnaître.

0 → 100

un modèle qui mûrit

Les premières heures, il manque de recul : les scores se stabilisent ensuite.

Aider le modèle

Calendriers : déclarer les jours fériés, soldes ou maintenances pour ne pas les apprendre comme normaux.

Règles personnalisées : ignorer les anomalies sans enjeu, par exemple « moins de 10 requêtes ».

query_delay : laisser aux données le temps d'arriver avant de les analyser.

LIRE UN SCORE D'ANOMALIE



Un score de 0 à 100, calculé à partir de la probabilité et normalisé par rapport à l'historique du job

Warning	Minor	Major	Critical
3 – 25	25 – 50	50 – 75	75 – 100

Actual / typical

La valeur observée et celle que le modèle attendait. L'écart est la première chose à lire.

Trois niveaux de score

Bucket (la tranche de temps), record (l'anomalie précise), influencer (l'entité en cause).

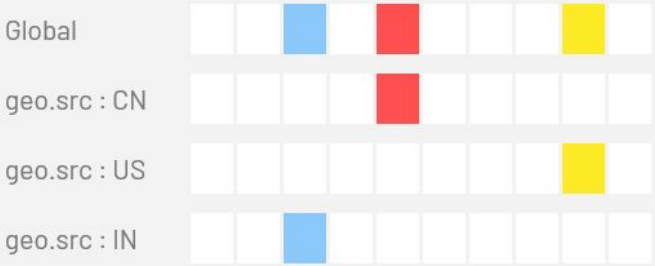
Un score relatif

Un 90 veut dire « parmi les plus rares de ce job », pas « grave pour le métier ».

LES VUES DE RÉSULTATS

Anomaly Explorer

Des bandes colorées par période, globales puis par influenceur



Un clic sur une case affiche le tableau des anomalies : actual, typical, influenceurs.

Single Metric Viewer

La série réelle dans la zone de confiance du modèle



Un point hors de la zone bleue est une anomalie. On peut aussi y lancer une prévision (Forecast).

CRÉER UN JOB PAR L'API



```
PUT _ml/anomaly_detectors/tp-logs-volume
{ "analysis_config": {
    "bucket_span": "1h",
    "detectors": [ { "function": "count" } ],
    "influencers": [ "geo.src" ] },
  "data_description": {
    "time_field": "@timestamp" } }

PUT _ml/datafeeds/datafeed-tp-logs-volume
{ "job_id": "tp-logs-volume",
  "indices": [ "kibana_sample_data_logs" ] }

POST _ml/anomaly_detectors/tp-logs-volume/_open
POST _ml/datafeeds/datafeed-tp-logs-volume/_start
```

`anomaly_detectors`
le job : ce qu'on analyse et comment

`datafeeds`
d'où viennent les données

`_open / _start`
charger le modèle, puis lancer la lecture

Résultats
`GET _ml/anomaly_detectors/<job>/results/records`



LES BONNES PRATIQUES

1. **Une anomalie n'est pas un incident**
   C'est un écart statistique : on enquête avant de conclure.

2. **Partir d'une question métier**
   Un job ciblé (« les ventes chutent-elles ? ») vaut mieux qu'un job qui surveille tout.

3. **Alerter sur les scores élevés**
   Règle Kibana sur un score ≥ 75, plutôt que sur chaque anomalie.

4. **Prévoir les ressources**
   Nœuds ml dédiés et mémoire du modèle (model_memory_limit) à surveiller.

5. **Licence Platinum ou Enterprise**
   Non disponible en Basic ; un essai de 30 jours suffit pour la formation.



15. POUR ALLER PLUS LOIN

ILM, les options payantes, Monitoring, etc.

X-PACK

Par défaut, Kibana et Elasticsearch ne sont soumis à sécurité. Cela veut dire que n'importe qui, qui dispose de l'URL de KIBANA, peut s'y connecter et réaliser des analyses.

Sur un environnement de production, il est nécessaire de sécuriser l'accès à la plateforme. La solution : **X-PACK**

X-PACK est un plugin pour Elasticsearch et Kibana, avec certaines parties gratuites et d'autres payantes.



GRATUIT vs PAYANT



Récemment, une partie des fonctionnalités de X-PACK sont devenues gratuites (Monitoring du cluster, un peu de sécurité, etc.)
Voici la liste des fonctionnalités encore payantes présentes dans X-PACK :

Sécurité

- Authentification à l'aide d'un annuaire LDAP ou Active Directory, SSO (SAML, Kerberos, etc.)
- Sécurité aux niveaux des champs (Sécurité à la ligne)
- Sécurité aux niveaux des documents (Sécurité à l'objet)

Alertes

- Alertes automatiques en cas de problèmes sur le cluster
- Notifications par e-mail, Slack, Jira, etc. (Notion d'Alerting)

Data Science

- Exploration de graphes (Data Mining)
- Machine Learning (Détection d'anomalies sur des séries temporelles, prédictif, etc.)

Autres

- Clients JDBC et ODBC
- Partages aux formats PDF et PNG

GRATUIT vs PAYANT

Souscriptions Elastic (self-managed) : ce que chaque niveau ajoute

Les niveaux sont cumulatifs : Platinum contient tout Basic, Enterprise contient tout Platinum

BASIC

Gratuit · sans support

SÉCURITÉ DE BASE

- ✓ TLS, utilisateurs natifs, rôles (RBAC), clés d'API
- ✓ Espaces Kibana

GESTION DES DONNÉES

- ✓ ILM, rollover, data streams, data tiers
- ✓ Snapshots / restauration
- ✓ Cross-cluster search

EXPLORATION

- ✓ Discover, Lens, Dashboards, Maps
- ✓ ES|QL, Query DSL, SQL (API REST)
- ✓ Export CSV, Stack Monitoring

ALERTING

- ✓ Règles Kibana, connecteurs *Index* et *Server log* uniquement

PLATINUM

Payant · facturé par nœud

Tout Basic +

⚠ Plus proposé aux nouveaux clients (clients existants uniquement)

SÉCURITÉ AVANCÉE

- ✓ Sécurité au niveau champ / document
- ✓ SSO : SAML, OIDC, LDAP / AD, Kerberos, PKI
- ✓ Audit logging, filtrage IP

ALERTING & REPORTING

- ✓ Connecteurs e-mail, Slack, Teams, webhook, Jira…
- ✓ Watcher
- ✓ Rapports PDF / PNG

ANALYSE & HAUTE DISPO

- ✓ Machine Learning : détection d'anomalies
- ✓ Cross-cluster replication (PRA)
- ✓ Graph, clients JDBC / ODBC

SUPPORT

- ✓ Support Elastic 24/7

ENTERPRISE

Payant · facturé à la RAM (ERU)

Tout Platinum +

STOCKAGE BAS COÛT

- ✓ Searchable snapshots : phases cold et frozen sur S3 / Azure / GCS
- ✓ Synthetic `_source` : plein gain de `logsdb` (8.17+)

IA & RECHERCHE

- ✓ Déploiement de modèles : ELSER, E5 (`semantic_text`), NLP
- ✓ AI Assistant, AIOps

SOLUTIONS & ORCHESTRATION

- ✓ Fonctions avancées Observability / Security
- ✓ ECE / ECK (orchestration des clusters)

SUPPORT

- ✓ Support le plus étendu

Gold a été retiré du catalogue. Les opérations cross-cluster exigent le même niveau de licence sur tous les clusters. Liste simplifiée : la grille de référence reste elastic.co/subscriptions, qui évolue à chaque version.

SECURISER KIBANA / ELASTICSEARCH

Pour mettre en place la sécurité par défaut, il faut activer l'option **xpack.security.enabled** (désactivée par défaut) dans le fichier elasticsearch.yml :



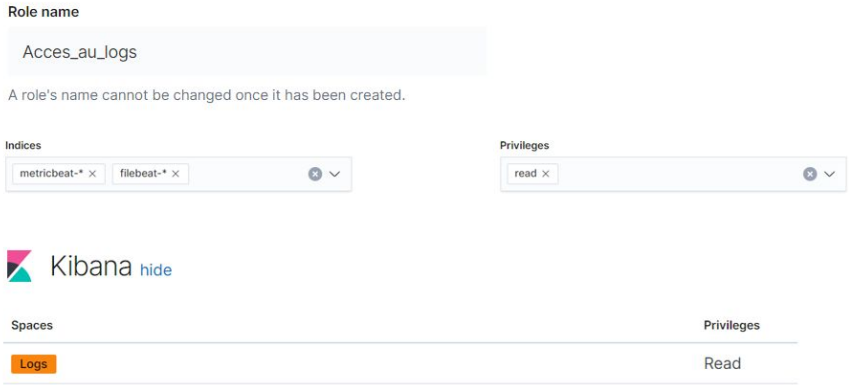
Attention : cette option doit être positionnée à true sur tous les nœuds du cluster. Egalement, si votre cluster contient plus d'un nœud, il faut mettre en place la sécurité TLS/SSL

Pour plus d'informations : https://www.elastic.co/guide/en/elasticsearch/reference/current/configuring-security.html)

SECURISER KIBANA / ELASTICSEARCH

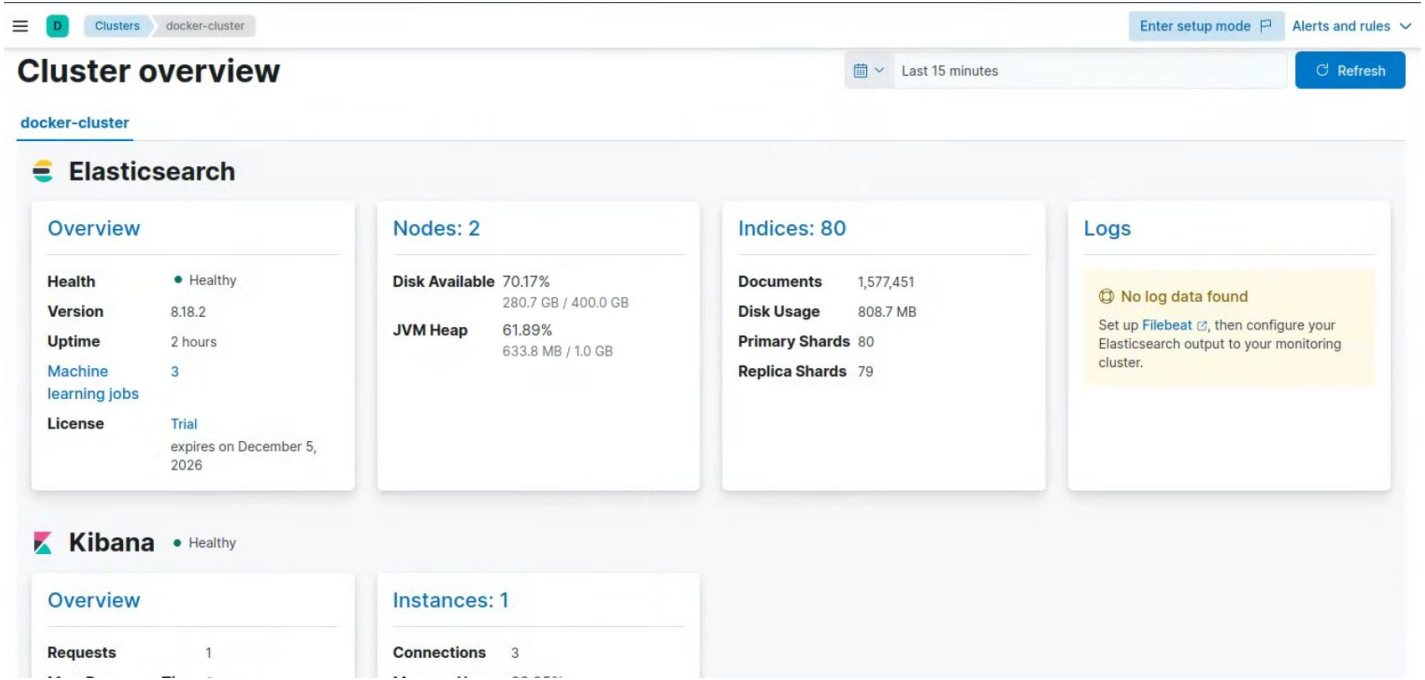
Il est ensuite possible :

- De créer des utilisateurs dans Kibana
- De mettre en place du SSO (connecter Kibana à un AD / LDAP)
- Créer des rôles dans Kibana (pour autoriser les accès à des espaces, des index particuliers, des applications particulières, etc.)
- Réaliser un mapping entre des groupes AD et des rôles Kibana
- etc.



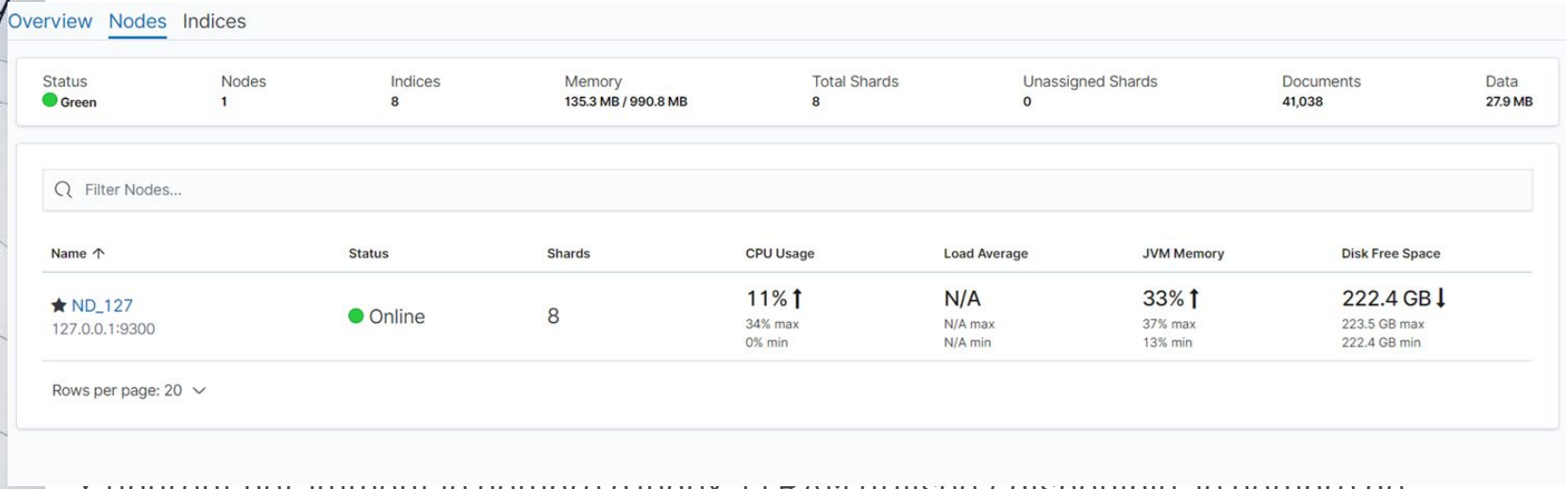
MONITORING

Dans son ancienne offre, il fallait se servir des API disponibles pour pouvoir monitorer son cluster ou s'offrir X-PACK. Aujourd'hui, le monitoring est gratuitement présent dans Kibana :



MONITORING

En cliquant sur menu "Nodes", on obtient des informations sur les nœuds que comprend le cluster :

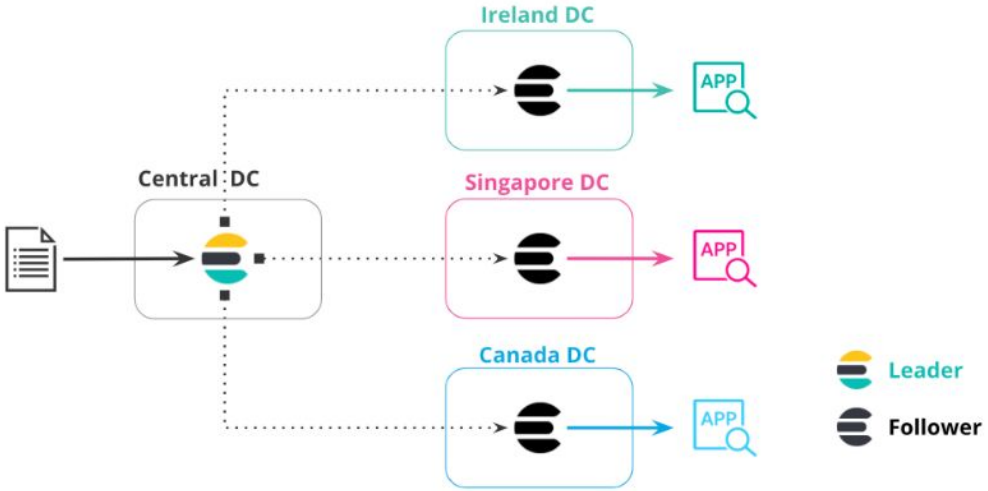


Y figurent notamment le nombre d'index, la RAM utilisée / disponible, le nombre de documents, l'espace disque occupé / disponible, la charge CPU ainsi que la charge de la JVM.

CROSS-CLUSTERING

Avec X-PACK, il est aussi possible de mettre en place du Cross-Clustering. Cela permet de synchroniser des index sur des cluster Elastic différents.

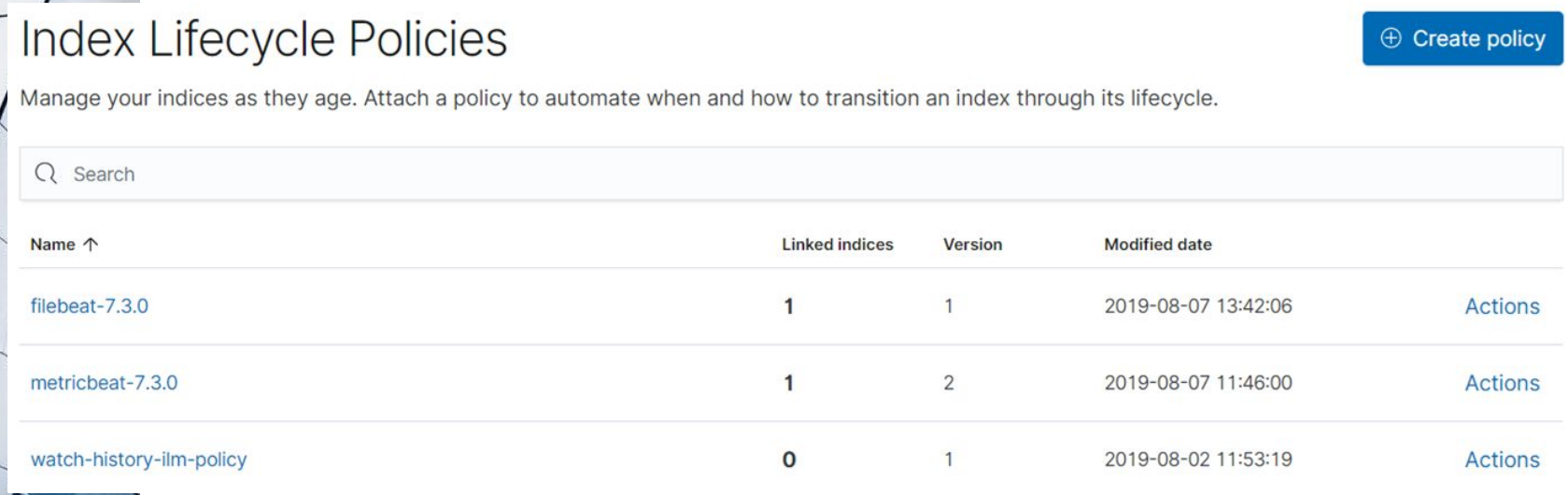
Cela peut permettre de mettre en place un PRA, ou bien de synchroniser des clusters géographiquement locaux et rapprocher la donnée des utilisateurs, comme dans l'exemple ci après :



CYCLE DE VIE DE LA DONNÉE

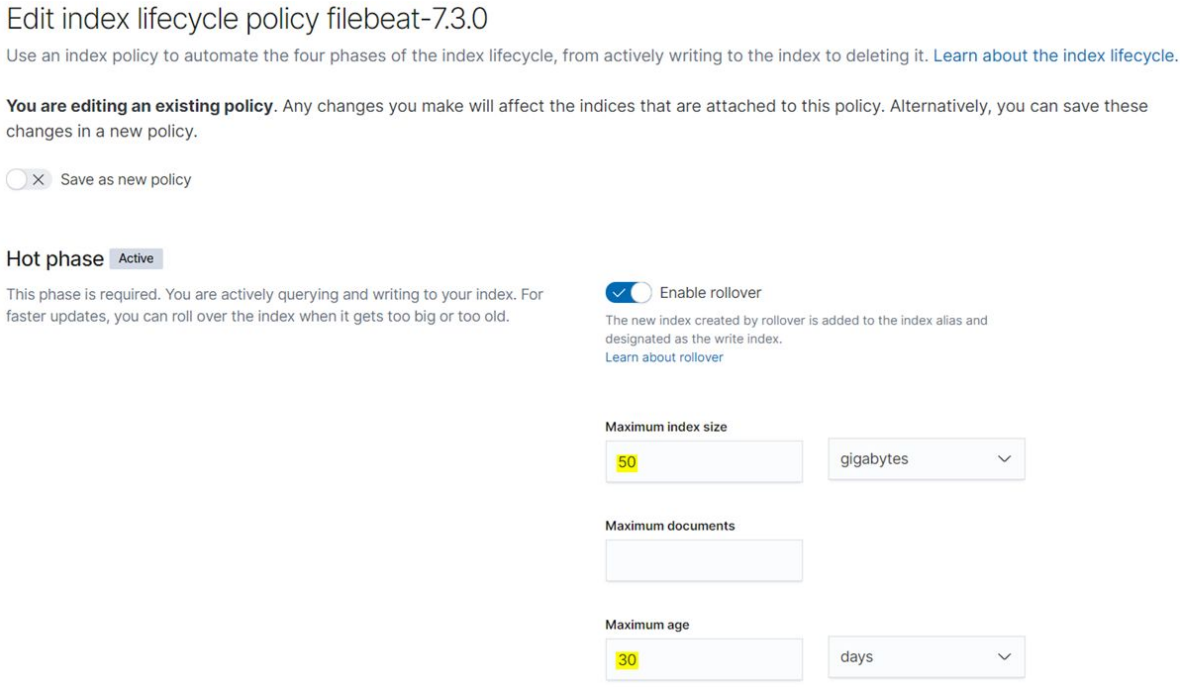
Depuis la version 6.6 d'Elasticsearch, il est autrement possible de mettre en place des politiques de cycles de vie d'index (**ILM** : Index Lifecycle Management).

Tout se gère directement depuis Kibana, au travers du menu « **Management > Index Lifecycle Management** » :



CYCLE DE VIE DE LA DONNÉE

En cliquant sur une politique, on découvre qu'il est possible de limiter la taille d'un index à une certaine taille, ou une certaine rétention programmable directement :



CYCLE DE VIE DE LA DONNÉE

L'interface propose également d'activer le **rollover**.

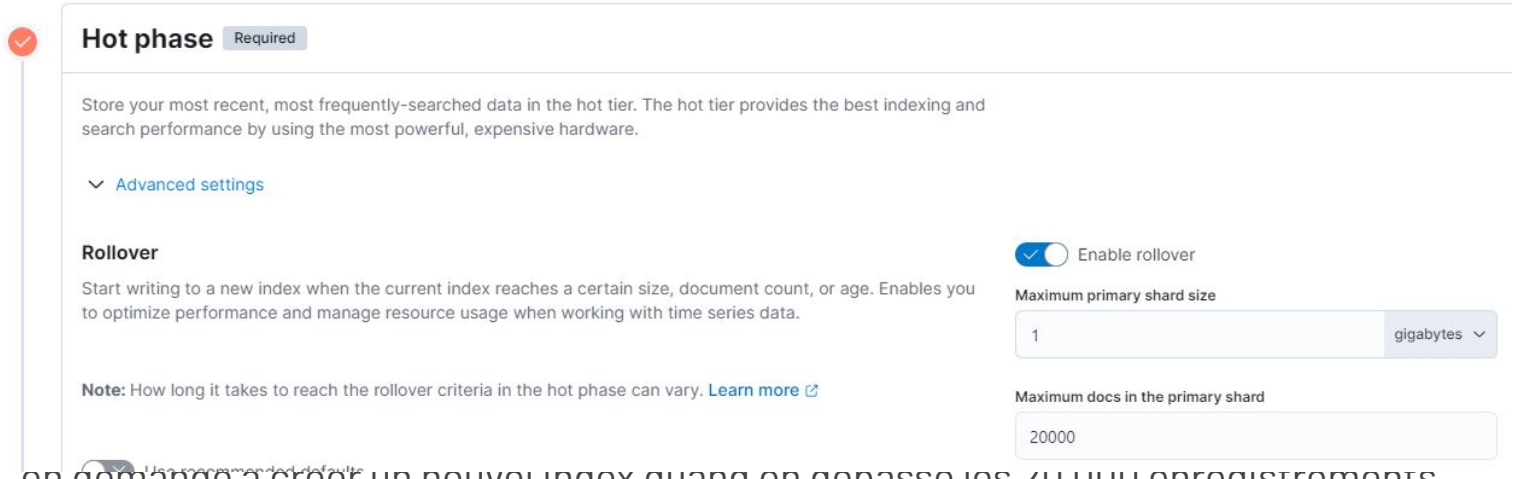
Cette fonctionnalité permet de créer un nouvel index et la donnée y sera poussée lorsque certaines conditions seront respectées (taille, ancienneté ou nombre de documents)

```
POST /alias1/_rollover/my-index-000002
{
  "conditions": {
    "max_age":   "7d",
    "max_docs":  1000,
    "max_size": "5gb"
  }
}
```

Cela génère donc un flux de données entre index, de manière automatisée.

CYCLE DE VIE DE LA DONNÉE

Exemple de rollover :

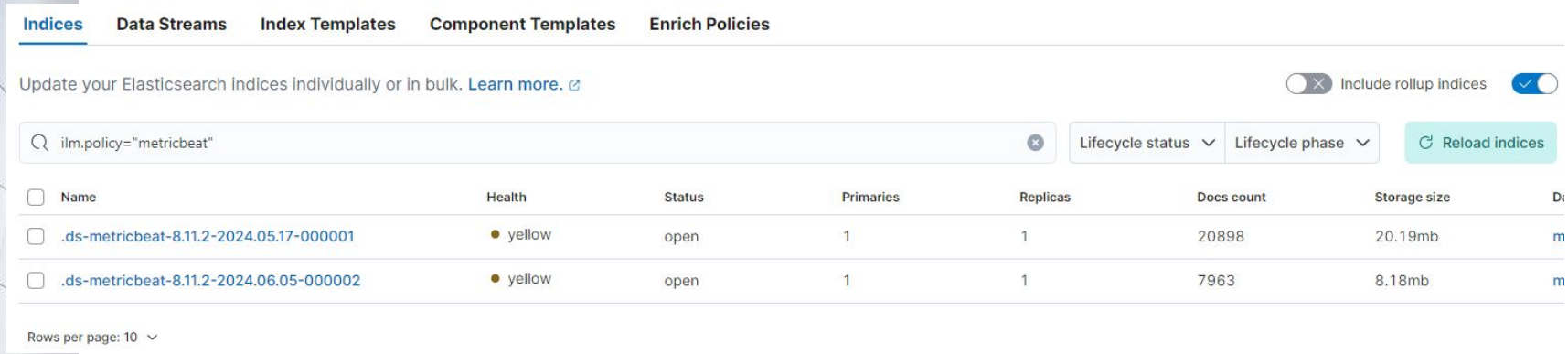


Ici, on demande à créer un nouvel index quand on dépasse les 20 000 enregistrements.

CYCLE DE VIE DE LA DONNÉE

Exemple de rollover :

Le résultat est le suivant : un nouvel index 000002 apparaît lorsque le premier atteint les 20 000 documents (exemple illustratif à ne pas reproduire en production) :



CYCLE DE VIE DE LA DONNÉE



Il est également possible de jouer sur les phases d'un index :

- **Hot phase (Obligatoire)** : Les données arrivent au tout début dans cette phase et pourront alors stockées sur des disques à faible latence (SSD), afin d'y accéder le plus rapidement possible. Ici, les données sont à la fois en lecture et en écriture
- **Warm phase :** Ici seront stockées les données auxquelles on accède de manière plus irrégulière. Il est alors possible de stocker ces données sur des supports à bas coûts (Disques). Les données qui résident dans cette phase ne sont plus qu'en lecture seule
- **Cold phase** : Les données stockées ici ne sont quasiment plus consultées. Il est alors possible de réduire le nombre de réplicas
- **Frozen phase (Licence Enterprise)** : Les index sont montés en searchable snapshots depuis un dépôt de Snapshot (S3, Azure, GCS, ...) avec un simple cache local. Très bas coût mais recherches plus lentes
- **Delete phase** : Ces index ne sont plus utilisés et peuvent être supprimés

Les `min_age` se mesurent depuis le rollover et ne s'additionnent pas.

EXEMPLE



1. Index template avec `data_stream`

```
PUT _index_template/formation-apache
{
  "index_patterns": ["formation-apache*"],
  "data_stream": {},
  "template": {
    "settings": {
      "index.lifecycle.name": "policy_30d"
    }
  }
}
```

2. Création du data stream

```
PUT _data_stream/formation-apache-20261106
```

3. Vérification des settings hérités

```
GET formation-apache-20261106/_settings
```

CYCLE DE VIE DE LA DONNÉE



Quelques actions possibles pour optimiser les index :

Forcemerge : fusionner les segments Lucene

- Un shard est composé de **segments**, des petits fichiers créés à chaque refresh. Tant qu'on écrit, Lucene les fusionne en continu.
- Forcemerge les fusionne en un nombre imposé, typiquement max_num_segments: 1. Effets :
  - **moins de disque**, car les documents supprimés ou mis à jour sont réellement purgés ;
  - **recherches plus rapides**, car il y a moins de fichiers à parcourir ;
  - moins de mémoire consommée.
- C'est une opération **coûteuse en I/O**, à réserver aux index en lecture seule. Sur un index qui reçoit encore des écritures, on recrée des segments et on se retrouve avec un énorme segment qui ne sera plus jamais fusionné.
- L'option `index_codec: "best_compression"` permet de recompresser au passage.

CYCLE DE VIE DE LA DONNÉE



Quelques actions possibles pour optimiser les index (à partir de la zone warm) en lecture seule :

Shrink : réduire le nombre de shards primaires

- En hot, on met parfois plusieurs shards pour **répartir l'écriture** sur plusieurs nœuds (par exemple 3 shards pour 3 nœuds hot).
- Une fois l'index en lecture seule, ce parallélisme ne sert plus à rien. Chaque shard coûte de la mémoire, de l'état de cluster et des threads de recherche.
- Shrink crée un nouvel index avec moins de shards, dans une proportion entière (3 → 1, 4 → 2). ILM bascule ensuite le data stream ou l'alias vers ce nouvel index (préfixe `shrink-`) et supprime l'ancien.
- Contraintes : une copie de tous les shards doit être regroupée sur un même nœud, et l'opération **est inutile si l'index n'a déjà qu'un shard**.

CYCLE DE VIE DE LA DONNÉE

Cycle de vie d'un index de logs — rétention 90 jours

Les `min_age` se mesurent depuis le **rollover** et ne s'additionnent pas : 7 j → 30 j → 90 j

HOT

rollover → 7 jours · SSD

7 j

lecture + écriture

ACTIONS

- **Rollover** dès `50gb` par shard primaire ou `1d`
- Plusieurs shards possibles pour répartir l'écriture
- **Force merge** possible juste après le rollover
- Priorité de récupération : 100

WARM

`min_age: 7d` · disques standards

23 j

lecture seule

ACTIONS

- **Read-only** : plus aucune écriture
- **Shrink** → 1 shard primaire
- **Force merge** → 1 segment, compression
- Migration vers les nœuds `data_warm`

COLD

`min_age: 30d` · stockage bas coût

60 j

lecture rare

ACTIONS

- Migration vers les nœuds `data_cold`
- Réplicas réduits (ou searchable snapshot, licence Enterprise)
- Priorité de récupération : 0

DELETE

`min_age: 90d`

J+90

ACTIONS

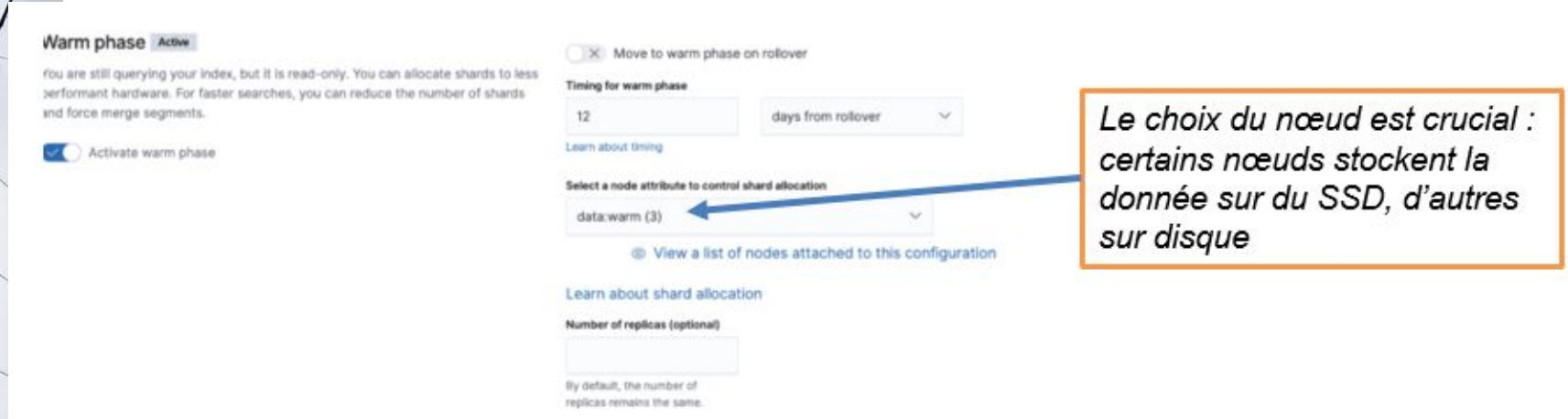
- Suppression de l'index entier
- Option : attendre un snapshot



Échelle proportionnelle. Durée de vie réelle d'un document = temps passé dans l'index d'écriture (≤ 1 j) + 90 j.

CYCLE DE VIE DE LA DONNÉE

En fonction des rôles donnés à des noeuds, il est possible de choisir où stocker la donnée :





Merci

2020, Vincent LE ROUX

Reproduction et diffusion interdits